

TRABAJO FIN DE MÁSTER

Generative AI Models for Functional Data

Máster Universitario en

Ciencia de Datos

Autor: CAMARERO LINARES, Diego

Tutor: SUÁREZ GONZALEZ, Alberto

Departamento de Ingeniería Informática

FECHA: septiembre, 2026

Abstract

In real-world applications, it is common to encounter observations consisting of functions that depend on continuous parameters, such as time or spatial location. Examples of these types of data are temperature curves in meteorology, time series of economic indicators, and electrocardiograms in medicine. Functional data are often costly to collect, can be distorted by measurement noise, or available only in limited quantities. Therefore, it is important to develop methods to generate synthetic samples that preserve the relevant characteristics of actual measurements. Given the remarkable success of generative artificial intelligence in the synthesis of complex data such as images, text, and audio, it is natural to extend and adapt these models to the specific characteristics of functional data.

Specifically, in this work we investigate the effectiveness of standard score-based diffusion and Flow Matching models for generating realistic functional samples that also exhibit diversity. We further consider an extension of the variance-preserving diffusion model in which the matrices that appear in the drift and diffusion terms of the Ornstein-Uhlenbeck stochastic differential equation characterizing the noise injection process can be non-diagonal. This introduces a dependence structure in the noise injected at different locations, thereby providing additional flexibility to the model. Specifically, we consider circulant matrices, which enable both a detailed theoretical analysis and computationally efficient implementation. The proposed extension can be viewed as a latent diffusion model in Fourier space, which makes it possible to control the intensity and rate of noise injection at different frequencies.

An experimental study is conducted to illustrate the ability of the investigated models to generate high-quality synthetic functional data samples using examples from a wide variety of domains. The samples generated with score-based diffusion models exhibit some residual high-frequency noise, a limitation that can probably be ascribed to the stochastic nature of the reverse-diffusion (denoising) process. In the experiments carried out, the additional flexibility of the proposed circulant diffusion framework does not lead to significant improvements over the standard variance preserving diffusion model. For functional data, Flow Matching, which has a simple training procedure, consistently produces the highest-quality samples.

Finally, this work motivates several directions for future research, such as developing latent diffusion models specifically designed for functional data to reduce computational costs and implementing advanced Flow Matching variants to explore whether they can improve the quality of the generated samples.

Contents

1	Introduction	1
1.1	Objectives	2
1.2	Report Organisation	3
2	Related Work	5
2.1	Functional Data Analysis (FDA)	5
2.2	Score-Based Diffusion Models	7
2.3	Flow Matching (FM)	11
3	Diffusion Models	13
3.1	Isotropic Diffusion Models	13
3.2	General Multivariate Ornstein–Uhlenbeck Process	18
3.3	Circulant Ornstein–Uhlenbeck Process	24
3.4	Multivariate Score-Based Diffusion Models	31
4	Flow Matching	35
4.1	Conditional Flow Matching	36
4.2	Interpolation Paths	37
5	Empirical Evaluation	39
5.1	Preliminary Experiment: Real Temperature Curves	41
5.2	Experiment 1: Constant Functions	42
5.3	Experiment 2: Damped Oscillatory Functions	45
5.4	Experiment 3: Frequency-Selective Diffusion	48
5.5	Experiment 4: Skyline Functions	53
5.6	Experiment 5: Real Electrocardiogram Signals	55
6	Conclusions and Future Work	59
6.1	Future Work	60
A	Circulant Matrices	65
B	Complex Distributions	77
C	Closed-Form Solutions for Linear Noise Schedules	93
D	Mathematical Proofs	97
E	Frequency-Selective Noise Injection Schedules	103

CHAPTER 1

Introduction

In recent years, generative artificial intelligence has become one of the most active research fields in machine learning. Generative models are now capable of producing highly realistic images, natural language, audio, and other complex data modalities. The remarkable success of these models has been largely driven by advances in deep learning, beginning with Generative Adversarial Networks (GANs) [1] and, more recently, with diffusion models [2].

Related to content generation, generative models have also become an important tool to address data scarcity. In the deep learning era, the availability of large, diverse, and representative datasets is a key requirement for training modern machine learning models. However, this requirement often presents a significant challenge in real-world applications, where collecting high-quality data is expensive or restricted by privacy or acquisition constraints. In these scenarios, generative models provide a data augmentation framework by synthesising new samples that preserve the properties and structure of the training data.

This paradigm goes beyond classical discriminative modelling. While discriminative models aim to predict the category of the data by estimating the conditional probability

$$p(y \mid \mathbf{x})$$

of a specific label y given the data observation $\mathbf{x}$, generative models seek to characterise and sample from the underlying data distribution directly, without conditioning on a specific label. Therefore, the objective of a generative model is to characterise the underlying probability distribution

$$p_0(\mathbf{x})$$

of a given dataset in order to produce new samples with similar statistical properties. Given a collection of observations sampled according to this unknown density, the goal is to construct a mechanism capable of generating new realisations that are approximately distributed according to $p_0(\mathbf{x})$.

Classical approaches often attempt to model the probability density function $p_0(\mathbf{x})$ directly. However, in high-dimensional settings, estimating and normalising such densities becomes computationally challenging, making explicit likelihood-based methods intractable, a phenomenon commonly referred to as *the curse of dimensionality*. This

limitation motivated alternative generative paradigms, most notably score-based diffusion models and, more recently, Flow Matching, both of which are reviewed in detail in Chapter 2.

Additionally, most successful applications of deep generative models have focused on high-dimensional Euclidean data, such as images or videos, or on sequential text data. Nevertheless, many datasets arising in scientific and engineering applications consist of observations of continuous processes recorded over a domain, typically time or space. Examples include electrocardiographic recordings, meteorological measurements, financial time series, and other signals obtained from physical or biological processes. Such observations are formally studied within the framework of *Functional Data Analysis (FDA)*, as they are naturally regarded as discretised realisations of underlying continuous functions [3].

The success of modern generative models in standard domains has motivated their extension to these infinite-dimensional spaces. This work contributes to this emerging area by developing and evaluating both diffusion-based and Flow Matching generative models explicitly adapted to functional observations.

1.1. Objectives

The main objective of this work is to develop a comprehensive theoretical and empirical framework for generative artificial intelligence models tailored to functional data, formulated explicitly through Flow Matching and diffusion-based models. To achieve this, the work is divided into two distinct core methodological objectives:

- **Flow Matching for Functional Data:** To adapt the continuous-time Flow Matching framework for functional observations, providing an alternative to traditional generative paradigms and evaluating its empirical synthesis capabilities.
- **Diffusion Models for Functional Data:** To formulate the Variance Preserving version of diffusion models and evaluate its performance in generating new samples of functional data.

Regarding the diffusion objective specifically, this work focuses on the development of a novel diffusion framework that incorporates the intrinsic global structure of functional observations through a specific diffusion process based on non-diagonal matrices. To achieve this, circulant matrices are introduced to model structured noise covariance, whose spectral diagonalisation in the Fourier basis naturally decouples the diffusion dynamics in the frequency domain [4]. Within this circulant diffusion framework, two specific sub-objectives are pursued:

- **Theoretical Frequency Analysis:** To establish a detailed theoretical framework for the decoupled diffusion system in the Fourier domain, investigating how diffusion in the frequency domain can analytically control the generation of functional data by prioritising low- or high-frequency components.

- **Implementation of the circulant diffusion model:** To implement and evaluate the circulant diffusion model using structured covariance matrices, leveraging the circulant structure and the properties developed in the theoretical section.

Finally, this work aims to establish a unified empirical benchmark to evaluate and compare the performance of both the Flow Matching and the diffusion models on functional data. This includes a qualitative experiment conducted on real-world datasets to demonstrate the practical potential of these models, alongside quantitative experiments on synthetic datasets to evaluate the quality of the generated functional samples.

1.2. Report Organisation

The remainder of this work is organised as follows.

Chapter 2 reviews the literature relevant to this work. It explores the fundamentals of Functional Data Analysis (FDA), score-based diffusion models, and Flow Matching methods, with particular emphasis on their application to functional data.

Chapter 3 presents the theoretical background developed in this work. It first introduces standard diffusion models and then extends them to an Ornstein–Uhlenbeck process with circulant drift and diffusion matrices. Moreover, it studies in detail the use of circulant matrices, highlighting their spectral decomposition in the Fourier domain.

Chapter 4 presents the theoretical foundations of Flow Matching and introduces Conditional Flow Matching, together with its application to functional data through interpolation paths.

Chapter 5 validates the proposed theoretical framework through one preliminary study and five different experiments, providing both qualitative and quantitative evaluations on synthetic and real-world datasets. The implementation developed throughout this work is publicly available at: <https://github.com/diegocamarerouam/Generative-AI-Models-for-Functional-Data>.

Chapter 6 synthesises the main findings of this work, analyses its limitations, and proposes new directions for future research.

The appendices provide complementary material, including mathematical derivations and proofs, and an extended background on circulant matrices and complex distributions. Moreover, to support the empirical evaluation, they include closed-form solutions for the proposed diffusion processes and detail the frequency-selective noise injection schedules.

CHAPTER 2

Related Work

This chapter provides a comprehensive review of the literature on generative artificial intelligence models for functional data to motivate the methodologies developed in this work. First, a foundational overview of Functional Data Analysis (FDA) is presented. This is followed by an in-depth examination of score-based diffusion models. Finally, the literature review concludes with an introduction to the paradigm of Flow Matching models.

2.1. Functional Data Analysis (FDA)

Functional Data Analysis (FDA) is the branch of statistics concerned with data represented by functions, typically depending on a continuous parameter. These functions map points from a subset of a metric space $\mathcal{T} \subset \mathbb{R}^p$ to another metric space $\mathbb{R}^q$, where $p, q \in \mathbb{N}$. In contrast to traditional multivariate statistics, where observations are represented as a dataset $\{x_1, \dots, x_N\}$ consisting of N realisations of random vectors in a finite-dimensional space $\mathbb{R}^d$, FDA considers observations as realisations of random functions belonging to a function space, usually denoted by $\mathcal{X}$.

A continuous function cannot be represented explicitly by storing its value at every point of its domain. To obtain a finite representation, one approach is to consider basis-function expansions, such as Fourier series or splines, while another perspective is to consider a discretisation framework. This work addresses the discretisation scenario.

Let $x \in \mathcal{X}$ be a function defined over a continuous domain $\mathcal{T} \subset \mathbb{R}^p$. Because $\mathcal{T}$ contains infinitely many points, evaluating x everywhere is computationally intractable. Therefore, a finite grid of M observation points $\mathbf{t} = (t_1, \dots, t_M)$ is defined, where the function x is evaluated. Consequently, instead of observing the function over the entire continuous domain, the available data is restricted to the function's values at these discrete points, denoted as:

$$x(\mathbf{t}) = (x(t_1), \dots, x(t_M)).$$

A dataset of functional observations $\{x_1, \dots, x_N\} \subset \mathcal{X}$ is thus numerically represented by a finite set of evaluations over a shared, predefined grid:

$$\{x_1(\mathbf{t}), \dots, x_N(\mathbf{t})\}, \quad \text{where } \mathbf{t} = (t_1, \dots, t_M) \in \mathcal{T}^M.$$

This work focuses exclusively on scalar functions depending on a continuous, unidimensional parameter ($p = q = 1$). Under this assumption, the corresponding functional dataset can be represented as a matrix in $\mathbb{R}^{N \times M}$, where each row corresponds to a unique sample function evaluated over the discretisation grid.

This notation allows each discretised functional sample $x_i(\mathbf{t})$ to be treated as an M -dimensional vector $\mathbf{x}_i$. This representation is conceptually analogous to image data; for instance, a greyscale image is defined by a two-dimensional grid of pixel intensities. Thus, functional data on a uniform grid can be interpreted as a one-dimensional image. This representation allows discretised functional data to be treated as one-dimensional signals, providing a natural connection with generative models for structured data. This idea has already been explored in recent literature; for example, applying diffusion models to time-series forecasting [5]. Another particularly relevant application arises in the medical domain, where functional data naturally emerge from the continuous monitoring of physiological signals, such as electrocardiograms (ECGs) [6].

Figure 2.1 presents an example of functional data, illustrating the evolution of daily temperatures recorded at 73 weather stations across Spain between 1980 and 2009. The dataset was originally obtained from the Agencia Estatal de Meteorología (AEMET) [7] and was accessed through the `fetch_aemet` function provided by the `scikit-fda` Python library [8].

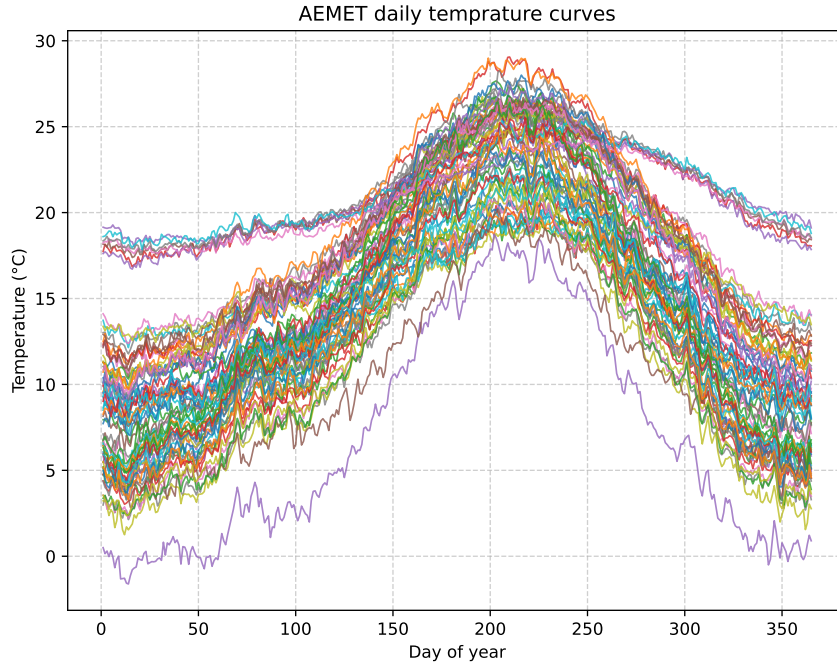


Figure 2.1: Daily temperature curves from the AEMET dataset [7, 8].

2.2. Score-Based Diffusion Models

Diffusion models play an important role in generative modelling across multiple data modalities, especially for image generation. They represent a family of generative architectures that generate synthetic samples by progressively denoising a sample drawn from a simple prior distribution, typically a Gaussian distribution. Rather than modelling the data distribution $p_0(\mathbf{x})$ explicitly, they introduce a forward stochastic process that progressively transforms it into a simpler reference distribution, generating a family of intermediate noised distributions

$$\{p(\mathbf{x}, \tau)\}_{\tau \in [0, T]},$$

and learn to reverse this process to generate new samples. A fundamental difficulty arises from the fact that these intermediate densities are generally unknown and difficult to compute explicitly. Score-based generative modelling provides a solution to this difficulty by instead estimating the logarithmic gradient of these intermediate densities, known as the *score function*. This approach is reviewed in the following subsection.

2.2.1. Foundations of score-based diffusion models

Several approaches to generative modelling via diffusion have been explored over the years. In 2019, Song & Ermon [9] proposed a score-based generative modelling framework based on learning the score-function, defined as the gradient of the log-density $\nabla_{\mathbf{x}} \log p(\mathbf{x})$, and generating new samples through Langevin dynamics. However, a key limitation was that the score near low density regions is hard to learn. As an alternative, in 2020, Ho et al. [2] formulated Denoising Diffusion Probabilistic models (DDPM) as discrete-time Markov chains, deriving a variational bound objective for learning the reverse denoising process to generate synthetic samples. In 2021, both perspectives were formally unified by Song et al. [10], where the diffusion process is generalised as a continuous time-SDE, demonstrating that score-based models and diffusion probabilistic models can be interpreted within a common continuous time framework based on reverse-time stochastic differential equations. In this paradigm, the forward diffusion process systematically injects noise into an original data sample. Let $\mathbf{x}(\tau)$ denote the noisy data sample at diffusion time $\tau \in [0, T]$. The forward process is governed by the following general Itô SDE:

$$d\mathbf{x}(\tau) = f(\mathbf{x}(\tau), \tau) d\tau + g(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(0) = \mathbf{x}_0 \sim p_0(\mathbf{x})$$

where $f(\mathbf{x}(\tau), \tau)$ is the scalar drift coefficient dictating the deterministic decay, and $g(\tau)$ denotes the scalar diffusion coefficient controlling the magnitude of the stochastic perturbation driven by the Wiener process $\mathbf{W}(\tau)$. The original data distribution is represented by $p_0(\mathbf{x})$.

The reverse-time formulation was originally derived by Anderson [11] in the context of stochastic processes. Let $p_\tau(\mathbf{x})$ denote the probability density of $\mathbf{x}(\tau)$. The backward diffusion process is governed by the following SDE:

$$d\overleftarrow{\mathbf{x}}(\tau) = (f(\overleftarrow{\mathbf{x}}(\tau), \tau) - g^2(\tau) \nabla_{\overleftarrow{\mathbf{x}}} \log p_\tau(\overleftarrow{\mathbf{x}})) d\tau + g(\tau) d\overleftarrow{\mathbf{W}}(\tau), \quad \overleftarrow{\mathbf{x}}(T) \sim p_T(\mathbf{x}),$$

where $\overleftarrow{\mathbf{W}}(\tau)$ denotes a reverse-time Wiener process moving from T down to 0, and the noise distribution is denoted as $p_T(\mathbf{x})$. The term $\nabla_{\mathbf{x}} \log p_{\tau}(\mathbf{x})$ is the time-dependent score function and it provides local geometric information about the distribution while avoiding the explicit computation of normalisation constants. Geometrically, the score function indicates the direction of maximal increase of the probability density. During the reverse diffusion process, this information is used to guide noisy samples back toward the high-density regions of the original data distribution. This strategy is considerably more tractable than classical approaches that attempt to explicitly learn the intermediate noised distributions $p(\mathbf{x}, \tau)$ because, for certain typical distributions, the score function $\nabla_x \log p(x, \tau)$ has a closed-form expression that makes the learning procedure easier. The Gaussian formulation is considered in Theorem D.4. Both forward and diffusion processes are illustrated in Figure 2.2.

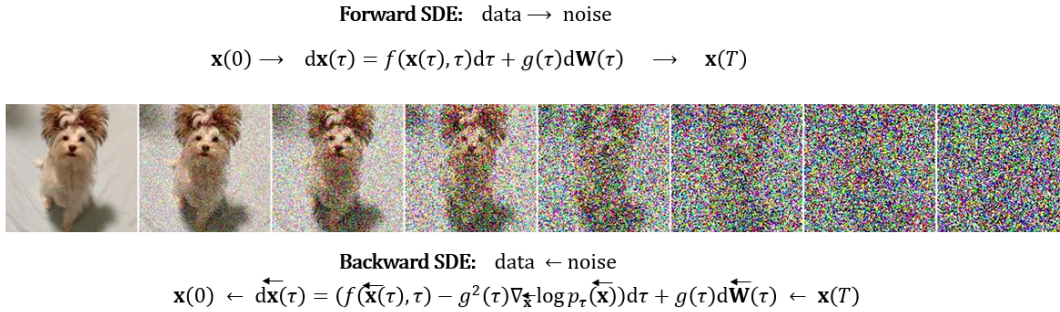


Figure 2.2: Forward and reverse processes in score-based diffusion models [10].

Since the true score function is generally unknown, Song et al. [10] leveraged this result for generative modelling by parameterising the score function with a neural network $\mathbf{s}_{\theta}(\mathbf{x}, \tau)$, which enables the numerical simulation of the reverse-time diffusion process and the generation of new samples. The training objective is given by the denoising score matching loss, defined as

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{U}(0, T)} \left[\gamma(\tau) \mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}, \tau | \mathbf{x}_0, 0)} \left[\|\mathbf{s}(\mathbf{x}, \tau; \theta) - \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0)\|_2^2 \right] \right] \right],$$

where $\gamma(\tau)$ is a positive scalar weighting function that controls the relative importance of different diffusion times by balancing the contribution of the score-matching error across time. Note that the score function appearing in the reverse-time SDE is the marginal score

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau),$$

while the denoising score matching objective is formulated using the conditional score function

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0).$$

At first sight, this may seem inconsistent. Nevertheless, the neural network only receives the perturbed sample $\mathbf{x}$ and the diffusion time τ as inputs, and therefore it cannot represent a function that depends explicitly on the unknown initial condition $\mathbf{x}_0$. As a consequence, the optimal predictor $\mathbf{s}_{\theta}(\mathbf{x}, \tau)$ under the mean squared error

loss is given by the conditional expectation

$$\mathbf{s}_\theta(\mathbf{x}, \tau) \approx \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_\tau} \left[\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0) \mid \mathbf{x}_\tau \right].$$

Using the following identity (see Theorem D.2)

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) = \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_\tau} \left[\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0) \mid \mathbf{x}_\tau \right],$$

which is commonly referred to as the denoising score matching identity. It follows that even though the training objective uses the conditional score function as the target, the neural network learns the marginal score function required in the reverse diffusion process:

$$\mathbf{s}_\theta(\mathbf{x}, \tau) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau).$$

Once this approximation is learnt, it can be used to define the reverse-time diffusion process, which allows for the generation of new synthetic data. The backward diffusion process is replaced by

$$d\overleftarrow{\mathbf{x}}(\tau) = (f(\overleftarrow{\mathbf{x}}(\tau), \tau) - g^2(\tau) \mathbf{s}_\theta(\overleftarrow{\mathbf{x}}(\tau))) d\tau + g(\tau) d\overleftarrow{\mathbf{W}}(\tau), \quad \overleftarrow{\mathbf{x}}(T) = \overleftarrow{\mathbf{x}}_T \sim p_T(\mathbf{x}).$$

The generated sample $\overleftarrow{\mathbf{x}}(0)$ will present statistical properties similar to the original data distribution $p_0(\mathbf{x})$.

2.2.2. From isotropic to structured noise injection

The foundational frameworks proposed by Song & Ermon [9] and Ho et al. [2] share a central paradigm: they inject noise independently into each component of the data vector. However, they achieve this through fundamentally distinct stochastic trajectories.

The original score-based formulation of Song and Ermon can be interpreted, within the continuous-time framework of Song et al. [10], as a Variance Exploding (VE) diffusion process governed by the SDE:

$$d\mathbf{x}(\tau) = \sigma d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\sigma \in \mathbb{R}^+$ represents the diffusion coefficient, and $\mathbf{W}(\tau)$ is a standard Wiener process in $\mathbb{R}^D$. This process corresponds to a multidimensional Brownian motion, whose distribution at time τ is Gaussian, with the following parameters:

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D(\mathbf{x}_0, \sigma^2 (\tau - t_0) I_D)}.$$

Note that the terminology *Variance Exploding* arises from the fact that the covariance matrix grows linearly and unboundedly in time.

Conversely, the continuous generalisation of the discrete Markov chain defined by Ho et al. [2] yields the Variance Preserving (VP) SDE framework. The time-independent version can be formulated as

$$d\mathbf{x}(\tau) = -\lambda \mathbf{x}(\tau) d\tau + \sqrt{2\lambda} d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\lambda \geq 0$. Observe the negative sign on the drift term. The analytical solution to this Ornstein–Uhlenbeck SDE follows the Gaussian distribution:

$$\mathbf{x}(\tau) \sim \mathcal{N}_D \left(e^{-\lambda(\tau-t_0)} \mathbf{x}_0, \left(1 - e^{-2\lambda(\tau-t_0)} \right) I_D \right).$$

As synthesised by Song et al. [10], while VE and VP diffusion processes construct entirely different variance schedules, both formulations rely strictly on *isotropic* Gaussian perturbations. This means noise is injected independently and with the same variance along every component, as dictated by the identity matrix. While mathematically convenient, this assumption does not take into account the dependence between nearby points that is often present in functional data.

Recent diffusion models have introduced structure into the noise injection process to mitigate the limitations of isotropic models and preserve spatial or structural geometric properties. In structured data, such as images or functional observations, nearby components can be strongly related and jointly determine the overall structure.

Early physically inspired attempts moved away from pure white noise by introducing a deterministic degradation operator together with a small blur noise, as seen in heat dissipation or blurring diffusion mechanisms [12, 13]. These methods progressively degrade the image according to its physical structure rather than injecting independent noise into each pixel. Moreover, they leave only fine-grained noise injection for the final stages of the process.

Another approach is to learn the noise structure during the diffusion process rather than specifying it in advance. Sahoo et al. [14] use Bayesian inference to learn a covariance structure for the injected noise. This introduces multivariate learnt noise schedules with a covariance structure, so the covariance is not the identity; rather, it is learnt from the input.

Another line of research focuses on the design of the injected noise itself. Moving beyond standard white noise, the integration of *blue noise* concepts [15] into diffusion models allows for a more uniform spatial distribution of randomness. This method balances low- and high-frequency synthesis, distancing itself further from standard isotropic formulations.

Recent work has advanced beyond fixed isotropic covariance structures toward diffusion processes with more general covariance structures. For example, Yu et al. [16] introduce a diffusion framework where the covariance structure varies spatially across the image domain, allowing noise levels to adapt to local geometric and semantic features. By coupling adaptive covariance modelling with structural consistency constraints, the method improves the preservation of edges and spatial details during generation and editing. However, it is computationally costly. An alternative direction consists of incorporating physically meaningful degradation mechanisms into the diffusion process. Shu et al. [17] establish a stochastic framework connecting physically based rendering and diffusion models. This formulation provides finer control over material appearance, illumination, and shading characteristics, extending diffusion models beyond purely statistical image synthesis, but it remains highly specialised.

However, methods that use general covariance structures can become computationally expensive. Some authors have addressed this problem by considering circulant matrices, as they allow diagonalisation in the frequency domain [4]. These properties make circulant matrices particularly useful for controlling noise at different frequencies. Related spectral approaches have also shown that frequency-dependent noise injection can influence the structure of the generated data. For instance, Fourier Diffusion Models [18] introduce score-based stochastic mechanisms designed to manipulate the modulation transfer function (MTF) in image generation and report better results than isotropic models. [19] also shows that implementing noise injection via frequency-diagonal covariance allows networks to learn baseline structures before high-level details.

The use of circulant matrices to represent structured covariance matrices has also been considered in the FDA framework. Dietrich & Newsam [20], for example, proposed using the circulant embedding of covariance matrices to achieve fast and exact simulation of stationary Gaussian processes.

This work focuses on applying structured noise injection to diffusion models for functional data. The aim is to take into account the underlying continuity and smoothness of functional observations while benefiting from the advantages of circulant matrices. The resulting circulant diffusion framework is developed in Chapter 3.

2.3. Flow Matching (FM)

Among the various generative modelling approaches, *Flow Matching* (FM), introduced by Lipman et al. [21], stands out because it combines conceptual simplicity with strong empirical performance. Originally evaluated on image datasets, Flow Matching has been demonstrated to offer more stable training dynamics, faster convergence, and superior sample quality compared to traditional diffusion processes.

Both Flow Matching and diffusion models share the same foundational objective: to generate samples from a target data distribution starting from a simple source distribution. However, their underlying mechanisms differ fundamentally. While diffusion models are characterised by injecting noise to the original data and then reversing the process to generate synthetic data, Flow Matching formulates a continuous, deterministic transformation that maps samples from a simple source distribution to samples that approximate the target data distribution. Rather than simulating two coupled stochastic processes (forward and backward), Flow Matching defines a unique, deterministic trajectory without requiring an SDE formulation.

While the foundational Flow Matching framework is formulated for data represented as vectors in Euclidean space, its application to Functional Data Analysis represents a novel and powerful paradigm shift. In particular, Functional Flow Matching [22] generalises this formulation to distributions over stochastic processes, treating functional observations as elements of a function space.

A deeper analysis of the Flow Matching model is provided in Chapter 4.

CHAPTER 3

Diffusion Models

This work offers a concrete approach to generative artificial intelligence modelling for functional data, utilising the framework of score-based diffusion models. Diffusion models establish a stochastic differential equation (SDE) framework in which a forward diffusion process systematically transforms the original data into a tractable noise distribution. A learnt reverse process then solves a corresponding denoising SDE, leveraging the estimated score-function to generate synthetic functional data samples from noise.

This chapter develops a rigorous mathematical treatment of score-based diffusion models, especially the Ornstein–Uhlenbeck process as the forward diffusion model adopted in this work, extending the isotropic framework to the multivariate setting required for structured functional data. Finally, it develops the circulant Ornstein–Uhlenbeck process and the corresponding training framework.

3.1. Isotropic Diffusion Models

The VE and VP processes were introduced in the literature review in Chapter 2 as established results. This section derives them rigorously and extends both formulations to the time-dependent and general Ornstein–Uhlenbeck framework.

While Variance Exploding models inject noise increasingly to obtain a noisy sample of the original data, Variance Preserving models are constructed so that the process converges toward a stable stationary distribution with bounded variance. Both formulations are represented through stochastic differential equations whose drift and diffusion operators are proportional to the identity matrix. This yields isotropic dynamics, where the components evolve independently and have the same variance.

3.1.1. Variance Exploding (VE)

Mathematically, a diffusion process is continuous in time and is modelled via a stochastic differential equation (SDE). Let $[t_0, T]$ denote a time interval. The simplest diffusion model is the **Variance Exploding (VE)**, which is represented by the following

stochastic differential equation defined on $[t_0, T]$:

$$d\mathbf{x}(\tau) = \sigma d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where σ is a real positive number that represents the diffusion coefficient, and $\mathbf{W}(\tau)$ is a standard Wiener process in $\mathbb{R}^D$. The analytical solution to the previous SDE is expressed in integral form as

$$\mathbf{x}(\tau) = \mathbf{x}_0 + \sigma \int_{t_0}^{\tau} d\mathbf{W}(u) = \mathbf{x}_0 + \sigma (\mathbf{W}(\tau) - \mathbf{W}(t_0)).$$

Using the independent increments property of the Wiener process

$$\mathbf{W}(\tau) - \mathbf{W}(t_0) \sim \mathcal{N}_D(\mathbf{0}, (\tau - t_0) I_D),$$

where the second term is the covariance matrix, yields

$$\mathbf{x}(\tau) = \mathbf{x}_0 + \sigma \sqrt{\tau - t_0} \mathbf{z},$$

where $\mathbf{z} \sim \mathcal{N}_D(\mathbf{0}, I_D)$. This process corresponds to a multidimensional Brownian motion, whose marginal distribution at time τ is Gaussian with the following parameters:

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D(\mathbf{x}_0, \sigma^2 (\tau - t_0) I_D)}.$$

Note that the terminology *Variance Exploding* arises from the fact that the covariance matrix grows linearly over time. Figure 3.1 illustrates this example.

The next step toward building more flexible diffusion processes is to introduce a non-zero deterministic drift term to the stochastic differential equation:

$$d\mathbf{x}(\tau) = \mu d\tau + \sigma d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\mu \in \mathbb{R}^D$ is the drift coefficient, while $\sigma \geq 0$ remains the diffusion coefficient. Integrating both sides and solving as in the Brownian motion SDE yields the solution for the commonly known *arithmetic Brownian motion*:

$$\mathbf{x}(\tau) = \mathbf{x}_0 + \mu (\tau - t_0) + \sigma \sqrt{\tau - t_0} \mathbf{z},$$

where $\mathbf{z} \sim \mathcal{N}_D(\mathbf{0}, I_D)$. Because it is a linear transformation of a Gaussian process, the marginal distribution of $\mathbf{x}(\tau)$ remains Gaussian:

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D(\mathbf{x}_0 + \mu (\tau - t_0), \sigma^2 (\tau - t_0) I_D)}.$$

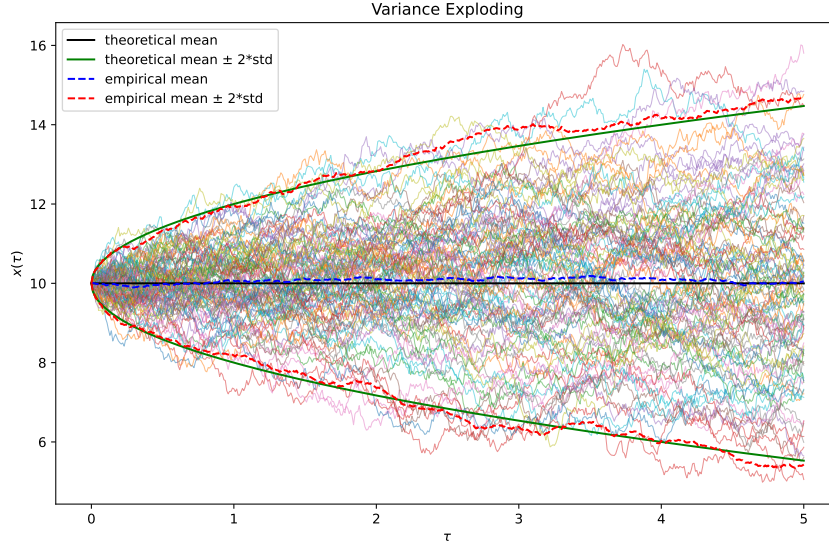


Figure 3.1: Simulation of 100 trajectories from the VE SDE with $\mu = 0$ and $\sigma = 1$.

3.1.2. Variance Preserving (VP)

While arithmetic Brownian motion introduces a deterministic trend through the drift coefficient, it generates structural limitations for generative modelling. Specifically, its mean and variance grow unbounded as time increases. For score-based diffusion models, a process that transforms the data to a stable distribution, usually a known stationary distribution, is required. For example, a standard model in this setting is the **Variance Preserving (VP)** model, where the stationary distribution is the standard Gaussian distribution.

Consider the following Ornstein–Uhlenbeck (OU) process:

$$d\mathbf{x}(\tau) = -\lambda \mathbf{x}(\tau) d\tau + \sqrt{2\lambda} d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\lambda \geq 0$, and $\mathbf{W}(\tau) \in \mathbb{R}^D$ is a multidimensional Wiener process. The negative sign in the drift term moves the process towards zero: positive components have a negative drift, while negative components have a positive drift. Consequently, the mean decays towards zero over time. To analytically solve this SDE, an integrating factor is introduced. Consider the function

$$\phi(\tau) = e^{\lambda(\tau-t_0)}, \quad \phi(t_0) = 1,$$

whose time derivative is given by

$$\frac{d\phi(\tau)}{d\tau} = \lambda \phi(\tau)$$

Using the integrating factor, gathering terms, and applying Itô's product rule (see [23, Theorem 4.1.2]) to the product $\phi(\tau)\mathbf{x}(\tau)$, it is obtained that

$$\begin{aligned}\phi(\tau) d\mathbf{x}(\tau) + \lambda \phi(\tau) \mathbf{x}(\tau) d\tau &= \sqrt{2\lambda} \phi(\tau) d\mathbf{W}(\tau) \\ d(\phi(\tau) \mathbf{x}(\tau)) &= \sqrt{2\lambda} \phi(\tau) d\mathbf{W}(\tau)\end{aligned}$$

Integrating both sides over the interval $[t_0, \tau]$ results in:

$$\phi(\tau) \mathbf{x}(\tau) = \mathbf{x}_0 + \sqrt{2\lambda} \int_{t_0}^{\tau} \phi(u) d\mathbf{W}(u),$$

and inverting the integrating factor

$$\mathbf{x}(\tau) = e^{-\lambda(\tau-t_0)} \mathbf{x}_0 + \sqrt{2\lambda} \int_{t_0}^{\tau} e^{-\lambda(\tau-u)} d\mathbf{W}(u).$$

Applying the multivariate version of Itô's lemma for stochastic integrals (see [23, Corollary 3.1.7 and Theorem 3.2.1]),

$$\mathbb{E} \left[\int_a^b g(u) d\mathbf{W}(u) \right] = 0, \quad \text{Cov} \left(\int_a^b g(u) d\mathbf{W}(u) \right) = \int_a^b g(u)^2 du I_D$$

the solution follows a Gaussian distribution, like

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D \left(e^{-\lambda(\tau-t_0)} \mathbf{x}_0, \left(1 - e^{-2\lambda(\tau-t_0)} \right) I_D \right)}.$$

Indeed, taking the asymptotic limit $\tau \rightarrow \infty$, and using that $\lambda \geq 0$, the initial condition effect on the mean parameter vanishes, and the obtained stationary distribution is

$$\mathbf{x}(\tau) \xrightarrow{\tau \rightarrow \infty} \mathcal{N}_D(\mathbf{0}, I_D).$$

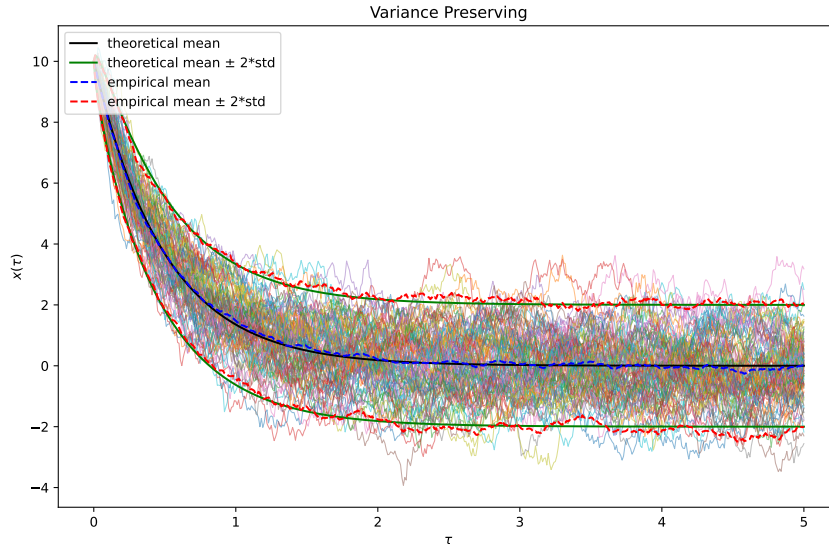


Figure 3.2: Simulation of 100 trajectories from the VP SDE with $\lambda = 2.0$.

Nevertheless, even though this process allows for transforming the original data into noise while simultaneously reaching a stable distribution, this might not be sufficient to establish a proper diffusion models framework. The purpose of diffusion models is not only to perturb the original data distribution toward noise but also to control the noise injection process. This is achieved by introducing time-dependence into the drift and diffusion coefficients, allowing the rate of noise injection to be scheduled and controlled throughout the diffusion process. This property is particularly important when approaching the reverse process to generate synthetic samples, since it is necessary to not inject too much noise at the end of the reverse process so that the generated samples retain the desired structure.

Mathematically, the time-dependence condition is introduced in the drift and diffusion coefficients. The Variance Preserving property is applied below to the time-dependent system. Consider the following time-dependent Ornstein–Uhlenbeck (OU) process:

$$d\mathbf{x}(\tau) = -\lambda(\tau) \mathbf{x}(\tau) d\tau + \sqrt{2\lambda(\tau)} d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\lambda(\tau)$ is defined over the time interval $[t_0, T]$ and still satisfies the positive constraint. Again, $\mathbf{W}(\tau) \in \mathbb{R}^D$ is a multidimensional Wiener process.

To obtain the solution, it is sufficient to replicate the time-independent reasoning. Note that, in the time-dependent case, the integrating factor is

$$\phi(\tau) = e^{\int_{t_0}^{\tau} \lambda(u) du}.$$

In this case, the integral form of the solution is

$$\mathbf{x}(\tau) = e^{-\int_{t_0}^{\tau} \lambda(u) du} \mathbf{x}_0 + \int_{t_0}^{\tau} e^{-\int_u^{\tau} \lambda(r) dr} \sqrt{2\lambda(u)} d\mathbf{W}(u),$$

and it follows that

$$\begin{aligned} \mathbb{E}[\mathbf{x}(\tau)] &= e^{-\int_{t_0}^{\tau} \lambda(u) du} \mathbf{x}_0 \\ \text{Cov}(\mathbf{x}(\tau)) &= \int_{t_0}^{\tau} e^{-2\int_u^{\tau} \lambda(r) dr} 2\lambda(u) du I_D \\ &= \left(1 - e^{-2\int_{t_0}^{\tau} \lambda(u) du}\right) I_D \end{aligned}$$

Therefore, the solution is

$$(3.1) \quad \boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D \left(e^{-\int_{t_0}^{\tau} \lambda(u) du} \mathbf{x}_0, \left(1 - e^{-2\int_{t_0}^{\tau} \lambda(u) du}\right) I_D \right)},$$

which satisfies the asymptotic condition

$$\mathbf{x}(\tau) \xrightarrow{\tau \rightarrow \infty} \mathcal{N}_D(\mathbf{0}, I_D).$$

As an illustration, the linear drift schedule defined as

$$\lambda(\tau) = \lambda_{\min} + \frac{\tau - t_0}{T - t_0} (\lambda_{\max} - \lambda_{\min})$$

for any fixed $\lambda_{\max} \geq \lambda_{\min} > 0$ parameters admits an explicit closed-form expression for the solution (3.1), which is derived in Appendix C.1 and illustrated in Figure 3.3.

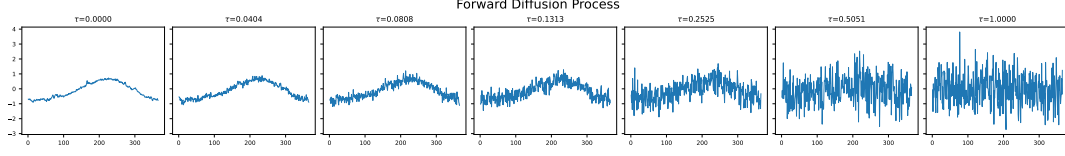


Figure 3.3: Illustration of the forward diffusion process applied to the AEMET dataset.

Finally, instead of forcing the forward process to end at a standard white noise distribution, one may want the stationary distribution to converge to a distribution with a specific covariance matrix. One option is to remove the VP condition by allowing the diffusion coefficient to vary independently from the drift coefficient. This gives a solution whose covariance matrix is proportional to the identity matrix. Consider the Ornstein–Uhlenbeck process:

$$d\mathbf{x}(\tau) = -\lambda(\tau) \mathbf{x}(\tau) d\tau + \sigma(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\lambda(\tau), \sigma(\tau) \geq 0$ for all $\tau \in [t_0, T]$, and $\mathbf{W}(t)$ is a standard Wiener process in $\mathbb{R}^D$. Introducing the same integrating factor as in the VP example, the resulting solution is

$$\mathbf{x}(\tau) = e^{-\int_{t_0}^{\tau} \lambda(u) du} \mathbf{x}_0 + \int_{t_0}^{\tau} e^{-\int_u^{\tau} \lambda(r) dr} \sigma(u) d\mathbf{W}(u).$$

Thus, the derived solution is

$$\mathbf{x}(\tau) \sim \mathcal{N}_D \left(e^{-\int_{t_0}^{\tau} \lambda(u) du} \mathbf{x}_0, \left(\int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda(r) dr} \sigma(u)^2 du \right) I_D \right).$$

In contrast with the VP construction, the asymptotic covariance matrix now depends on the complete time evolution of the coefficients $\lambda(\tau)$ and $\sigma(\tau)$. Consequently, the limiting distribution is not necessarily a standard Gaussian, although independence is still preserved, since the covariance matrix remains proportional to the identity matrix.

3.2. General Multivariate Ornstein–Uhlenbeck Process

In the previous formulations, all coordinates of the solution were governed by the same process and evolved independently with identical diffusion intensity. Nevertheless, while this assumption greatly simplifies the mathematical analysis, it may be insufficient for modelling high-dimensional structured data. In many applications, including Functional Data Analysis, dependencies naturally arise between different coordinates, spatial locations, or temporal observations. To capture these interactions, the drift and diffusion coefficients can be generalised from scalar parameters to full matrices. This extension leads to multivariate Ornstein–Uhlenbeck processes with coupled dynamics.

3.2.1. Time-Independent OU Process

Consider now drift and diffusion matrices that are not necessarily diagonal. Define over the time interval $[t_0, T]$ the following SDE:

$$d\mathbf{x}(\tau) = -\mathbf{K} \mathbf{x}(\tau) d\tau + \mathbf{S} d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\mathbf{K}$ is the drift matrix and $\mathbf{S}$ is the diffusion matrix. Assume that all eigenvalues of $\mathbf{K}$ have a strictly positive real part, ensuring the asymptotic stability of the process.

Solution Derivation

To solve this SDE, similarly to the diagonal case in Section 3.1, define the matrix integrating factor

$$\Phi(\tau) = e^{(\tau-t_0)\mathbf{K}}, \quad \Phi(t_0) = I_D.$$

The time derivative of this matrix exponential satisfies

$$\frac{d\Phi(\tau)}{d\tau} = \mathbf{K} \Phi(\tau).$$

Applying Itô's formula to the matrix-vector product (see [23, Section 4.2]) yields:

$$\begin{aligned} \Phi(\tau) d\mathbf{x}(\tau) + d\Phi(\tau) \mathbf{x}(\tau) &= \Phi(\tau) \mathbf{S} d\mathbf{W}(\tau) \\ d(\Phi(\tau) \mathbf{x}(\tau)) &= \Phi(\tau) \mathbf{S} d\mathbf{W}(\tau). \end{aligned}$$

Note that the cancellation relies on the property that $\mathbf{K}$ commutes with its own exponential matrix, i.e., $\mathbf{K} e^{(\tau-t_0)\mathbf{K}} = e^{(\tau-t_0)\mathbf{K}} \mathbf{K}$. Integrating both sides from t_0 to τ gives:

$$\Phi(\tau) \mathbf{x}(\tau) = \mathbf{x}_0 + \int_{t_0}^{\tau} \Phi(u) \mathbf{S} d\mathbf{W}(u).$$

Using the inverse matrix $\Phi(\tau)^{-1} = e^{-(\tau-t_0)\mathbf{K}}$, it follows that

$$(3.2) \quad \mathbf{x}(\tau) = e^{-(\tau-t_0)\mathbf{K}} \mathbf{x}_0 + \int_{t_0}^{\tau} e^{-(\tau-u)\mathbf{K}} \mathbf{S} d\mathbf{W}(u).$$

The Gaussian distribution parameters are derived using Itô's rules for stochastic integrals (see Itô isometry [23, Corollary 3.1.7]). The mean vector is

$$\mu_{\mathbf{x}}(\tau) = \mathbb{E}[\mathbf{x}(\tau)] = e^{-(\tau-t_0)\mathbf{K}} \mathbf{x}_0.$$

By applying the multivariate Itô isometry, the covariance matrix is represented by the integral:

$$(3.3) \quad \Sigma_{\mathbf{x}}(\tau) = \text{Cov}(\mathbf{x}(\tau)) = \int_{t_0}^{\tau} e^{-(\tau-u)\mathbf{K}} \mathbf{S} \mathbf{S}^{\top} e^{-(\tau-u)\mathbf{K}^{\top}} du.$$

Unlike the scalar time-independent case, if $\mathbf{K}$ and $\mathbf{K}^\top$ do not commute, Eq. (3.3) cannot be simplified into an algebraic fraction using standard matrix inversion. Instead, from the time-independent version of Theorem D.1, it follows that $\Sigma_{\mathbf{x}}(\tau)$ is the unique solution to the following differential Lyapunov equation:

$$\frac{d\Sigma_{\mathbf{x}}(\tau)}{d\tau} = -\mathbf{K}\Sigma_{\mathbf{x}}(\tau) - \Sigma_{\mathbf{x}}(\tau)\mathbf{K}^\top + \mathbf{S}\mathbf{S}^\top, \quad \Sigma_{\mathbf{x}}(t_0) = \mathbf{0}.$$

Therefore, the marginal distribution of the process is

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}\left(e^{-(\tau-t_0)\mathbf{K}}\mathbf{x}_0, \Sigma_{\mathbf{x}}(\tau)\right)}.$$

Stationary Distribution

Assume that all eigenvalues of $\mathbf{K}$ have a strictly positive real part, then the effects of the initial condition decay asymptotically. As $\tau \rightarrow \infty$, the mean vector vanishes

$$\lim_{\tau \rightarrow \infty} \mu_{\mathbf{x}}(\tau) = \mathbf{0},$$

and the covariance matrix converges to a constant matrix Σ_∞ :

$$\Sigma_\infty = \lim_{\tau \rightarrow \infty} \Sigma_{\mathbf{x}}(\tau) = \int_{t_0}^{\infty} e^{-u\mathbf{K}}\mathbf{S}\mathbf{S}^\top e^{-u\mathbf{K}^\top} du.$$

By setting $\frac{d\Sigma_{\mathbf{x}}(\tau)}{d\tau} = \mathbf{0}$ in the above mentioned Lyapunov equation, it is evident that the stationary covariance matrix Σ_∞ satisfies the following equation:

$$(3.4) \quad \mathbf{K}\Sigma_\infty + \Sigma_\infty\mathbf{K}^\top = \mathbf{S}\mathbf{S}^\top.$$

Thus, the stationary distribution of the general coupled multivariate OU process is:

$$\mathbf{x}(\tau) \xrightarrow{\tau \rightarrow \infty} \mathcal{N}(\mathbf{0}, \Sigma_\infty).$$

This Lyapunov equation allows to derive the conditions that drift matrix $\mathbf{K}$ and diffusion matrix $\mathbf{S}$ must satisfy to preserve the generalised Variance Preserving property. Consider Σ_∞ to be the covariance matrix of a standard Gaussian distribution, which means that

$$\Sigma_\infty = I_D.$$

Therefore, substituting on (3.4), the sufficient and necessary condition over the drift and diffusion matrices is

$$\mathbf{K} + \mathbf{K}^\top = \mathbf{S}\mathbf{S}^\top.$$

Furthermore, if $\mathbf{K}$ and $\mathbf{S}$ are symmetric matrices, the previous condition turns into $2\mathbf{K} = \mathbf{S}^2$.

3.2.2. Time-Dependent OU Process

In generative modelling, particularly within the framework of score-based diffusion models, constructing a system of constant coefficients limits its capacity to schedule noise effectively. To control the progressive transformation of original data, the system dynamics may vary over time. However, introducing time-dependence to a coupled multivariate system introduces a major mathematical issue: non-commutativity across time.

This section introduces and analyses the Ornstein–Uhlenbeck process in its general time-dependent form, deriving its analytical solution and fundamental statistical properties.

Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent drift and diffusion matrices defined over the time interval $[t_0, T]$. Consider the multivariate stochastic process $\mathbf{x}(\tau) \in \mathbb{R}^D$ governed by the following linear Itô stochastic differential equation (SDE):

$$(3.5) \quad d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\mathbf{W}(\tau)$ is a standard D -dimensional Wiener process, and $\mathbf{x}_0 \in \mathbb{R}^D$ is a deterministic initial condition. The existence and uniqueness of a strong solution to (3.5) are guaranteed under standard conditions (see [23, Theorem 5.2.1]).

Solution Derivation using Time-Ordering Operator

Similarly to the time independent process (see Section 3.2.1), the solution is derived by introducing the matrix integrating factor $\Phi(\tau) \in \mathbb{R}^{D \times D}$ defined as the unique solution to the following right-handed matrix initial value problem:

$$(3.6) \quad \begin{cases} \frac{d}{d\tau} \Phi(\tau) = \Phi(\tau) \mathbf{K}(\tau), & \tau \in [t_0, T], \\ \Phi(t_0) = I_D. \end{cases}$$

In general, matrices $\mathbf{K}(\tau)$ may not commute at different times. This means that

$$\mathbf{K}(\tau_1) \mathbf{K}(\tau_2) \neq \mathbf{K}(\tau_2) \mathbf{K}(\tau_1), \quad \tau_1, \tau_2 \in [t_0, T].$$

Consequently, in general, the fundamental matrix $\Phi(\tau)$ cannot be written as the ordinary matrix exponential of an integral

$$\exp \left(\int_{t_0}^{\tau} \mathbf{K}(s) ds \right).$$

This limitation arises because the derivative of a matrix exponential does not satisfy

$$\frac{d}{d\tau} e^{\mathbf{A}(\tau)} = e^{\mathbf{A}(\tau)} \mathbf{A}'(\tau)$$

unless $\mathbf{A}(\tau)$ commutes with $\mathbf{A}'(\tau)$. Note that this problem has appeared when the time-dependence condition is introduced. Instead, $\Phi(\tau)$ must be represented by the time-ordered exponential

$$\Phi(\tau) = \mathcal{T} \exp \left(\int_{t_0}^{\tau} \mathbf{K}(s) ds \right),$$

where $\mathcal{T}$ denotes the time-ordering operator that arranges matrix products according to the order of time variables. Integrating the right-handed matrix initial value problem (3.6) yields

$$\Phi(\tau) = I_D + \int_{t_0}^{\tau} \Phi(s) \mathbf{K}(s) ds,$$

and substituting $\Phi(s)$ with this expression yields the Dyson-series expansion for $\Phi(\tau)$

$$\begin{aligned} \Phi(\tau) &= I_D + \sum_{n=1}^{\infty} \int_{t_0}^{\tau} \int_{t_0}^{\tau_1} \cdots \int_{t_0}^{\tau_{n-1}} \mathbf{K}(\tau_n) \cdots \mathbf{K}(\tau_1) d\tau_n \cdots d\tau_1 \\ &= I_D + \sum_{n=1}^{\infty} \int_{t_0 < \tau_n < \tau_{n-1} < \cdots < \tau_1 < \tau} \mathbf{K}(\tau_n) \cdots \mathbf{K}(\tau_1) d\tau_n \cdots d\tau_1 \\ &= I_D + \int_{t_0}^{\tau} \mathbf{K}(\tau_1) d\tau_1 + \int_{t_0 < \tau_2 < \tau_1 < \tau} \mathbf{K}(\tau_2) \mathbf{K}(\tau_1) d\tau_2 d\tau_1 + \cdots \end{aligned}$$

If $\mathbf{K}(\tau)$ commutes with itself at all times, this expansion reduces to the standard matrix exponential.

Define the transformed process $\mathbf{y}(\tau) := \Phi(\tau) \mathbf{x}(\tau)$. Applying the multivariate Itô formula (see [23, Section 4.2]) yields:

$$\begin{aligned} d\mathbf{y}(\tau) &= d(\Phi(\tau) \mathbf{x}(\tau)) \\ &= d\Phi(\tau) \mathbf{x}(\tau) d\tau + \Phi(\tau) d\mathbf{x}(\tau) \\ &= \Phi(\tau) \mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \Phi(\tau) (-\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau)) \\ &= \Phi(\tau) \mathbf{S}(\tau) d\mathbf{W}(\tau). \end{aligned}$$

This transformation yields an SDE with zero drift:

$$\begin{cases} d\mathbf{y}(\tau) = \Phi(\tau) \mathbf{S}(\tau) d\mathbf{W}(\tau), & \tau \in [t_0, T], \\ \mathbf{y}(t_0) = \mathbf{x}_0. \end{cases}$$

Integrating this system from t_0 to τ gives

$$\mathbf{y}(\tau) = \mathbf{x}_0 + \int_{t_0}^{\tau} \Phi(u) \mathbf{S}(u) d\mathbf{W}(u).$$

By inverting the integrating factor, one returns to the original variable space:

$$(3.7) \quad \mathbf{x}(\tau) = \Phi(\tau)^{-1} \mathbf{x}_0 + \int_{t_0}^{\tau} \Phi(\tau, u) \mathbf{S}(u) d\mathbf{W}(u),$$

where $\Phi(\tau, u) := \Phi(\tau)^{-1} \Phi(u)$. Consequently, for each $\tau \in [t_0, T]$, the random vector $\mathbf{x}(\tau) \in \mathbb{R}^D$ is a linear transformation of a Gaussian process. Then, it follows a multivariate normal distribution

$$\mathcal{N}(\mathbb{E}[\mathbf{x}(\tau)], \Sigma(\tau)),$$

where the mean is derived using the linearity of expectations and Ito's rules for stochastic integrals

$$\mathbb{E}[\mathbf{x}(\tau)] = \mathbb{E}[\Phi(\tau)^{-1} \mathbf{x}_0] + \mathbb{E}\left[\int_{t_0}^{\tau} \Phi(\tau, u) \mathbf{S}(u) d\mathbf{W}(u)\right] = \Phi(\tau)^{-1} \mathbf{x}_0,$$

and the covariance matrix $\Sigma(\tau) := \text{Cov}(\mathbf{x}(\tau))$ is given by (see Itô isometry [23, Corollary 3.1.4]):

$$\Sigma(\tau) = \int_{t_0}^{\tau} \Phi(\tau, u) \mathbf{S}(u) \mathbf{S}(u)^\top \Phi(\tau, u)^\top du.$$

In conclusion, for each $\tau \in [t_0, T]$, the exact marginal distribution of the general time-dependent Ornstein–Uhlenbeck process (3.5) is given by

$$(3.8) \quad \boxed{\mathbf{x}(\tau) \sim \mathcal{N}_D \left(\Phi(\tau)^{-1} \mathbf{x}_0, \int_{t_0}^{\tau} \Phi(\tau)^{-1} \Phi(u) \mathbf{S}(u) \mathbf{S}(u)^\top (\Phi(\tau)^{-1} \Phi(u))^\top du \right)},$$

where $\Phi(\tau)$ is the time-ordered exponential

$$\Phi(\tau) = \mathcal{T} \exp \left(\int_{t_0}^{\tau} \mathbf{K}(s) ds \right).$$

Stationary Distribution

In the general time-dependent framework, the existence of a unique stationary distribution is not universally guaranteed, as the system parameters $\mathbf{K}(\tau)$ and $\mathbf{S}(\tau)$ continuously evolve. However, in the context of score-based diffusion models, these schedules are chosen such that the system either stabilises asymptotically as $\tau \rightarrow \infty$.

Assuming that the drift matrix settles to an asymptotic limit $\mathbf{K}_\infty = \lim_{\tau \rightarrow \infty} \mathbf{K}(\tau)$ whose eigenvalues all possess positive real parts, and the diffusion matrix approaches $\mathbf{S}_\infty = \lim_{\tau \rightarrow \infty} \mathbf{S}(\tau)$, the influence of the initial condition $\mathbf{x}_0$ decays completely:

$$\lim_{\tau \rightarrow \infty} \boldsymbol{\mu}(\tau) = \lim_{\tau \rightarrow \infty} \Phi(\tau, t_0)^{-1} \mathbf{x}_0 = \mathbf{0}.$$

Concurrently, the time-derivative of the covariance matrix in the differential Lyapunov equation vanishes ($\lim_{\tau \rightarrow \infty} \frac{d}{d\tau} \Sigma(\tau) = \mathbf{0}$). From Theorem D.1, it follows that the covariance matrix Σ_∞ satisfies the algebraic Lyapunov equation:

$$\mathbf{K}_\infty \Sigma_\infty + \Sigma_\infty \mathbf{K}_\infty^\top = \mathbf{S}_\infty \mathbf{S}_\infty^\top.$$

Under these specific parametric limits, the process converges to the stationary distribution:

$$\mathbf{x}(\tau) \xrightarrow{\tau \rightarrow \infty} \mathcal{N}_D(\mathbf{0}, \Sigma_\infty).$$

In multivariate Ornstein–Uhlenbeck processes, $\mathbf{K}(\tau)$ and $\mathbf{S}(\tau)$ are balanced precisely such that $\Sigma_\infty = I_D$, leading to a standard Gaussian stationary distribution. For example, by taking

$$\mathbf{K}(\tau) + \mathbf{K}(\tau)^\top = \mathbf{S}(\tau) \mathbf{S}(\tau)^\top,$$

or in the case of symmetric coefficients: $2\mathbf{K}(\tau) = \mathbf{S}(\tau)^2$.

3.3. Circulant Ornstein–Uhlenbeck Process

In arbitrary multivariate linear SDEs, time-dependent coefficients introduce severe analytical complications because the drift matrices evaluated at different time instants do not generally commute. Consequently, the fundamental solution operator cannot be written as a simple exponential matrix, and instead requires complex time-ordered expansions such as the Dyson series. Implementing these systems directly in high-dimensional machine learning problems quickly becomes computationally impractical. For instance, for a state space of dimension D , computing the fundamental matrix $\Phi(t)$ through a Dyson series or solving the associated continuous-time Lyapunov equation involves matrix multiplications and inversions with computational cost on the order of $\mathcal{O}(D^3)$. In modern generative models, such as diffusion models for images, audio, or video where dimensions can easily reach $D \approx 10^5$ to 10^7 , these operations are simply too expensive to perform directly.

A natural way to overcome these difficulties is to restrict the space of valid matrices. For example, considering the space of circulant matrices provides a tractable alternative, since this non-commutativity bottleneck vanishes entirely. As shown in Appendix A, every circulant matrix shares the exact same eigenbasis defined by the conjugate-paired Discrete Fourier Transform matrix $\mathbf{F}$, regardless of its specific entry values or temporal arguments. Consequently, any two circulant matrices commute:

$$\mathbf{K}(\tau_1) \mathbf{K}(\tau_2) = \mathbf{K}(\tau_2) \mathbf{K}(\tau_1), \quad \forall \tau_1, \tau_2 \in [t_0, T].$$

This property eliminates the need for time-ordering operators, allowing the fundamental solution to be obtained using ordinary matrix exponentials and standard calculus. At the same time, circulant matrices are completely determined by their first row. Furthermore, diagonalisation in the Fourier basis enables matrix operations to be performed via the Fast Fourier Transform (FFT). These costs are shown in Table 3.1.

Table 3.1: Comparison of computational complexity for general and circulant matrices.

Operation	General matrix	Circulant matrix
Storage	$\mathcal{O}(D^2)$	$\mathcal{O}(D)$
Matrix-vector multiplication	$\mathcal{O}(D^2)$	$\mathcal{O}(D \log D)$
Matrix inversion	$\mathcal{O}(D^3)$	$\mathcal{O}(D \log D)$
Matrix square root	$\mathcal{O}(D^3)$	$\mathcal{O}(D \log D)$
Eigenvalue decomposition	$\mathcal{O}(D^3)$	$\mathcal{O}(D \log D)$ (via FFT)

This restriction considerably simplifies the analysis and implementation of the process. Indeed, working in Fourier space avoids many of the computational difficulties associated with time-dependent matrix dynamics, while still retaining a rich multivariate structure. However, addressing Fourier space solutions requires a solid background in complex distributions, which is developed in Appendix B.

The remainder of this section derives the solution for general time-dependent circulant Ornstein–Uhlenbeck processes and subsequently considers the symmetric model and a specific Variance Preserving (VP) condition as special cases.

3.3.1. Closed-Form Solution via Spectral Integration

Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent, continuous circulant matrices over the interval $[t_0, T]$. Consider the multivariate Ornstein–Uhlenbeck process governed by the linear Itô SDE:

$$(3.9) \quad d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

where $\mathbf{W}(\tau)$ is a standard D -dimensional Wiener process, and $\mathbf{x}_0 \in \mathbb{R}^D$ is a deterministic initial condition.

1. Diagonalisation. A detailed spectral decomposition of circulant matrices is given in Appendix A, specifically in Theorem A.3. Using this framework, the system is transformed into frequency space at every time step τ . The drift and diffusion matrices are simultaneously diagonalised as:

$$\mathbf{K}(\tau) = \mathbf{F}^\dagger \mathbf{\Lambda}(\tau) \mathbf{F}, \quad \mathbf{S}(\tau) = \mathbf{F}^\dagger \mathbf{B}(\tau) \mathbf{F},$$

where $\mathbf{\Lambda}(\tau)$ and $\mathbf{B}(\tau)$ are time-dependent diagonal matrices containing the time-varying complex eigenvalues $\lambda_j(\tau)$ and $\beta_j(\tau)$ with the following structure.

If D is even:

$$\begin{aligned} \mathbf{\Lambda}(\tau) &= \text{diag} \left(\lambda_0(\tau) \mid \lambda_1(\tau) \mid \lambda_{-1}(\tau) \mid \cdots \mid \lambda_{\frac{D}{2}-1}(\tau) \mid \lambda_{-(\frac{D}{2}-1)}(\tau) \mid \lambda_{\frac{D}{2}}(\tau) \right), \\ \mathbf{B}(\tau) &= \text{diag} \left(\beta_0(\tau) \mid \beta_1(\tau) \mid \beta_{-1}(\tau) \mid \cdots \mid \beta_{\frac{D}{2}-1}(\tau) \mid \beta_{-(\frac{D}{2}-1)}(\tau) \mid \beta_{\frac{D}{2}}(\tau) \right), \end{aligned}$$

and if D is odd:

$$\begin{aligned} \mathbf{\Lambda}(\tau) &= \text{diag} \left(\lambda_0(\tau) \mid \lambda_1(\tau) \mid \lambda_{-1}(\tau) \mid \cdots \mid \lambda_{\frac{D-1}{2}}(\tau) \mid \lambda_{-\frac{D-1}{2}}(\tau) \right), \\ \mathbf{B}(\tau) &= \text{diag} \left(\beta_0(\tau) \mid \beta_1(\tau) \mid \beta_{-1}(\tau) \mid \cdots \mid \beta_{\frac{D-1}{2}}(\tau) \mid \beta_{-\frac{D-1}{2}}(\tau) \right). \end{aligned}$$

2. Decoupling in Fourier Space. After applying the Discrete Fourier Transform (DFT), the following complex-valued stochastic processes are defined

$$\mathbf{y}(\tau) = \mathbf{F} \mathbf{x}(\tau), \quad \widetilde{\mathbf{W}}(\tau) = \mathbf{F} \mathbf{W}(\tau).$$

Since $\mathbf{F}$ is unitary, $\widetilde{\mathbf{W}}(\tau)$ is a complex-valued Gaussian process characterised by (see Example B.24)

$$\widetilde{\mathbf{W}}(\tau) \sim \mathcal{CN}_D(0, \tau I_D, \tau \mathbf{F} \mathbf{F}^\top).$$

From Theorem A.4, the structure of $\mathbf{F} \mathbf{F}^\top$ groups the conjugate pairs together:

$$\mathbf{F} \mathbf{F}^\top = \text{diag} \left(1, \underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}}_{\text{pairs } (k, -k)}, \epsilon \right), \quad \begin{cases} \epsilon = 1 & \text{if } D \text{ is even,} \\ \epsilon & \text{is absent if } D \text{ is odd.} \end{cases}$$

Differentiating $\mathbf{y}(\tau)$ gives

$$\begin{aligned} d\mathbf{y}(\tau) &= \mathbf{F} d\mathbf{x}(\tau) \\ &= -\mathbf{F} \mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{F} \mathbf{S}(\tau) d\mathbf{W}(\tau) \\ &= -\mathbf{F} \mathbf{F}^\dagger \mathbf{\Lambda}(\tau) \mathbf{F} \mathbf{x}(\tau) d\tau + \mathbf{F} \mathbf{F}^\dagger \mathbf{B}(\tau) \mathbf{F} d\mathbf{W}(\tau) \\ &= -\mathbf{\Lambda}(\tau) \mathbf{y}(\tau) d\tau + \mathbf{B}(\tau) d\widetilde{\mathbf{W}}(\tau) \end{aligned}$$

with the transformed initial condition $\mathbf{y}_0 = \mathbf{F} \mathbf{x}_0$. Thus, the coupled SDE system elegantly decouples into the frequency domain:

$$(3.10) \quad \boxed{d\mathbf{y}(\tau) = -\mathbf{\Lambda}(\tau) \mathbf{y}(\tau) d\tau + \mathbf{B}(\tau) d\widetilde{\mathbf{W}}(\tau), \quad \mathbf{y}(t_0) = \mathbf{y}_0.}$$

Because both the drift $\mathbf{\Lambda}(\tau)$ and diffusion $\mathbf{B}(\tau)$ matrices are purely diagonal, the system decouples into D scalar SDEs. However, these equations are not independent due to the pseudo-covariance structure of the transformed Wiener process $\widetilde{\mathbf{W}}(\tau)$. For each Fourier mode $j = 0, \dots, D-1$:

$$\boxed{dy_j(\tau) = -\lambda_j(\tau) y_j(\tau) d\tau + \beta_j(\tau) d\widetilde{\mathbf{W}}_j(\tau), \quad y_j(t_0) = y_0^{(j)}.$$

3. Solution of Individual Fourier Modes. Although the decoupled SDE resembles the real-valued process solved in Section 3.1, it operates in the complex plane due to the $d\widetilde{\mathbf{W}}(\tau)$ term. However, even though the mathematical derivation changes, the methodology is quite similar. To solve the time-dependent scalar equation for the j -th mode, the following time-varying integrating factor is introduced:

$$\Phi_j(\tau) = e^{\int_{t_0}^{\tau} \lambda_j(u) du}.$$

Applying Itô's product rule (see [23, Theorem 4.1.2]) to the product $\Phi_j(\tau)y_j(\tau)$ eliminates the drift term, leaving:

$$d(\Phi_j(\tau) y_j(\tau)) = \beta_j(\tau) \Phi_j(\tau) d\widetilde{\mathbf{W}}_j(\tau).$$

Integrating this expression directly over the temporal interval $[t_0, \tau]$ and dividing by the integrating factor yields the analytical solution for the j -th frequency mode:

$$y_j(\tau) = y_0^{(j)} e^{-\int_{t_0}^{\tau} \lambda_j(u) du} + \int_{t_0}^{\tau} e^{-\int_u^{\tau} \lambda_j(r) dr} \beta_j(u) d\widetilde{\mathbf{W}}_j(u).$$

Reassembling all D equations back into a unified matrix representation yields the frequency space vector:

$$(3.11) \quad \mathbf{y}(\tau) = \mathbf{\Phi}(\tau, t_0) \mathbf{y}_0 + \int_{t_0}^{\tau} \mathbf{\Phi}(\tau, u) \mathbf{B}(u) d\widetilde{\mathbf{W}}(u),$$

where $\mathbf{\Phi}(\tau, u) \in \mathbb{C}^{D \times D}$ is the fundamental diagonal transition matrix defined as:

$$\mathbf{\Phi}(\tau, u) = e^{-\int_u^{\tau} \mathbf{\Lambda}(r) dr} = \text{diag} \left(e^{-\int_u^{\tau} \lambda_j(r) dr} \right)_{j=0}^{D-1}.$$

4. Distribution in Fourier Space. Since $\mathbf{y}(\tau)$ is obtained as a linear transformation of a Gaussian process, it remains a Gaussian process. In the complex setting, this distribution is fully characterised by its mean vector, covariance matrix, and pseudo-covariance matrix. For any $\tau \in [t_0, T]$, the Fourier space solution follows a complex multivariate normal distribution:

$$(3.12) \quad \boxed{\mathbf{y}(\tau) \sim \mathcal{CN}_D(\boldsymbol{\mu}_{\mathbf{y}}(\tau), \mathbf{C}_{\mathbf{y}}(\tau), \mathbf{P}_{\mathbf{y}}(\tau))}.$$

The time-varying mean vector is given by:

$$(3.13) \quad \boldsymbol{\mu}_{\mathbf{y}}(\tau) = e^{-\int_{t_0}^{\tau} \boldsymbol{\Lambda}(u) du} \mathbf{y}_0.$$

Applying the complex Itô isometry for time-dependent integrands and the commutativity of circulant matrices, the diagonal covariance matrix evolves as:

$$\begin{aligned} \mathbf{C}_{\mathbf{y}}(\tau) &= \int_{t_0}^{\tau} \boldsymbol{\Phi}(\tau, u) \mathbf{B}(u) \mathbf{B}(u)^{\dagger} \boldsymbol{\Phi}(\tau, u)^{\dagger} du \\ &= \text{diag} \left(\int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \text{Re}(\lambda_j(r)) dr} |\beta_j(u)|^2 du \right)_{j=0}^{D-1}. \end{aligned}$$

The pseudo-covariance captures the coupling between conjugate modes by incorporating the static structure of $\mathbf{F}\mathbf{F}^{\top}$:

$$\mathbf{P}_{\mathbf{y}}(\tau) = \int_{t_0}^{\tau} e^{-\int_u^{\tau} \boldsymbol{\Lambda}(r) dr} \mathbf{B}(u) \mathbf{F}\mathbf{F}^{\top} \mathbf{B}(u)^{\top} e^{-\int_u^{\tau} \boldsymbol{\Lambda}(r)^{\top} dr} du.$$

Evaluating this expression block by block and using the conjugate symmetry relation $\lambda_{-m}(\tau) = \overline{\lambda_m(\tau)}$, the non-vanishing entries are defined for $m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor$ as:

$$\begin{aligned} [\mathbf{P}_{\mathbf{y}}(\tau)]_{0,0} &= \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda_0(r) dr} \beta_0(u)^2 du, \\ [\mathbf{P}_{\mathbf{y}}(\tau)]_{2m-1, 2m} &= \int_{t_0}^{\tau} e^{-\int_u^{\tau} \lambda_m(r) dr} e^{-\int_u^{\tau} \lambda_{-m}(r) dr} \beta_m(u) \beta_{-m}(u) du, \\ &= \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \text{Re}(\lambda_m(r)) dr} |\beta_m(u)|^2 du, \end{aligned}$$

and if D is even, the Nyquist mode satisfies:

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{D-1, D-1} = \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda_{\frac{D}{2}}(r) dr} \beta_{\frac{D}{2}}(u)^2 du.$$

All other terms are identically zero.

Observe that the covariance and pseudocovariance components corresponding to the DC mode (and to the Nyquist mode, if it exists) coincide, reflecting the conjugate symmetry of the Fourier transform of real-valued processes. Note also that both the covariance and pseudo-covariance matrices are real-valued. By Theorem B.21, this implies that the real and imaginary parts of $\mathbf{y}(\tau)$ are uncorrelated, though they are not independent in general since the pseudo-covariance does not vanish. In particular,

$$y_{-j}(\tau) = \overline{y_j(\tau)}, \quad j = 1, \dots, D-1,$$

while $\mathbf{y}_0(\tau)$ is real-valued, which ensures that the inverse transform $\mathbf{x}(\tau) = \mathbf{F}^{\dagger} \mathbf{y}(\tau)$ remains real-valued.

5. Transformation back to Physical Space. The solution in the original spatial coordinates is recovered by applying the inverse DFT to the frequency space solution.

$$\mathbf{x}(\tau) = \mathbf{F}^\dagger \mathbf{y}(\tau).$$

Substituting the Fourier space integral representation (3.11) yields:

$$(3.14) \quad \mathbf{x}(\tau) = e^{-\int_{t_0}^{\tau} \mathbf{K}(u) du} \mathbf{x}_0 + \int_{t_0}^{\tau} e^{-\int_u^{\tau} \mathbf{K}(r) dr} \mathbf{S}(u) d\mathbf{W}(u).$$

Since $\mathbf{x}(\tau)$ is an affine transformation of a Wiener process, it follows a multivariate normal distribution in $\mathbb{R}^D$. Using standard properties of Itô integrals (see [23, Corollary 3.1.4 and Theorem 3.2.1]),

$$\mathbf{x}(\tau) \sim \mathcal{N} \left(e^{-\int_{t_0}^{\tau} \mathbf{K}(u) du} \mathbf{x}_0, \int_{t_0}^{\tau} e^{-\int_u^{\tau} \mathbf{K}(r) dr} \mathbf{S}(u) \mathbf{S}(u)^\top e^{-\int_u^{\tau} \mathbf{K}(r)^\top dr} du \right).$$

Furthermore, because the matrices $\mathbf{K}(\tau)$, $\mathbf{K}(\tau)^\top$, and $\mathbf{S}(u) \mathbf{S}(u)^\top$ commute for all τ , u (since they are circulant matrices), then the covariance simplifies to

$$(3.15) \quad \boxed{\mathbf{x}(\tau) \sim \mathcal{N} \left(e^{-\int_{t_0}^{\tau} \mathbf{K}(u) du} \mathbf{x}_0, \int_{t_0}^{\tau} e^{-\int_u^{\tau} (\mathbf{K}(r) + \mathbf{K}(r)^\top) dr} \mathbf{S}(u) \mathbf{S}(u)^\top du \right)}.$$

As a consistency check, one can recover this exact physical distribution by applying the transform formulas from Example B.23 directly to the complex distribution established in (3.12). This is shown in Appendix D.2.

3.3.2. Symmetric Time-Dependent Case

Enforcing symmetry on the circulant operators yields substantial advantages both analytically and computationally. Mathematically, the symmetry property of circulant matrices is equivalent to the fact that their Fourier eigenvalues are strictly real-valued (see Theorem A.2). This ensures that the diagonal spectral matrices $\mathbf{\Lambda}(\tau)$ and $\mathbf{B}(\tau)$ contain exclusively real entries, which simplifies the Fourier space representation $\mathbf{y}(\tau)$ as well as the solution in the original coordinates $\mathbf{x}(\tau)$.

Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent matrices defined on $\tau \in [t_0, T]$, and assume they are both **circulant** and **symmetric**. Consider the stochastic process $\mathbf{x}(\tau) \in \mathbb{R}^D$ governed by the Ornstein–Uhlenbeck process:

$$d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0.$$

Under this symmetry assumption,

$$\boxed{\mathbf{y}(\tau) \sim \mathcal{CN}_D(\boldsymbol{\mu}_{\mathbf{y}}(\tau), \mathbf{C}_{\mathbf{y}}(\tau), \mathbf{P}_{\mathbf{y}}(\tau)),}$$

where the formulas from (3.12) simplify to

$$\begin{aligned} \boldsymbol{\mu}_{\mathbf{y}}(\tau) &= e^{-\int_{t_0}^{\tau} \mathbf{\Lambda}(u) du} \mathbf{y}_0, \\ \mathbf{C}_{\mathbf{y}}(\tau) &= \int_{t_0}^{\tau} e^{-2\int_u^{\tau} \mathbf{\Lambda}(r) dr} \mathbf{B}(u)^2 du, \end{aligned}$$

and all operations are understood element-wise due to the diagonal structure. The pseudo-covariance formula simplifies to

- **DC mode:**

$$[\mathbf{P}_y(\tau)]_{0,0} = \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda_0(r) dr} \beta_0(u)^2 du,$$

- **Conjugate pairs:** for $m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor$:

$$[\mathbf{P}_y(\tau)]_{2m-1, 2m} = \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda_m(r) dr} \beta_m(u)^2 du,$$

- **Nyquist mode (if D is even):**

$$[\mathbf{P}_y(\tau)]_{D-1, D-1} = \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \lambda_{\frac{D}{2}}(r) dr} \beta_{\frac{D}{2}}(u)^2 du.$$

- Otherwise, $[\mathbf{P}_y(\tau)]_{j,k} = 0$.

Moreover, the solution in physical space $\mathbf{x}(\tau)$ (3.15) also simplifies to

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N} \left(e^{-\int_{t_0}^{\tau} \mathbf{K}(u) du} \mathbf{x}_0, \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \mathbf{K}(r) dr} \mathbf{S}(u)^2 du \right)},$$

where the square is understood as a matrix operator.

3.3.3. Variance-Preserving Constraint

In the context of modern generative diffusion models, it is highly desirable to construct a forward process whose stationary distribution is standard normal, $\mathcal{N}(\mathbf{0}, I_D)$, ensuring that data cleanly degrades into white noise over long times. To achieve this, a specific *Variance Preserving* (VP) constraint is introduced, matching the scales of the drift and diffusion forces.

Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent matrices defined on $\tau \in [t_0, T]$, and assume they are both **circulant** and **symmetric**. Consider the stochastic process $\mathbf{x}(\tau) \in \mathbb{R}^D$ governed by the Ornstein–Uhlenbeck equation

$$d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(t_0) = \mathbf{x}_0,$$

and assume in addition the following **Variance Preserving** condition:

$$\mathbf{S}(\tau)^2 = 2 \mathbf{K}(\tau), \quad \forall \tau \in [t_0, T].$$

Since $\mathbf{K}(\tau)$ and $\mathbf{S}(\tau)$ are circulant and symmetric, they both are simultaneously diagonalised via the conjugate-paired DFT matrix $\mathbf{F}$:

$$\mathbf{K}(\tau) = \mathbf{F}^\dagger \mathbf{\Lambda}(\tau) \mathbf{F}, \quad \mathbf{S}(\tau) = \mathbf{F}^\dagger \mathbf{B}(\tau) \mathbf{F},$$

where $\mathbf{F}$ is the conjugate-paired Discrete Fourier Transform matrix defined in Section A.4, and $\mathbf{\Lambda}(\tau)$ and $\mathbf{B}(\tau)$ are real-valued diagonal matrices. The Variance Preserving condition is therefore equivalent to

$$\mathbf{B}(\tau)^2 = 2 \mathbf{\Lambda}(\tau), \quad \forall \tau \in [t_0, T].$$

That is

$$\beta_m(\tau)^2 = 2 \lambda_m(\tau), \quad \forall \tau \in [t_0, T], \forall m = 0, \dots, D-1.$$

Hence, substituting this identity into the frequency-space formulations (3.13) yields

$$\mathbf{y}(\tau) \sim \mathcal{CN}_D(\boldsymbol{\mu}_{\mathbf{y}}(\tau), \mathbf{C}_{\mathbf{y}}(\tau), \mathbf{P}_{\mathbf{y}}(\tau)),$$

where the expressions in (3.13) simplify to

$$\boldsymbol{\mu}_{\mathbf{y}}(\tau) = e^{-\int_{t_0}^{\tau} \mathbf{\Lambda}(u) du} \mathbf{y}_0,$$

and using that

$$\frac{d}{du} \left[\exp \left(-2 \int_u^{\tau} \mathbf{\Lambda}(r) dr \right) \right] = 2 \mathbf{\Lambda}(u) \exp \left(-2 \int_u^{\tau} \mathbf{\Lambda}(r) dr \right),$$

the covariance expression is

$$\begin{aligned} \mathbf{C}_{\mathbf{y}}(\tau) &= \int_{t_0}^{\tau} e^{-2 \int_u^{\tau} \mathbf{\Lambda}(r) dr} 2 \mathbf{\Lambda}(u) du \\ &= I_D - e^{-2 \int_{t_0}^{\tau} \mathbf{\Lambda}(u) du}, \end{aligned}$$

where all operations are understood element-wise due to the diagonal structure. The pseudo-covariance matrix simplifies to

- **DC mode:**

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{0,0} = 1 - e^{-2 \int_{t_0}^{\tau} \lambda_0(u) du},$$

- **Conjugate pairs:** for $m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor$:

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{2m-1, 2m} = 1 - e^{-2 \int_{t_0}^{\tau} \lambda_m(u) du},$$

- **Nyquist mode (if D is even):**

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{D-1, D-1} = 1 - e^{-2 \int_{t_0}^{\tau} \lambda_{\frac{D}{2}}(u) du},$$

- Otherwise, $[\mathbf{P}_{\mathbf{y}}(\tau)]_{j,k} = 0$.

Moreover, the solution in physical space satisfies

$$\mathbf{x}(\tau) \sim \mathcal{N} \left(e^{-\int_{t_0}^{\tau} \mathbf{K}(u) du} \mathbf{x}_0, I_D - e^{-2 \int_{t_0}^{\tau} \mathbf{K}(u) du} \right).$$

The contractive property of the stable drift matrix ($\lambda_m > 0$) guarantees that the conditional mean vanishes, while the covariance matrix approaches identity. Consequently, the stationary distribution for this specific Variance Preserving symmetric circulant Ornstein–Uhlenbeck process converges cleanly to:

$$\mathbf{x}(\tau) \xrightarrow{\tau \rightarrow \infty} \mathcal{N}(\mathbf{0}, I_D).$$

As a concrete illustration, the linear drift schedule under this Variance Preserving condition admits explicit closed-form expressions for the mean and covariance, which are derived in Appendix C.2.

3.4. Multivariate Score-Based Diffusion Models

The formulation of isotropic diffusion models established in Section 2.2.1 extends directly to the general multivariate setting, which is the framework considered in this work. Let $\mathbf{x}(\tau) \in \mathbb{R}^D$ be a D -dimensional stochastic process governed by the Itô stochastic differential equation

$$d\mathbf{x}(\tau) = \mathbf{f}(\mathbf{x}(\tau), \tau) d\tau + \mathbf{G}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(0) \sim p_0(\mathbf{x}),$$

where $\mathbf{f} : \mathbb{R}^D \times [0, T] \rightarrow \mathbb{R}^D$ is the drift vector field, $\mathbf{G}(\tau) \in \mathbb{R}^{D \times D}$ is the diffusion matrix, and $\mathbf{W}(\tau) \in \mathbb{R}^D$ is a standard D -dimensional Wiener process.

As in the scalar case, the forward diffusion process progressively transforms the original data distribution into a simpler prior distribution $p_T(\mathbf{x})$, generating a family of intermediate distributions

$$\{p_\tau(\mathbf{x})\}_{\tau \in [0, T]}.$$

The corresponding score function is now given by

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) \in \mathbb{R}^D,$$

which is a vector field describing the local geometry of the probability density. Following the denoising score matching framework of Section 2.2.1, the training objective generalises to

$$(3.16) \quad \mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{U}(0, T)} \left[\mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}, \tau | \mathbf{x}_0, 0)} \left[\left\| \mathbf{\Gamma}(\tau) (\mathbf{s}(\mathbf{x}, \tau; \theta) - \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0)) \right\|_2^2 \right] \right] \right].$$

where $\mathbf{\Gamma}(\tau)$ is a matrix weighting function equivalent to $\sqrt{\gamma(\tau)}$ in the scalar case. Note that the square root is taken because $\gamma(\tau)$ is moved inside the Euclidean norm.

The reverse-time diffusion process is governed by the stochastic differential equation

$$d\overleftarrow{\mathbf{x}}(\tau) = \left(\mathbf{f}(\overleftarrow{\mathbf{x}}(\tau), \tau) - \mathbf{G}(\tau) \mathbf{G}(\tau)^\top \nabla_{\overleftarrow{\mathbf{x}}} \log p_\tau(\overleftarrow{\mathbf{x}}) \right) d\tau + \mathbf{G}(\tau) d\overleftarrow{\mathbf{W}}(\tau),$$

where $\overleftarrow{\mathbf{x}}(T) \sim p_T(\mathbf{x})$ and $\overleftarrow{\mathbf{W}}(\tau)$ denotes a reverse-time Wiener process.

This multidimensional framework forms the mathematical basis for the Ornstein–Uhlenbeck diffusion model studied in this work.

3.4.1. Ornstein–Uhlenbeck Framework

In this work, the diffusion process is modelled by a multivariate Ornstein–Uhlenbeck stochastic differential equation. This choice is particularly convenient for both theoretical and computational reasons. The Ornstein–Uhlenbeck process admits explicit Gaussian marginal distributions for all times $\tau > 0$, which allows a closed-form characterisation of the associated score function. This is one of the main advantages of this model in the context of score-based generative modelling. For this reason, the derivation of the score function of a Gaussian distribution becomes a fundamental step in the construction of the training objective. That derivation can be found at Theorem D.4.

Consider the stochastic differential equation governed by the general OU process defined in (3.5). Let $D \in \mathbb{N}$ and $T > 0$. Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent matrices defined for $\tau \in [t_0, T]$, then the stochastic process $\mathbf{x}(\tau) \in \mathbb{R}^D$ satisfies:

$$d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(0) = \mathbf{x}_0,$$

where $\mathbf{W}(\tau)$ is a standard D -dimensional Wiener process, and $\mathbf{x}_0 \in \mathbb{R}^D$ is deterministic.

In Chapter 3, there is a completely detailed derivation of the solution of several versions of this equation. However, the fundamental fact about the Ornstein–Uhlenbeck process is that marginal solutions follow a Gaussian distribution. Therefore, considering $\mathbf{x}_0$ as a stochastic initial condition, the solution can be denoted as

$$\mathbf{x}(\tau) \mid \mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{x}_0}(\tau), \boldsymbol{\Sigma}(\tau)),$$

This representation allows one to write it as

$$\mathbf{x}(\tau) = \boldsymbol{\mu}_{\mathbf{x}_0}(\tau) + \mathbf{L}_\tau \mathbf{z},$$

where $\boldsymbol{\Sigma}(\tau) = \mathbf{L}_\tau \mathbf{L}_\tau^\top$ is the Cholesky decomposition of the covariance matrix and $\mathbf{z} \sim \mathcal{N}(0, I_D)$. Then, following from Theorem D.4, the score function can be expressed as

$$\begin{aligned} \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0) &= -\boldsymbol{\Sigma}(\tau)^{-1} (\mathbf{x}(\tau) - \boldsymbol{\mu}_{\mathbf{x}_0}(\tau)) = -\boldsymbol{\Sigma}(\tau)^{-1} \mathbf{L}_\tau \mathbf{z} \\ &= -(\mathbf{L}_\tau \mathbf{L}_\tau^\top)^{-1} \mathbf{L}_\tau \mathbf{z} = -(\mathbf{L}_\tau^\top)^{-1} \mathbf{z}. \end{aligned}$$

and the loss function (3.16) as

$$\begin{aligned} \mathcal{L}(\theta) &= \mathbb{E}_{\tau \sim \mathcal{U}(0, T)} \left[\mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0)} \left[\|\boldsymbol{\Gamma}(\tau) [\mathbf{s}(\mathbf{x}, \tau; \theta) + (\mathbf{L}_\tau^\top)^{-1} \mathbf{z}]\|_2^2 \right] \right] \right] \\ &= \mathbb{E}_{\tau \sim \mathcal{U}(0, T)} \left[\mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0)} \left[\|\boldsymbol{\Gamma}(\tau) (\mathbf{L}_\tau^\top)^{-1} [\mathbf{L}_\tau^\top \mathbf{s}(\mathbf{x}, \tau; \theta) + \mathbf{z}]\|_2^2 \right] \right] \right] \end{aligned}$$

Choosing $\boldsymbol{\Gamma}(\tau) \propto \mathbf{L}_\tau^\top$ yields the final expression for the loss function is

$$\mathcal{L}(\theta) \propto \mathbb{E}_{\tau \sim \mathcal{U}(0, T)} \left[\mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0)} \left[\|\mathbf{L}_\tau^\top \mathbf{s}(\mathbf{x}, \tau; \theta) + \mathbf{z}\|_2^2 \right] \right] \right],$$

which can be numerically estimated using Monte Carlo method. Let $\tau_n \sim \mathcal{U}(0, T)$. Let $\mathbf{z}_n \sim N(0, I_D)$:

$$\mathcal{L}(\theta) \propto \frac{1}{N} \sum_{n=1}^N \|\mathbf{L}_{\tau_n}^\top \mathbf{s}(\mathbf{x}_n, \tau_n; \theta) + \mathbf{z}_n\|_2^2,$$

where $\mathbf{x}_n = \mathbf{x}(\tau_n) = \mu(\tau_n) + \mathbf{L}_{\tau_n} \mathbf{z}_n$. Finally, the corresponding backward SDE is

$$d\overleftarrow{\mathbf{x}}(\tau) = \left(-\mathbf{K}(\tau) \overleftarrow{\mathbf{x}}(\tau) - \mathbf{S}(\tau) \mathbf{S}(\tau)^\top \nabla_{\mathbf{x}} \log p(\overleftarrow{\mathbf{x}}(\tau)) \right) d\tau + \mathbf{S}(\tau) d\overleftarrow{\mathbf{W}}(\tau), \quad \tau \in [T, 0]$$

where $\overleftarrow{\mathbf{W}}(\tau)$ is a Wiener process from T to 0. Using the estimator $\mathbf{s}_\theta(\mathbf{x}, \tau)$, the backward diffusion process is defined, for $\tau \in [T, 0]$, as

$$d\overleftarrow{\mathbf{x}}(\tau) = \left(-\mathbf{K}(\tau) \overleftarrow{\mathbf{x}}(\tau) - \mathbf{S}(\tau) \mathbf{S}(\tau)^\top \mathbf{s}_\theta(\overleftarrow{\mathbf{x}}(\tau), \tau) \right) d\tau + \mathbf{S}(\tau) d\overleftarrow{\mathbf{W}}(\tau),$$

where $\overleftarrow{\mathbf{x}}(T) \sim p_T(\overleftarrow{\mathbf{x}})$.

CHAPTER 4

Flow Matching

This chapter introduces the Flow Matching framework and its Conditional Flow Matching formulation, before describing the interpolation strategy adopted in this work and its application to discretely observed functional data.

The score-based diffusion framework developed in Chapter 3 generates synthetic samples by simulating a reverse-time stochastic differential equation, where the learnt score function guides a noisy sample back toward the data distribution. However, stochasticity is not a fundamental requirement for this process. Song et al. [10] demonstrated that every diffusion SDE admits a deterministic equivalence: the *probability flow ODE*. This ODE produces samples with identical marginal distributions without injecting any noise during generation, as shown in Figure 4.1. For the general Itô SDE introduced in Chapter 2, the corresponding probability flow ODE is given by

$$d\mathbf{x}(\tau) = \left(f(\mathbf{x}(\tau), \tau) - \frac{1}{2} g(\tau)^2 \nabla_{\mathbf{x}} \log p_{\tau}(\mathbf{x}) \right) d\tau,$$

where the stochastic Wiener process term has been entirely replaced by a deterministic drift correction driven by the score function. This reveals that the score function alone is sufficient to define a deterministic trajectory from noise to data.

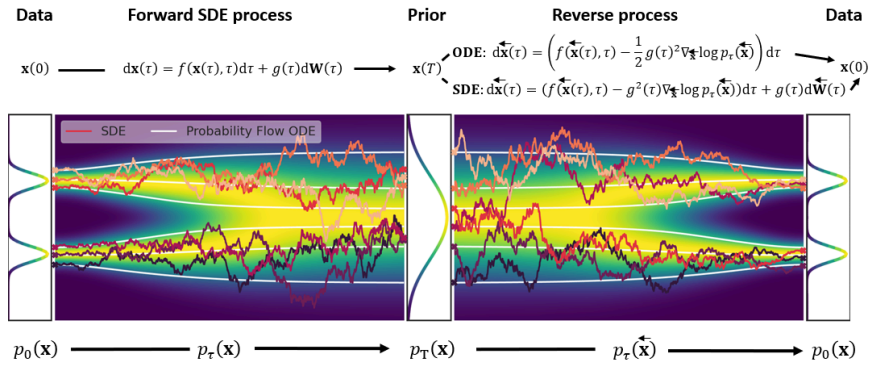


Figure 4.1: Relationship between the diffusion process and its corresponding probability flow ODE [10].

Flow Matching, introduced by Lipman et al. [21], generalises this perspective by entirely removing the SDE construction and the score function. Rather than first defining a stochastic forward process and then deriving its deterministic equivalent, Flow Matching directly parameterises the ODE vector field that interpolates samples from a simple source distribution to the target data distribution. This removes the need for any explicit noise schedule or score estimation, yielding a simpler training objective and straighter generative trajectories. The remainder of this chapter develops this framework formally and describes its adaptation to functional data, focusing on the simplest formulation of the framework in order to support a clean empirical comparison against the diffusion models of Chapter 3.

4.1. Conditional Flow Matching

Let $p_0(\mathbf{x})$ denote the source distribution from pure noise (typically a standard Gaussian), and let $p_1(\mathbf{x})$ be the target distribution associated with the original data. The central idea of Flow Matching is to define a family of intermediate distributions $\{p(\mathbf{x}(\tau), \tau)\}_{\tau \in [0,1]}$ that connect the source distribution and the target distribution, in such a way that $p(\mathbf{x}(0), 0)$ coincides with the source distribution $p_0(\mathbf{x})$ and $p(\mathbf{x}(1), 1)$ coincides with the target distribution $p_1(\mathbf{x})$.

The evolution of the intermediate samples $\{\mathbf{x}(\tau)\}_{\tau \in [0,1]}$ is given by a generally time-dependent velocity field $\mathbf{v}(\mathbf{x}(\tau), \tau)$ that satisfies the following ordinary differential equation (ODE):

$$\frac{d\mathbf{x}(\tau)}{d\tau} = \mathbf{v}(\mathbf{x}(\tau), \tau).$$

In practice, this marginal field is not directly learnt, but is instead constructed from a sample-level object. Given a fixed pair $(\mathbf{x}_0, \mathbf{x}_1)$ drawn from $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$ respectively, one can define a conditional path $\mathbf{x}(\tau) \mid \mathbf{x}_0, \mathbf{x}_1$ connecting $\mathbf{x}_0$ to $\mathbf{x}_1$, together with its associated *conditional velocity field* $\mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1)$, satisfying, for that fixed pair,

$$\frac{d\mathbf{x}(\tau)}{d\tau} = \mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1).$$

Because the conditional vector field depends on knowing both endpoints $(\mathbf{x}_0$ and $\mathbf{x}_1)$, it cannot be used directly for generation. However, computing the marginal velocity field directly as

$$\mathbf{v}(\mathbf{x}(\tau), \tau) = \mathbb{E}[\mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1) \mid \mathbf{x}(\tau)],$$

is generally intractable, since it requires access to the (unknown) distribution of $\mathbf{x}_0, \mathbf{x}_1$ given only the intermediate state $\mathbf{x}(\tau)$. Lipman et al. [21] solved this problem using the *Conditional Flow Matching (CFM) objective*, as they showed that minimising the Conditional Flow Matching Objective is equivalent to minimising the marginal Flow Matching objective, so that both losses share the same optimal solution. Therefore, they proposed the following loss function:

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{U}[0,1], \mathbf{x}_0 \sim p_0, \mathbf{x}_1 \sim p_1} [\|(\mathbf{v}(\mathbf{x}(\tau), \tau; \theta)) - \mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1)\|^2].$$

This equivalence is what justifies training on individual sample pairs while guaranteeing that the optimal estimator $\mathbf{v}_\theta$ satisfies

$$\mathbf{v}_\theta(\mathbf{x}(\tau), \tau) \approx \mathbf{v}(\mathbf{x}(\tau), \tau)$$

This objective is estimated by Monte Carlo method by considering N independent pairs of samples $(\mathbf{x}_0, \mathbf{x}_1)$ drawn from the source and target distributions

$$\{\mathbf{x}_{\mathbf{n}0}\}_{n=1}^N \sim p_0(\mathbf{x}), \quad \{\mathbf{x}_{\mathbf{n}1}\}_{n=1}^N \sim p_1(\mathbf{x}),$$

and sampling a random time step $\tau_n \sim \mathcal{U}(0, 1)$ to construct the corresponding conditional path point $\mathbf{x}_n(\tau_n)$, so that

$$\mathcal{L}(\theta) \approx \frac{1}{N} \sum_{n=1}^N \|\mathbf{v}(\mathbf{x}_n(\tau_n), \tau_n; \theta) - \mathbf{v}_c(\mathbf{x}_n(\tau_n), \tau_n \mid \mathbf{x}_{\mathbf{n}0}, \mathbf{x}_{\mathbf{n}1})\|^2.$$

After training, new samples must be generated. This is achieved by solving the ODE satisfied by the learnt velocity field $\mathbf{v}_\theta(\mathbf{x}(\tau), \tau)$ (which is a learnt approximation of the marginal velocity field) starting from an initial point $\mathbf{x}(0)$ drawn from the source distribution:

$$\frac{d\mathbf{x}(\tau)}{d\tau} = \mathbf{v}_\theta(\mathbf{x}(\tau), \tau), \quad \mathbf{x}(0) \sim p_0(\mathbf{x}).$$

The final integrated state $\mathbf{x}(1)$ is then taken as the generated synthetic sample.

4.2. Interpolation Paths

A common choice to construct such conditional paths is a linear or nonlinear interpolation between the samples $\mathbf{x}_0$ and $\mathbf{x}_1$. Define, for $\tau \in [0, 1]$, the time dependent differentiable functions $\alpha(\tau), \beta(\tau)$ such that $\alpha(0) = 1$ and $\alpha(1) = 0$, and $\beta(0) = 0$ and $\beta(1) = 1$. One possible conditional interpolation path is given by:

$$\mathbf{x}(\tau) = \alpha(\tau) \mathbf{x}_0 + \beta(\tau) \mathbf{x}_1, \quad \mathbf{x}_0 \sim p_0(\mathbf{x}), \quad \mathbf{x}_1 \sim p_1(\mathbf{x}).$$

This construction ensures that $\mathbf{x}(0) = \mathbf{x}_0$ and $\mathbf{x}(1) = \mathbf{x}_1$, yielding a continuous path from noise to data. Differentiating this interpolation path with respect to τ yields the explicit conditional velocity field

$$\mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1) = \alpha'(\tau) \mathbf{x}_0 + \beta'(\tau) \mathbf{x}_1,$$

and the corresponding Monte Carlo estimate of the loss becomes

$$\mathcal{L}(\theta) \approx \frac{1}{N} \sum_{n=1}^N \|\mathbf{v}(\mathbf{x}_n(\tau_n), \tau_n; \theta) - (\alpha'(\tau_n) \mathbf{x}_{\mathbf{n}0} + \beta'(\tau_n) \mathbf{x}_{\mathbf{n}1})\|^2.$$

An elegant and practical formulation for a Flow Matching model is the linear interpolation scheme:

$$\alpha(\tau) = 1 - \tau, \quad \beta(\tau) = \tau, \quad \tau \in [0, 1],$$

which defines the straight interpolation trajectory:

$$\mathbf{x}(\tau) = (1 - \tau) \mathbf{x}_0 + \tau \mathbf{x}_1, \quad \mathbf{x}_0 \sim p_0(\mathbf{x}), \quad \mathbf{x}_1 \sim p_1(\mathbf{x}).$$

Observe that at time $\tau = 0$, the interpolation path is simply pure noise, while at time $\tau = 1$ it is the real data. In this case, the velocity target simplifies to a constant direction vector:

$$\mathbf{v}_c(\mathbf{x}(\tau), \tau \mid \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0.$$

Therefore, the estimated loss function associated with this process is

$$\mathcal{L}(\theta) \approx \frac{1}{N} \sum_{n=1}^N \|\mathbf{v}(\mathbf{x}_n(\tau_n), \tau_n; \theta) - (\mathbf{x}_{n1} - \mathbf{x}_{n0})\|^2.$$

In this context, representing discretely sampled functions as vectors allows the optimal velocity field $\mathbf{v}_\theta(\mathbf{x}(\tau), \tau)$ to be modelled using architectures tailored for 1D grids. Because the ordered coordinates of $\mathbf{x}(\tau)$ preserve the spatial or temporal order of the underlying continuous function, deep learning architectures like 1D U-Nets can be efficiently deployed to learn these velocity fields.

This approach provides a principled justification for combining FDA and FM frameworks, demonstrating that Flow Matching can be successfully adapted to functional observations, providing a highly stable, fast, and mathematically rigorous alternative to functional diffusion models. This work provides an implementation for Flow Matching on discretely sampled functional data, interpreting this approach as a finite-dimensional realisation of the Functional Flow Matching framework proposed by [22]. This perspective enables an empirical comparison between stochastic and deterministic generative modelling paradigms for functional data.

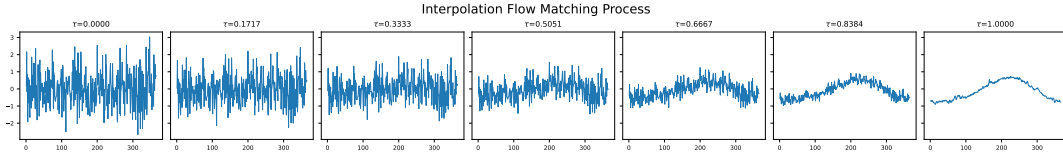


Figure 4.2: Illustration of the Flow Matching linear interpolation paths applied to the AEMET dataset.

Unlike the diffusion framework developed in Chapter 3, which was extended to structured noise processes, the Flow Matching model considered here uses the simplest instantiation of the framework: a linear interpolation with an isotropic Gaussian source distribution. This choice is deliberate: the goal of this work is not to obtain the best-performing or fastest Flow Matching model, but rather to establish a minimal baseline against which the diffusion models of Chapter 3 can be empirically compared. Extending the interpolation path and source distribution to structured analogues is left as a natural direction for future work (see Section 6.1).

CHAPTER 5

Empirical Evaluation

The rigorous theoretical framework for the generative modelling of functional data developed in Chapters 2, 3 and 4 is validated in this chapter by an exhaustive experimental study. The experimental analysis is organised into six experiments of increasing complexity. It begins with a preliminary qualitative example based on real temperature curves to illustrate the potential of generative AI models for functional data. Four experiments on synthetic datasets are then conducted to compare the performance of score-based diffusion and Flow Matching models. Finally, the study concludes with a real-world application to electrocardiogram (ECG) signals. Depending on the objective of each experiment, the evaluation is either qualitative, based on visual inspection of the generated functions, or quantitative, using specific metrics introduced throughout the chapter. All source code developed for this dissertation has been made publicly available in the accompanying GitHub repository: <https://github.com/diegocamarerouam/Generative-AI-Models-for-Functional-Data>.

In all experiments, the empirical comparison is based on two baseline generative frameworks: a standard Variance Preserving diffusion model and a Flow Matching model, both trained under a common experimental setup. In addition, Experiment 5.4 extends this comparison by incorporating two structured diffusion models with circulant matrices as the drift and diffusion coefficients. These models formulate the diffusion process in the frequency domain, allowing the evolution of each frequency component to be explicitly controlled during the forward process, as described in Section 3.3.

On the one hand, all diffusion models considered in this chapter operate under the same Variance Preserving (VP) condition. The isotropic implementation follows the formulation introduced in Sections 3.1 and C.1. Under this framework, the forward SDE governing each functional observation $\mathbf{x}(\tau)$ is

$$d\mathbf{x}(\tau) = -\lambda(\tau) \mathbf{x}(\tau) d\tau + \sqrt{2\lambda(\tau)} d\mathbf{W}(\tau), \quad \mathbf{x}(0) = \mathbf{x}_0,$$

where $\lambda(\tau)$ denotes the linear noise schedule

$$\lambda(\tau) = \lambda_{\min} + \tau (\lambda_{\max} - \lambda_{\min}),$$

whose parameters depend on the corresponding experiment. For each experiment, the noise schedule was selected from all combinations of

$$\lambda_{\min} \in \{0.0005, 0.005, 0.05, 0.5\}, \quad \lambda_{\max} \in \{2.5, 5.0, 7.5, 15.0\},$$

with the best-performing configuration used for the reported results.

Experiment 5.4 additionally considers a circulant extension of this framework, developed in Sections 3.3.3 and C.2, in which the scalar coefficients are replaced by circulant matrices:

$$d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau), \quad \mathbf{x}(0) = \mathbf{x}_0,$$

where $\mathbf{K}(\tau)$ and $\mathbf{S}(\tau)$ are circulant matrices that induce decoupled dynamics in the Fourier domain, as described in Section 3.3. In both standard and extended cases, the reverse process is initialised from a standard multivariate Gaussian distribution, according to the established Variance Preserving condition.

On the other hand, all Flow Matching models rely on a linear interpolation path between the source and target samples:

$$\mathbf{x}(\tau) = (1 - \tau) \mathbf{x}_0 + \tau \mathbf{x}_1, \quad \mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, I_M), \quad \mathbf{x}_1 \sim p_1(\mathbf{x}),$$

where $p_1(\mathbf{x})$ denotes the distribution of the target functions, and the source distribution is a standard multivariate Gaussian. Section 4.2 provides a detailed explanation of this model.

Every generative process involved in the experiments is defined over the continuous time interval $\tau \in [0, 1]$. However, the score function of the diffusion models diverges as $\tau \rightarrow 0$ (see Section 3.4). To ensure numerical stability and avoid this singularity, diffusion models are trained over the restricted interval $\tau \in [t_\varepsilon, 1]$, with $t_\varepsilon = 10^{-3}$. Flow Matching models exhibit no such singularity, since the conditional vector field introduced by the linear interpolation path is constant in τ ; consequently, they are trained and sampled over the full interval $\tau \in [0, 1]$ without restriction.

All models are trained using the Adam optimiser with a learning rate of 10^{-3} and a batch size of 32 functions. The neural network architecture is based on the U-Net proposed in [24], adapted to one-dimensional functional data by replacing two-dimensional convolutions with one-dimensional convolutions. The resulting encoder-decoder architecture, with skip connections between corresponding encoder and decoder resolution levels, receives the time-dependent sample $\mathbf{x}_\tau$ as input. The time variable τ is embedded using Gaussian Random Fourier Features [10] and injected into each layer of the network, allowing the network to condition its output on the diffusion or interpolation time. The network outputs either the score function for diffusion models or the velocity field for Flow Matching while preserving, in both cases, the dimensionality of the input data. For each Flow Matching experiment, several kernel-size configurations were evaluated: (9, 9, 9, 9), (15, 15, 15, 15) and (21, 17, 9, 9), selecting the best-performing architecture for the reported results. The best-performing architecture was then adopted for the diffusion models to ensure a fair comparison.

During inference, Flow Matching models generate samples by numerically integrating the corresponding ODE using the Euler method over the full interval $[0, 1]$. In

contrast, diffusion models solve the reverse-time SDE through the Euler–Maruyama scheme over the restricted time interval $[1, t_\varepsilon]$, followed by a final Tweedie denoising step (see Appendix D.5) to compensate for the truncated training interval. For experiments involving quantitative distributional evaluation, each model generates the same number of samples as contained in the corresponding reference dataset, denoted as

$$\hat{\mathcal{D}} = \{\hat{\mathbf{x}}_i\}_{i=1}^I = \{\hat{\mathbf{x}}_i = (\hat{x}_i(t_1), \dots, \hat{x}_i(t_M))\}_{i=1}^I.$$

This ensures a fair comparison when computing distributional metrics.

5.1. Preliminary Experiment: Real Temperature Curves

As an initial qualitative example, this section presents the application of the proposed generative models to a real-world dataset of temperature curves, which has already been presented in Section 2.1. Unlike the subsequent experiments, which evaluate specific properties of the generative models, this preliminary experiment aims to provide an initial qualitative assessment of the generated functions.

This dataset shows the evolution of the daily temperature at 73 Spanish weather stations over the period 1980–2009. The data were originally collected by the Agencia Estatal de Meteorología (AEMET) and retrieved using the `fetch_aemet` function available in the `scikit-fda` Python library. The dataset consists of $N = 73$ functional observations $\mathbf{x}(t)$ represented by $M = 365$ daily observations on the interval $t \in [1, 365]$, corresponding to the mean daily temperature over one calendar year. The dataset is then normalised using the following formulas:

$$\widetilde{\mathbf{x}}_n = (\widetilde{x}_n(t_1), \dots, \widetilde{x}_n(t_M)), \quad \widetilde{x}_n(t_m) = \frac{x_n(t_m) - \mu_{\mathcal{D}}}{\sigma_{\mathcal{D}}}$$

where the dataset mean and the standard deviation are computed as:

$$\begin{aligned} \mu_{\mathcal{D}} &= \frac{1}{N} \sum_{n=1}^N \frac{1}{M} \sum_{m=1}^M x_n(t_m) = \frac{1}{N M} \sum_{n=1}^N \sum_{m=1}^M x_n(t_m) \\ \sigma_{\mathcal{D}} &= \sqrt{\frac{1}{N \cdot M - 1} \sum_{n=1}^N \sum_{m=1}^M (x_n(t_m) - \mu_{\mathcal{D}})^2}. \end{aligned}$$

Both models are trained for 10000 epochs, requiring approximately 30 minutes. For the diffusion model, the linear noise schedule is defined by $\lambda_{\min} = 0.05$ and $\lambda_{\max} = 5.0$. The kernel-size configuration selected for this experiment is $(21, 17, 9, 9)$.

Figure 5.1 compares the real dataset with the corresponding temperature curves generated by each model. Both models successfully reproduce representative samples from the original dataset and capture its two main clusters of temperature curves. However, they struggle to reproduce the distribution of outliers, as reflected by the lower-temperature curves. This may be related to the relatively small number of observations in this region of the dataset. Although the Flow Matching model appears

to capture the distribution of outliers slightly better, no remarkable differences are observed between the two models.

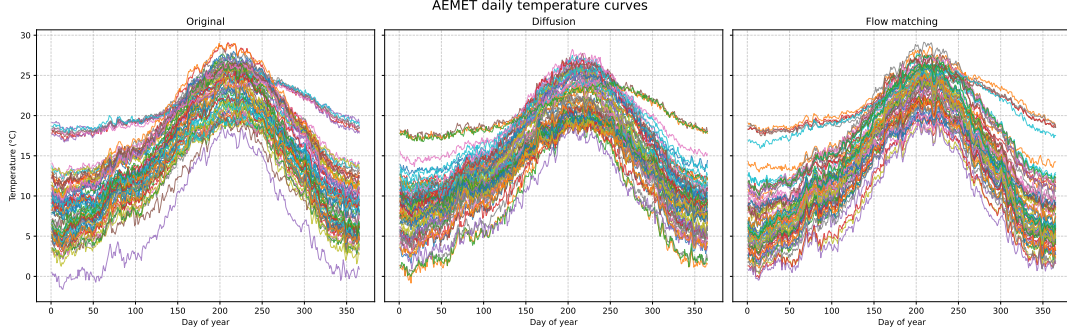


Figure 5.1: Comparison of real and generated AEMET temperature curves.

Figure 5.2 illustrates the generation process for both models, which consists of transforming an initial noisy sample into a synthetic temperature curve. While the diffusion model is visualised using a logarithmic time scale, the Flow Matching model uses a linear time scale, reflecting the different dynamics of their respective generation processes. This convention is maintained throughout the remainder of the chapter.

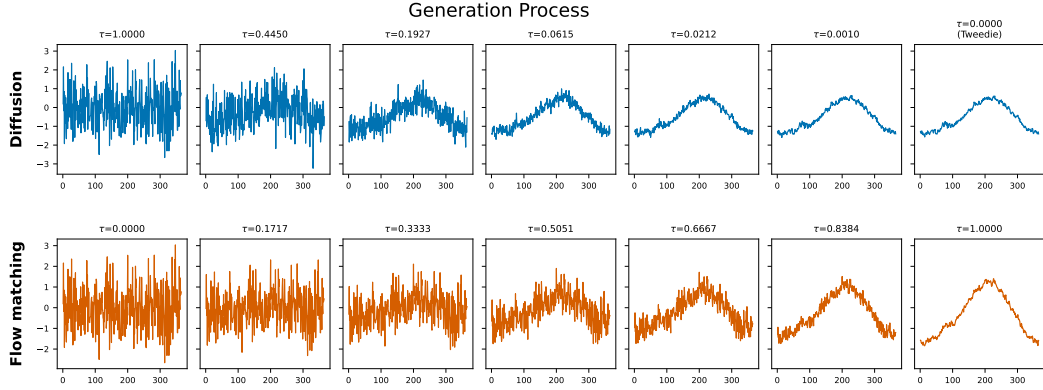


Figure 5.2: Generation process for the diffusion and Flow Matching models.

5.2. Experiment 1: Constant Functions

The objective of this first synthetic experiment is to evaluate and compare the basic generative capabilities of the two baseline frameworks addressed in this work: score-based diffusion models and Flow Matching models. This experiment serves as a simple introductory example to verify that both models can generate functions without residual noise, while reproducing the underlying distribution of function values.

5.2.1. Constant Uniform Dataset

This experiment uses a synthetic dataset consisting of $N = 2000$ functional observations $\mathbf{x}(t)$ defined on the interval $t \in [0, 2\pi]$. In this case, the continuous domain is discretised into $M = 128$ equally spaced evaluation points. Denoting the discretisation grid by $t = (t_1, \dots, t_M)$, the dataset is represented as

$$\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N = \{\mathbf{x}_n = (x_n(t_1), \dots, x_n(t_M))\}_{n=1}^N.$$

The dataset consists of constant functions $\mathbf{x}(t)$ whose constant value is uniformly sampled over the continuous vertical range $[-1, 1]$. Since each function is constant, using the established notation, it follows that $x_n(t_i) = x_n(t_j)$ for all $i, j \in \{1, \dots, M\}$. Consequently, the representation of the dataset simplifies to

$$\mathcal{D} = \{\mathbf{x}_n = c_n\}_{n=1}^N, \quad c_n \sim \mathcal{U}(-1, 1).$$

5.2.2. Generative Models

The common experimental setup described at the beginning of this chapter is adopted. Both models are trained for 1000 epochs, requiring approximately 17 minutes. The diffusion model employs a linear noise schedule defined by $\lambda_{\min} = 0.05$ and $\lambda_{\max} = 2.5$. The kernel-size configuration selected is $(15, 15, 15, 15)$.

5.2.3. Quality Metrics

To evaluate the quality of the samples generated by both models, two different metrics are introduced. The first metric is the mean squared error (MSE) of each function $\hat{\mathbf{x}}_i$ with respect to its empirical mean $\hat{c}_i$. Then, the MSE associated with the generated sample $\hat{\mathbf{x}}_i$ is

$$\text{MSE}_i = \frac{1}{M} \sum_{m=1}^M (\hat{x}_i(t_m) - \hat{c}_i)^2, \quad \hat{c}_i = \frac{1}{M} \sum_{m=1}^M \hat{x}_i(t_m).$$

Because the target functions are perfectly constant, any variation in a generated function is due to noise. This metric, therefore, measures the amount of high-frequency noise remaining in each generated function. The second metric evaluates the global structure of the set of generated samples rather than the quality of each function individually. Specifically, it analyses the distribution of the mean values of the functions, denoted by the empirical set $\mathcal{C} = \{\hat{c}_i\}_{i=1}^I$. Ideally, a perfect generative model would recover the reference uniform distribution $\mathcal{U}(-1, 1)$. To quantify the discrepancy between the empirical distribution of the generated means and the true reference distribution, the first Wasserstein distance ($p = 1$) is employed:

$$W_1(\mathcal{C}, \mathcal{U}(-1, 1)) = \int_{\mathbb{R}} |F_{\mathcal{C}}(x) - F_{\mathcal{U}}(x)| dx.$$

where $F_{\mathcal{C}}$ and $F_{\mathcal{U}}$ denote the corresponding cumulative distribution functions (CDF). In practice, this distance is evaluated using the `wasserstein_distance` function from the `scipy.stats` scientific library.

5.2.4. Results

Figure 5.3 compares the real constant functions with those generated by the diffusion and Flow Matching models. Unlike the temperature curves experiment, noticeable differences can be observed in this case. First, the generated functions are not perfectly constant, particularly for the diffusion model, where the noise component is clearly visible. In the Flow Matching model, the noise component is less pronounced, although it is still noticeable near the edges of the domain. Another noticeable difference is observed in the distribution of the generated samples. The diffusion model tends to generate functions slightly below the centre of the interval $[-1, 1]$, whereas both the real dataset and the Flow Matching model exhibit a distribution closer to a uniform one. Overall, the Flow Matching model appears to produce higher-quality samples.

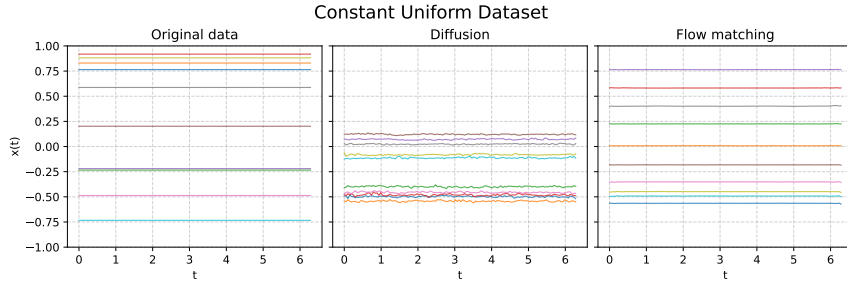


Figure 5.3: Comparison of real and generated constant functions.

Figure 5.4 confirms the observations described above. The MSE obtained by the Flow Matching model is two orders of magnitude lower than that of the diffusion model. Moreover, the Wasserstein distances, together with the corresponding histograms of the function means, show that the Flow Matching model produces a distribution of function means that is closer to that of the real data. Nevertheless, both models struggle to reproduce the boundaries of the reference interval, as they generate samples outside $[-1, 1]$. This behaviour can be explained by the fact that deviations in the central region are compensated by the closest samples, whereas no such compensation exists near the boundaries. The quality-metrics values are summarised in Table 5.1.

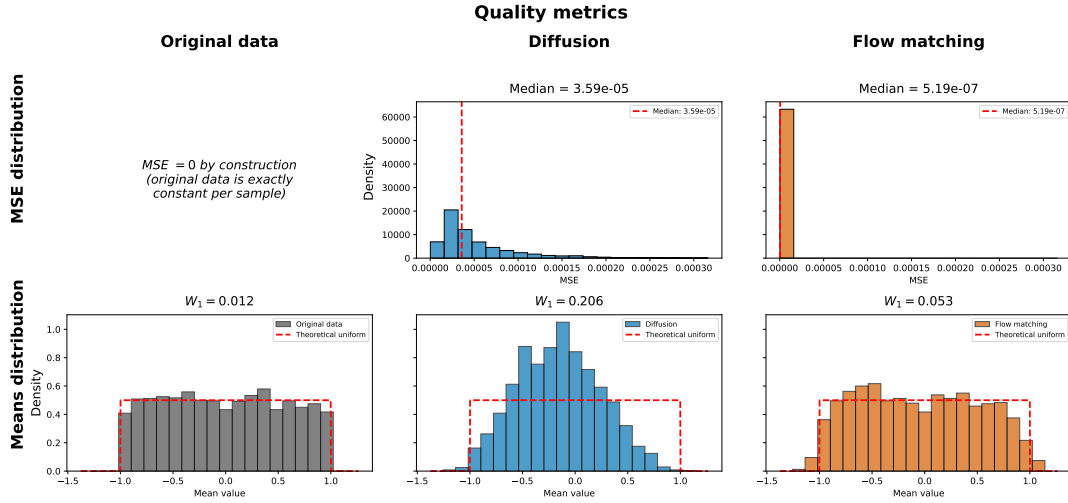


Figure 5.4: Quality metrics for the real dataset and the generated samples.

Table 5.1: Comparison of quality metrics for the generated constant functions.

Metric	Original Data	Diffusion	Flow Matching
MSE (median)	0	3.59×10^{-5}	5.19×10^{-7}
W_1 distance	0.012	0.206	0.053

5.3. Experiment 2: Damped Oscillatory Functions

This experiment follows the same experimental setup as Experiment 5.2. However, the target functions in this scenario exhibit higher geometric complexity, providing a more challenging benchmark for the generative models framework.

5.3.1. Exponentially Decaying Oscillatory Dataset

The dataset follows the same overall structure as that used in Experiment 5.2, which is $N = 2000$ and $M = 128$, but each observation is now an exponentially decaying oscillatory function defined by:

$$\mathbf{x}(t) = a + b e^{-\alpha t} \cos(\omega t + \phi), \quad t \in [0, 2\pi],$$

where the underlying functional parameters are independently sampled from their respective uniform reference distributions:

$$a \sim \mathcal{U}(-1, 1), \quad b \sim \mathcal{U}(-1, 1), \quad \alpha \sim \mathcal{U}(0.1, 1), \quad \omega \sim \mathcal{U}(2, 4), \quad \phi \sim \mathcal{U}\left(-\frac{\pi}{2}, \frac{\pi}{2}\right).$$

5.3.2. Generative Models

The generative models are identical to those employed in Experiment 5.2, including the number of epochs and the noise schedule. However, the kernel-size configuration that showed better results this time is (9, 9, 9, 9).

5.3.3. Quality Metrics

As in Experiment 5.2, two complementary metrics are introduced to evaluate the performance of the trained models. However, because the target data is defined by an underlying parametric structure, each generated synthetic sample $\hat{\mathbf{x}}_i$ must first be fitted to the reference parametric model to perform an accurate parameter-level comparison:

$$(5.1) \quad f(t, \boldsymbol{\theta}) = a + b e^{-\alpha t} \cos(\omega t + \phi), \quad \boldsymbol{\theta} = (a, b, \alpha, \omega, \phi)$$

Therefore, each generated sample $\hat{\mathbf{x}}_i$ will be fitted to the parametric model by solving the following least squares problem

$$\mathbf{x}_i^* = f(t, \boldsymbol{\theta}_i^*), \quad \boldsymbol{\theta}_i^* = \arg \min_{\boldsymbol{\theta}} \frac{1}{M} \sum_{m=1}^M (\hat{\mathbf{x}}_i(t_m) - f(t_m, \boldsymbol{\theta}))^2.$$

The optimisation problem is solved using the Adam optimiser.

Once each generated sample $\hat{\mathbf{x}}_i$ is fitted to the parametric model as $\mathbf{x}_i^*$, distributional deviations are quantified. First, for each generated function, the residual mean square error (MSE) between the raw generated sample $\hat{\mathbf{x}}_i$ and its corresponding parametric fit $\mathbf{x}_i^*$ is computed:

$$\text{MSE}_i = \frac{1}{M} \sum_{m=1}^M (\hat{\mathbf{x}}_i(t_m) - \mathbf{x}_i^*(t_m))^2.$$

This metric quantifies, individually for each sample, the ability of the trained model to generate a new functional observation that can be explained by the reference parametric model. It measures the high-frequency generative noise present in the sample.

The second metric evaluates each parameter distribution independently. To quantify the difference between the empirical parameter distributions and their true uniform reference distributions, the first Wasserstein distance ($p = 1$) is utilised. Given that the parametric space is five-dimensional, this distance is computed independently for each parameter component:

$$W_1(\mathcal{C}_\gamma, \mathcal{U}_\gamma) = \int_{\mathbb{R}} |F_{\mathcal{C}_\gamma}(x) - F_{\mathcal{U}_\gamma}(x)| dx,$$

where $\gamma \in \{a, b, \alpha, \omega, \phi\}$, $\mathcal{C}_\gamma$ represents the empirical distribution of the estimated parameter component and $\mathcal{U}_\gamma$ denotes the corresponding reference uniform distribution. As in the constant functions experiment, these values are evaluated computationally using the `wasserstein_distance` function from the `scipy.stats` library.

5.3.4. Results

Figure 5.5 compares the real and generated exponentially decaying oscillatory functions used in this experiment. Both models appear to reproduce the general structure of these functions, as both generate decaying oscillatory signals. As in the constant functions experiment (Experiment 5.2), the samples generated by the diffusion model exhibit a noticeable amount of noise, whereas the Flow Matching model produces considerably smoother samples.

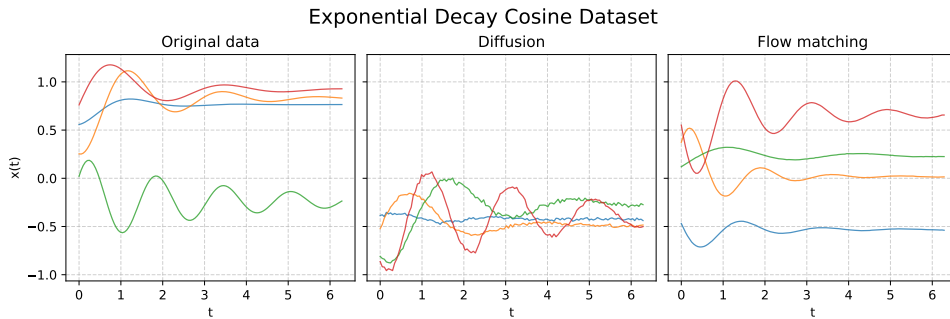


Figure 5.5: Comparison of real and generated exponentially decaying oscillatory functions.

Figure 5.6 confirms that the Flow Matching model performs noticeably better than the diffusion model, with an MSE two orders of magnitude lower. In these terms, the Flow Matching model generates samples that are indistinguishable from real data.

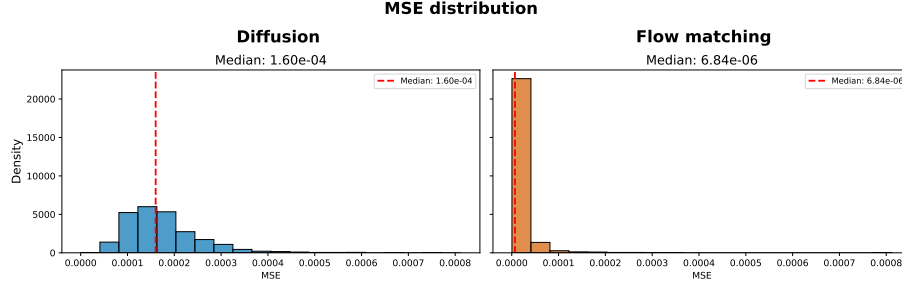


Figure 5.6: Comparison of the MSE for the real dataset and the generated samples.

Figure 5.7 presents a detailed analysis of the parameter distributions for each model. It also includes the distributions obtained by fitting the original dataset to the parametric model (5.1), distinguishing the fitting stage errors from those attributable to the generative models. Overall, the estimated parameter distributions closely match those of the original dataset. The phase parameter ϕ stands out, as its distribution is less accurately represented by the parametric fitting procedure than those of the remaining parameters. The diffusion model struggles to reproduce the distribution of parameter a , exhibiting the same centralisation effect observed in the constant functions experiment, whereas the Flow Matching model reproduces this distribution much more accurately. Regarding the phase parameter ϕ , both models achieved a similar performance to the fitted reference data, suggesting that their generated samples exhibit a similar distribution for this parameter. For the remaining parameters, both models perform similarly and slightly less accurately than the fitting model.

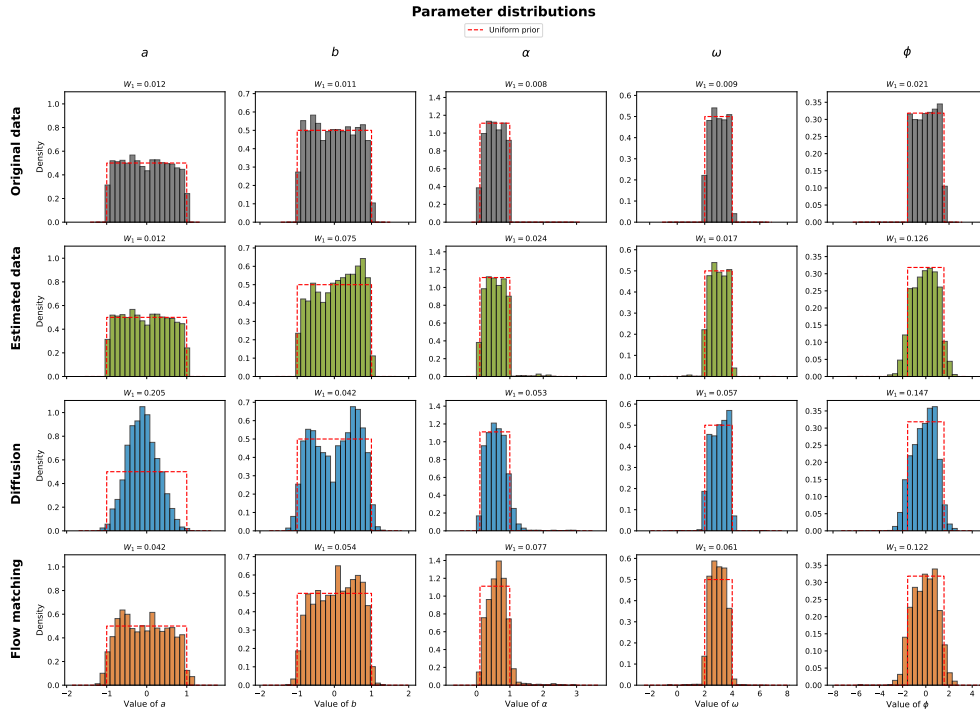


Figure 5.7: Comparison of the Wasserstein distance for the parameter distributions.

Overall, both models produce parameter distributions similar to those obtained from the fitted reference data. However, the Flow Matching model clearly outperforms the diffusion model in terms of the MSE, producing samples that more closely resemble the real functions.

5.4. Experiment 3: Frequency-Selective Diffusion

The previous experiments share a common limitation: each point of the function evolves independently of the others. In diffusion models, this independence is given by the scalar drift and diffusion coefficients. In Flow Matching, the independence arises because the interpolation coefficients $\alpha(\tau)$ and $\beta(\tau)$ are scalar functions. In both cases, no global structure is considered. However, functional observations are generally assumed to be continuous or smooth, meaning that nearby points are strongly correlated.

This property motivates the circulant diffusion framework developed in Chapter 3. The theoretical framework for these models was developed in Section 3.2. In particular, by choosing circulant drift and diffusion matrices, the evolution of different frequency components can be controlled independently during the forward diffusion process, as shown in Appendix A. Sections 3.3 and 3.4 provide the theoretical and training frameworks for circulant diffusion models. This experiment compares the baseline standard diffusion and Flow Matching models with two circulant diffusion models. One model prioritises the attenuation of low-frequency components during the forward process, whereas the other first attenuates the high-frequency components.

Unlike the previous experiments, the objective is not only to reproduce the target distribution but also to investigate whether frequency-selective noise injection influences the quality of the samples generated.

5.4.1. Double-Frequency Cosine Dataset

The dataset proposed for this experiment consists of $N = 2000$ functions drawn according to the following parametric model:

$$\mathbf{x}(t) = \cos(2\pi t + \phi) + 0.3 \cos(14\pi t + \phi), \quad t \in [0, 1], \quad \phi \sim \mathcal{U}(0, 2\pi),$$

where the interval $[0, 1]$ is discretised in $M = 128$ points. This parametric model is particularly well suited to the objective of the experiment, since each function has a low and a high frequency cosine component. The low frequency component has frequency $\omega_{\text{low}} = 1$, whereas the high frequency component has frequency $\omega_{\text{high}} = 7$. The corresponding amplitudes are $A_{2\pi} = 1.0$ and $A_{14\pi} = 0.3$, respectively.

5.4.2. Generative Models

The baseline diffusion and Flow Matching models are trained using the same configuration as in Experiment 5.3, which consists of 1000 training epochs, a kernel-

size configuration of $(9, 9, 9, 9)$, and a diffusion noise schedule with $\lambda_{\min} = 0.05$ and $\lambda_{\max} = 2.5$.

In addition, two structured diffusion models with circulant drift and diffusion coefficients are evaluated. The first employs a schedule in which the low-frequency components are attenuated before the high-frequency components, whereas the second adopts the opposite strategy. A detailed explanation of this frequency-selective diffusion is provided in Appendix E. Both models are trained for 1000 epochs and employ the same experimental setup as the baseline models.

5.4.3. Quality Metrics

To evaluate the performance of these generative models, three complementary metrics are considered. Following the methodology introduced in Experiment 5.3, each generated sample is first fitted to the underlying parametric model by solving:

$$\mathbf{x}_i^* = f(t, \phi_i^*), \quad \phi_i^* = \arg \min_{\phi} \frac{1}{M} \sum_{m=1}^M (\hat{\mathbf{x}}_i(t_m) - f(t_m, \phi))^2,$$

where the parametric model is now defined as

$$f(t, \phi) = \cos(2\pi t + \phi) + 0.3 \cos(14\pi t + \phi), \quad \phi \sim \mathcal{U}(0, 2\pi).$$

After fitting each generated sample to the defined parametric model, the performance of the trained models is evaluated. Firstly, as in the second experiment with damped oscillatory functions, the residual mean square error (MSE) between the generated sample $\hat{\mathbf{x}}_i$ and the adjusted one $\mathbf{x}_i^*$ is computed to evaluate the quality of each generated sample individually:

$$\text{MSE}_i = \frac{1}{M} \sum_{m=1}^M (\hat{\mathbf{x}}_i(t_m) - \mathbf{x}_i^*(t_m))^2.$$

This metric quantifies the residual fitting error and the capacity of the trained model to learn the parametric distribution. The second metric is again the Wasserstein distance. In this case, it measures the discrepancy between the empirical adjusted phases' distribution and the reference one:

$$W_1(\mathcal{C}_\phi, \mathcal{U}(0, 2\pi)) = \int_{\mathbb{R}} |F_{\mathcal{C}_\phi}(x) - F_{\mathcal{U}}(x)| dx,$$

where $\mathcal{C}_\phi$ denotes the corresponding empirical phase distribution.

Furthermore, in order to better illustrate the global structure of the set of generated samples and study any possible discrepancy between the low- and high-frequency components of the sinusoidal functions, an additional metric is introduced to assess whether this strategy affects the amplitudes of the low- and high-frequency components differently. This metric evaluates the mean and standard deviation of the

empirical distributions of $\hat{a}_{2\pi}, \hat{a}_{14\pi}$ separately. Therefore, to obtain the value of each amplitude for each individual function, a new optimisation problem is introduced:

$$(a_{2\pi,i}^*, a_{14\pi,i}^*) = \arg \min_{a_{2\pi}, a_{14\pi}} \frac{1}{M} \sum_{m=1}^M (\hat{\mathbf{x}}_i(t_m) - g_i(t_m, a_{2\pi}, a_{14\pi}))^2,$$

with the following parametric model associated with the generated sample $\hat{\mathbf{x}}_i$:

$$g_i(t, a_{2\pi}, a_{14\pi}) = a_{2\pi} \cos(2\pi t + \phi_i^*) + a_{14\pi} \cos(14\pi t + \phi_i^*), \quad a_{2\pi}, a_{14\pi} \geq 0,$$

where ϕ_i^* was computed in the previous optimisation problem. This optimisation problem is again implemented using an Adam optimiser.

5.4.4. Results

For simplicity, the model in which the low-frequency components decay more rapidly is referred to as the *low-frequency model*, whereas the model in which the high-frequency components decay more rapidly is referred to as the *high-frequency model*. Figure 5.8 compares the real functions with those generated by the standard diffusion, frequency-selective diffusion, and Flow Matching models. All models appear to reproduce the main structure of the data, as the generated functions closely resemble the real ones. A closer inspection reveals a higher level of high-frequency noise in the standard and high-frequency diffusion models, whereas both the low-frequency diffusion and Flow Matching models generate noticeably smoother functions, particularly around the waveform peaks.

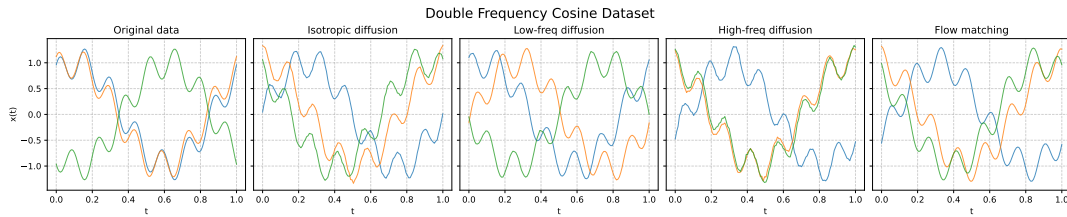


Figure 5.8: Comparison of real and generated double-frequency cosine functions

Figure 5.9 illustrates the forward process for the diffusion and Flow Matching models. The effect of frequency-selective diffusion is clearly visible. The low-frequency diffusion rapidly loses its oscillatory structure, while the high-frequency diffusion preserves it until the very end of the process. The standard diffusion model exhibits an intermediate behaviour between the two. Figure 5.10 illustrates the corresponding reverse (generation) process.

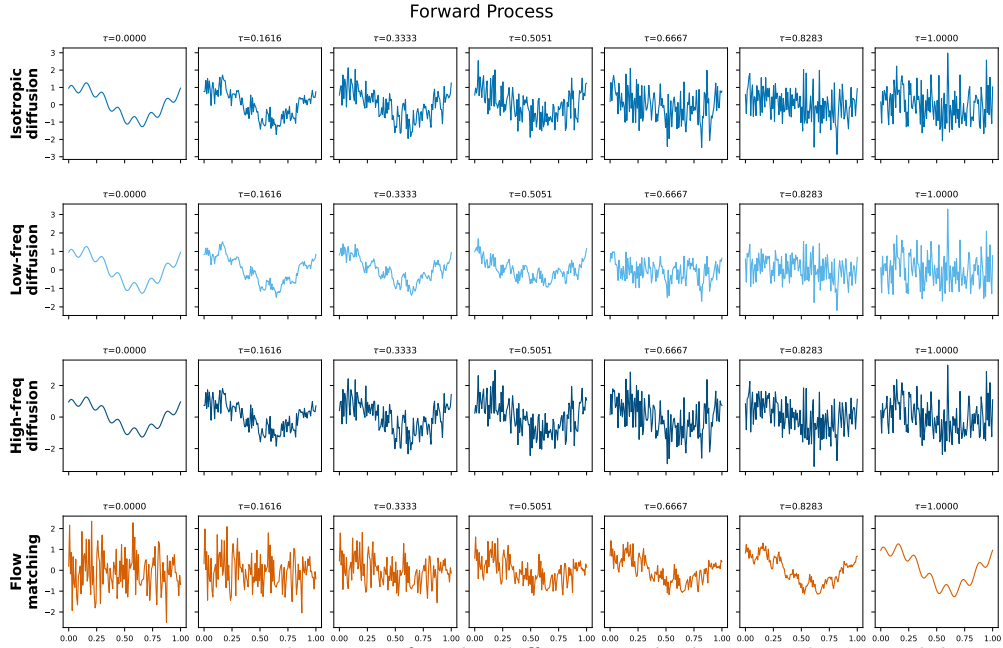


Figure 5.9: Forward process for the diffusion and Flow Matching models.

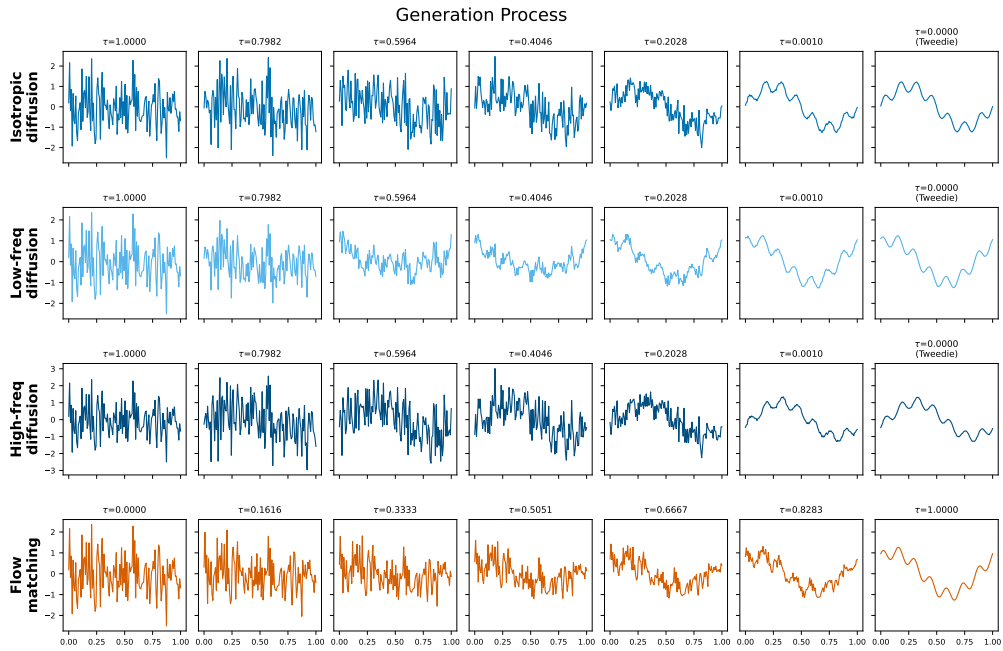


Figure 5.10: Generation process for the diffusion and Flow Matching models.

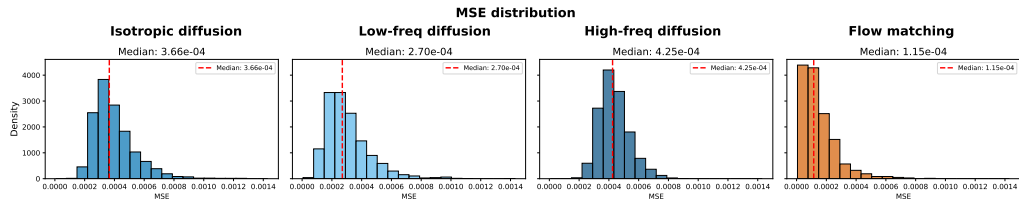


Figure 5.11: Comparison of the MSE for the real dataset and the generated samples.

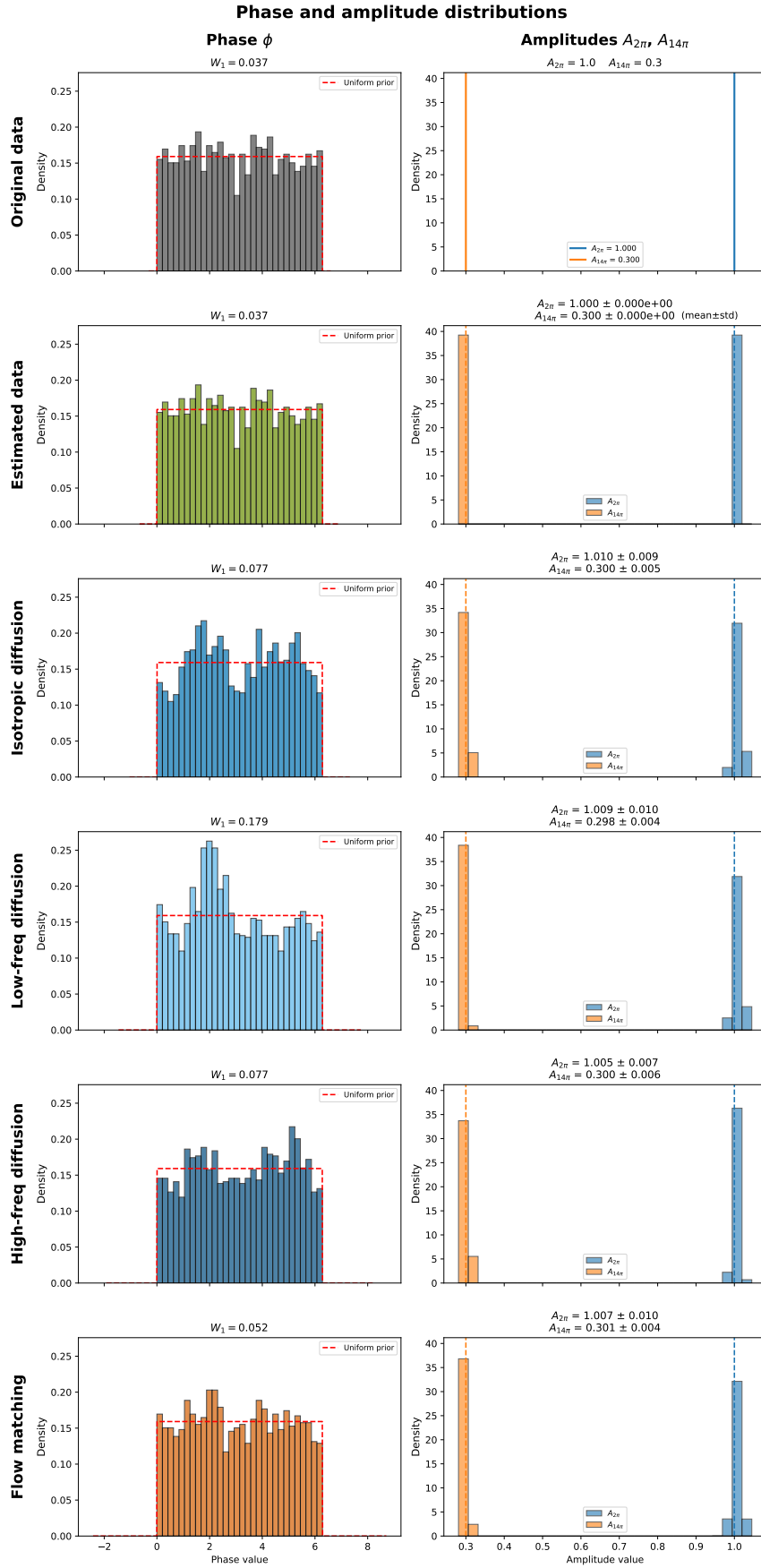


Figure 5.12: Comparison of the phase and amplitudes distributions.

Figure 5.11 confirms the qualitative observations from Figure 5.8. The Flow Matching model achieves the lowest MSE, followed by the low-frequency, standard, and high-frequency diffusion models. However, the differences are less pronounced than in the previous experiments, as all models achieve MSE values of the same order of magnitude.

Figure 5.12 provides a detailed analysis of the phase distribution and amplitude statistics. As in the damped oscillatory functions experiment (Section 5.3), the estimated data parameters are also included to enable a fair comparison among the generative models.

Table 5.2: Comparison of phase and amplitude distributions for different generative models.

Model	$W_1(\phi)$	$A_{2\pi}$ (mean $\pm$ std)	$A_{14\pi}$ (mean $\pm$ std)
Original data	0.037	1.000 ± 0.000	0.300 ± 0.000
Estimated data	0.037	1.000 ± 0.000	0.300 ± 0.000
Standard diffusion	0.077	1.010 ± 0.009	0.300 ± 0.005
Low-frequency diffusion	0.179	1.009 ± 0.010	0.298 ± 0.004
High-frequency diffusion	0.077	1.005 ± 0.007	0.300 ± 0.006
Flow matching	0.052	1.007 ± 0.010	0.301 ± 0.004

The results in Table 5.2 show that all methods accurately reproduce the amplitude statistics, whereas larger differences are observed in the phase distribution. The estimated data provides a close reference to the original data, allowing to attribute all discrepancies to the generative models.

Among the generative models, Flow Matching provides the best performance on the phase distribution while also achieving competitive accuracy in the amplitude estimates. The standard and high-frequency diffusion models exhibit identical performance on the phase distribution, while the high-frequency model reproduces the amplitude values slightly more accurately than the standard diffusion model, and even slightly better than the Flow Matching model. In contrast, the low-frequency diffusion model performs substantially worse in both metrics, particularly in terms of phase distribution.

Overall, all models reproduce the main characteristics of the target data. The Flow Matching model achieves the lowest MSE, whereas all methods exhibit comparable performance when compared with the parameter distributions of the original data. Although frequency-selective noise injection substantially modifies both the forward diffusion and the generation processes, these differences are not reflected in the final sample quality for this dataset.

5.5. Experiment 4: Skyline Functions

This final synthetic experiment evaluates the ability of the proposed generative models to reproduce piecewise constant functions containing sharp discontinuities. Since the

objective of this experiment is primarily qualitative, the evaluation simplifies to a visual analysis of the generated shapes rather than traditional parametric models and metrics.

5.5.1. Piecewise Constant Skyline Dataset

The dataset compiled for this experiment consists of $N = 2000$ piecewise constant functions, defined over $t \in [0, 2\pi]$, discretised into $M = 128$ points. Since the aim of this experiment is to generate functions whose overall shape resembles a skyline, the dataset is constructed to ensure that each function exhibits exactly 15 discontinuities restricted within the vertical interval $[0, 1]$.

Furthermore, to impose a realistic architectural constraint on the dataset, two restrictions are applied: the first restriction is that the height for the leftmost and rightmost constant segments must be in the interval $[0, \frac{1}{3}]$, effectively simulating the horizon view at the edges. The second restriction is that each constant segment must span at least 5% of the total domain width, ensuring that the functions do not contain excessively narrow towers or spikes.

5.5.2. Generative Models

The baseline standard diffusion and Flow Matching models are trained for 1000 epochs using the common experimental setup described previously, with a kernel-size configuration of $(9, 9, 9, 9)$ and a linear noise injection schedule with $\lambda_{\min} = 0.05$ and $\lambda_{\max} = 2.5$.

5.5.3. Results

Figure 5.13 compares representative real and generated functions for the skyline dataset. The most noticeable difference is the amount of noise present in the diffusion-generated samples, highlighting the difficulty of the diffusion model in reproducing sharp discontinuities and constant functions. Although the Flow Matching model performs considerably better, it also struggles to reproduce the high-frequency and non-smooth features of the original dataset, particularly the vertical jumps. Nevertheless, both models successfully reproduce the two constraints introduced to produce more realistic skyline samples: the lower height boundaries and the minimum building width. As a result, both models generate functions that resemble realistic skylines. Moreover, from a different perspective, the high-frequency noise present in the diffusion-generated samples could also be interpreted as high-level details of the skyline.

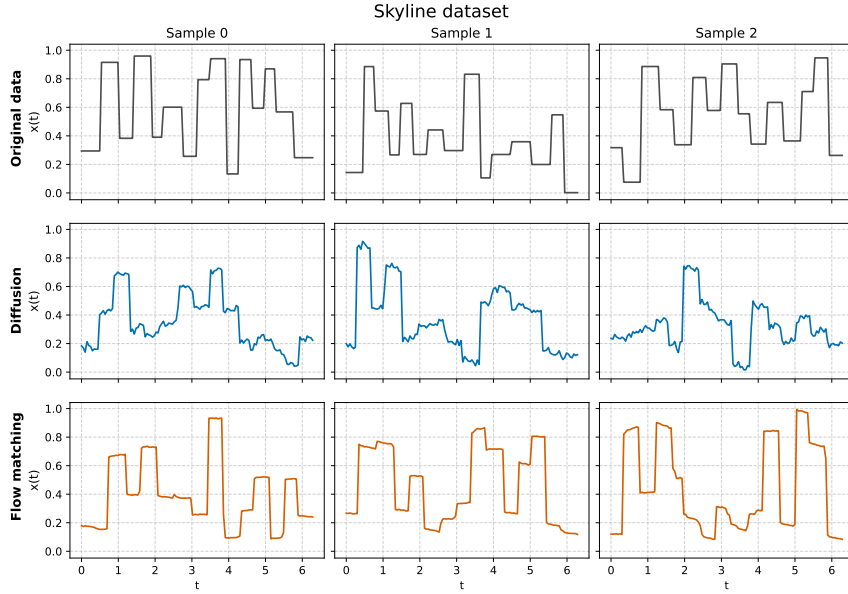


Figure 5.13: comparison of real and generated skyline functions.

5.6. Experiment 5: Real Electrocardiogram Signals

This final experiment evaluates the proposed generative models on a real-world functional dataset of electrocardiogram (ECG) recordings. Unlike the synthetic datasets considered in the previous experiments, ECG signals arise from a real physiological process and therefore do not admit a known parametric representation. This makes them an appropriate benchmark for validating the proposed generative models in realistic scenarios.

5.6.1. Electrocardiogram Dataset

The dataset is derived from the electrocardiogram (ECG) recordings in the PTB-XL database [25], using the Python module `wfdb`.

Although the PTB-XL database contains recordings corresponding to several diagnostic classes, including myocardial infarction and hypertrophy, only ECG recordings labelled as normal are considered in this experiment. This selection ensures that the variability observed in the generated samples reflects the performance of the generative models rather than differences arising from underlying cardiac conditions. Each recording originally has a duration of 10 seconds and was sampled at a frequency of 100 Hz, resulting in 1000 observations per signal. For visualisation purposes, only the first 5 seconds of each sample are considered, corresponding to the first 500 sampling points. This preprocessing yields a dataset of $N = 9246$ functional observations $\mathbf{x}(t)$ represented by $M = 500$ sampling points on the interval $t \in [0, 5]$, representing the signal evolution over time. Moreover, the dataset is normalised using the same formulas as in the temperature-curves experiment 5.1.

5.6.2. Generative Models

The diffusion model employs a linear noise schedule with $\lambda_{\min} = 0.05$ and $\lambda_{\max} = 5.0$, whereas Flow Matching does not require this additional schedule. This is the same noise schedule used in the preliminary temperature-curves experiment. Both models are trained for 500 epochs during 35 minutes. The kernel-size configuration selected for this experiment differs from the temperature-curves experiment and coincides with the one employed over all the synthetic experiments (except Experiment 5.2), which is (9, 9, 9, 9).

5.6.3. Quality Metrics

Unlike the synthetic datasets considered previously, ECG signals do not admit a known parametric representation. Nevertheless, several physiological characteristics can be computed and used to assess the realism of the generated signals. Since this work focuses on generative modelling rather than clinical analysis, only a small number of standard ECG components are considered. These components are illustrated in Figure 5.14. The relevant concepts are:

- **R-peak:** the highest-amplitude point of each cardiac cycle, detected using the `wfdb.processing.xqrs_detect` algorithm.
- **RR interval (seconds):** the time elapsed between two consecutive R-peaks.
- **Heart rate:** number of times the heart beats per minute (bpm), computed from the RR intervals.

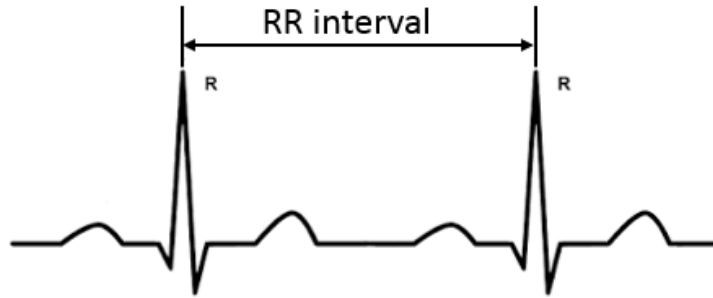


Figure 5.14: Illustration of the main components of an ECG signal.

The generated signals are evaluated using the following physiological metrics:

- **R-peak detection success rate:** reports the percentage of ECGs for which the detector successfully identifies a plausible sequence of R-peaks.
- **Mean RR interval and standard deviation:** reports the mean and standard deviation of the RR intervals for each ECG signal.
- **Average heart rate:** reports the average heart rate estimated for each ECG signal.

A detected sequence of R-peaks is considered plausible if it contains at least three detected peaks and all instantaneous heart rates lie between 30 and 240 bpm. Since each recording has a duration of five seconds, these physiological limits correspond to RR intervals between 0.25 and 2 seconds. The average heart rate (HR) is computed as

$$\text{HR} = \frac{60}{\text{RR}_{\text{mean}}}.$$

For each ECG signal, the mean RR interval, the RR interval standard deviation, and the average heart rate are first computed. Their empirical distributions are then summarised by their sample mean and standard deviation, while the distributional discrepancy between the real and generated datasets is quantified through the first Wasserstein distance.

5.6.4. Results

Figure 5.15 compares the real and generated ECG signals. As a first qualitative assessment, both the diffusion and Flow Matching models appear to capture the global structure of the signals, generating samples with several R-peaks and intermediate lower-amplitude components. In this case, it is difficult to assess the noise level of the generated samples, since the original signals already contain a noticeable noise component. A more relevant aspect to analyze is the periodicity of the heart rate, as it is essential to generate realistic ECG signals.

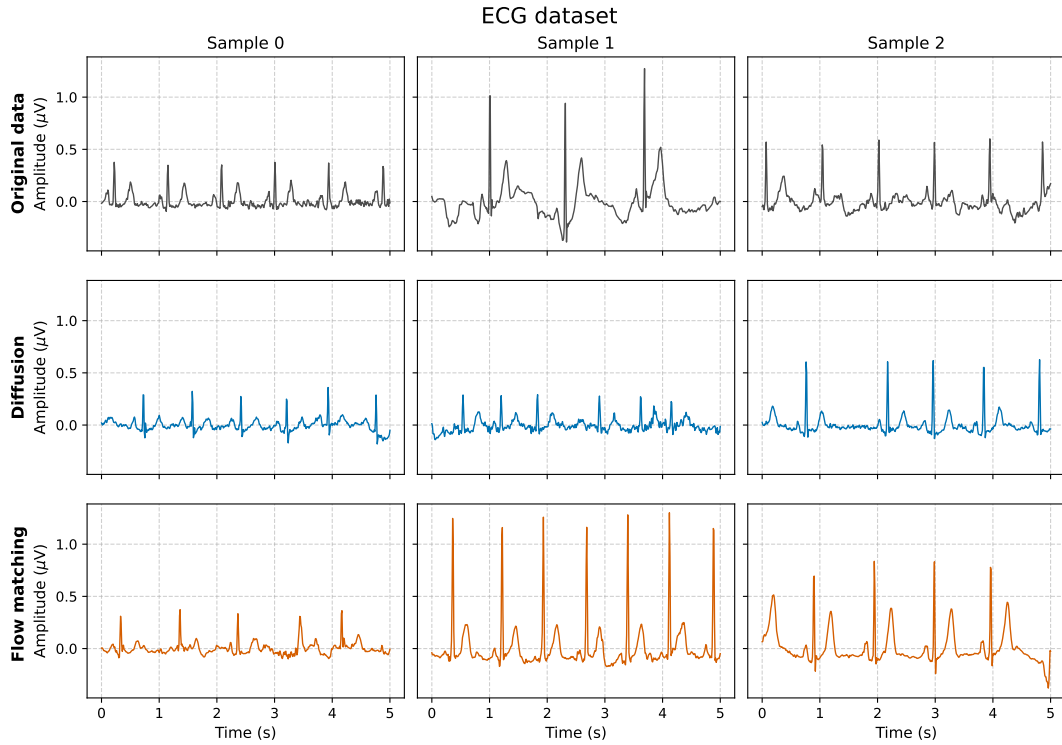


Figure 5.15: Comparison of real and generated ECG signals.

Figure 5.16 provides a quantitative comparison of the generative models, with the corresponding numerical results summarised in Table 5.3. The first metric is the *R-peak detection success rate*. Flow Matching achieves a slightly higher success rate than diffusion, indicating that it generates valid ECG signals more consistently. However, the diffusion model produces distributions of all three physiological parameters closer to those of the original data. This difference is particularly evident for the *heart rate*, where Flow Matching tends to generate signals with lower heart rates, resulting in a higher mean RR interval. These results indicate that, while Flow Matching produces valid ECG signals more consistently, diffusion produces samples with physiological properties closer to those observed in the real data.

Both generative models also exhibit room for improvement in reproducing the variability of the RR intervals. Specifically, both models overestimate the standard deviation of the RR intervals, suggesting that the generated R-peaks are less regularly distributed than those in the original dataset.

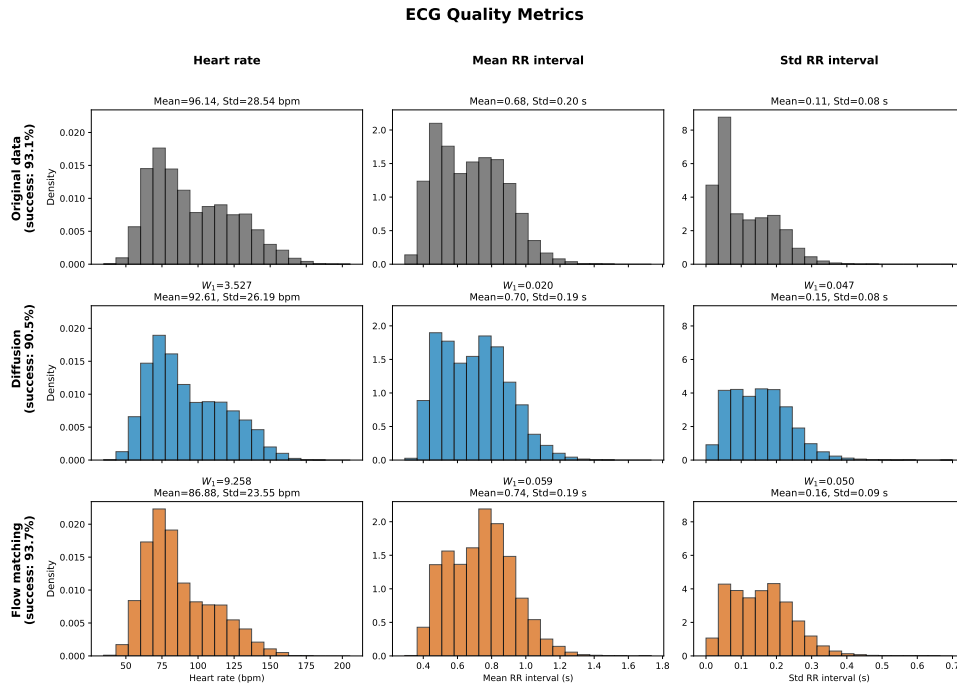


Figure 5.16: Comparison of physiological metrics for the real dataset and the generated samples.

Table 5.3: Comparison of ECG quality and physiological parameter distributions.

Model	Success (%)	Heart rate Mean $\pm$ Std	Mean RR interval Mean $\pm$ Std	Std RR interval Mean $\pm$ Std
Original data	93.1	96.14 $\pm$ 28.54	0.68 $\pm$ 0.20	0.11 $\pm$ 0.08
Diffusion	90.5	92.61 $\pm$ 26.19	0.70 $\pm$ 0.19	0.15 $\pm$ 0.08
Flow Matching	93.7	86.88 $\pm$ 23.55	0.74 $\pm$ 0.19	0.16 $\pm$ 0.09

CHAPTER 6

Conclusions and Future Work

This chapter synthesises the research objectives achieved throughout this work and outlines the significant conclusions drawn from both mathematical derivations and empirical evaluations. Furthermore, it includes promising approaches for future research in the domain of generative artificial intelligence modelling for functional data.

The central objective of this work was to establish a rigorous theoretical and empirical framework for deep generative AI models tailored to functional data. To this end, continuous-time generative frameworks were investigated, specifically Flow Matching (FM) models, alongside standard and extended Variance Preserving score-based diffusion models.

The conclusions achieved in this work are presented below:

- **Generative Modelling on Functional Spaces:** The empirical study has provided evidence that both Flow Matching and Diffusion Models successfully capture the underlying distribution of functional trajectories, both for real-world and synthetic datasets.
- **Flow Matching versus Diffusion Models:** From an empirical point of view, Flow Matching consistently demonstrated superior performance in terms of the Mean Squared Error (MSE) metric. Furthermore, it offered a significantly simpler implementation setup compared to traditional diffusion models.
- **Successful Spectral Decoupling:** The major theoretical achievement of this work is the development of a generative model that explicitly accounts for the underlying global structure of functional data. The circulant Ornstein–Uhlenbeck process has been studied in-depth, allowing multivariate time-dependent linear SDEs to be decoupled in the Fourier domain, which opened up the possibility to manage frequency-selective noise injection schedules. Notably, achieving a similar analytical decoupling is not feasible with the current Flow Matching formulation.
- **Frequency-Selective Noise Schedules:** As a complement to the spectral decoupling framework, the circulant framework allowed for explicit control over high- and low-frequency degradation during the forward diffusion process. While

this frequency-selective noise injection was successfully implemented in the empirical study, this version did not yield superior results compared to standard diffusion models and fell short of the performance achieved by the Flow Matching model.

Methodological Reflection: Model Capacity over Mathematical Sophistication

An important methodological suggestion from this work is that the expressive capacity of the neural network often has a greater impact on the generative performance than increasing the mathematical complexity of the diffusion process itself. Although the proposed circulant diffusion framework offers an elegant and computationally efficient mechanism for explicit frequency-domain control, the experiments conducted indicate that model capacity is the primary factor in the quality of the synthesis. Nevertheless, structured formulations such as the proposed circulant SDE remain valuable when mathematical interpretability or explicit control over data features is required.

6.1. Future Work

Based on the theoretical and empirical results of this work, several promising directions for future research are proposed.

- **Spectral Diffusion:** Implement the circulant diffusion model in the frequency domain, as this work has implemented the spatial diffusion. This change will not yield better results, as the underlying stochastic process remains mathematically invariant to the choice of representation space. However, it will take advantage of its decoupled formulation during training.
- **Alternative Noise Injection Schedules:** Explore other possible schedules for the noise injection process, such as non-linear, polynomial, or cosine-based schedules. Particular emphasis should be placed on adaptive, frequency-selective noise injection schedules, since this work restricted its focus to linear schedules.
- **Latent Diffusion Models for Functional Data (LDMs):** Propose the training of functional generative processes within highly compressed latent spaces. This preserves the underlying infinite-dimensional geometric properties of the functions while significantly reducing computation costs since diffusion occurs in a compact space, within a low-dimensional representation of the function [26, 27].
- **Advanced Flow Matching Variants:** investigate more sophisticated velocity-field formulations, such as Rectified Flow or optimal transport-based paths. These variants could further optimise sample efficiency, flatten the probability trajectories for functional generation, and allow empirical comparison against Flow Matching models.

Bibliography

- [1] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf.
- [3] James Ramsay. Functional data analysis. *Encyclopedia of Statistics in Behavioral Science*, 2005. doi: 10.1002/0470013192.bsa239.
- [4] Robert M. Gray. *Toeplitz And Circulant Matrices: A Review (Foundations and Trends(R) in Communications and Information Theory)*. Now Publishers Inc., Hanover, MA, USA, 2006. ISBN 1933019239.
- [5] Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Wang. Predict, refine, synthesize: self-guiding diffusion models for probabilistic time series forecasting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [6] Duy Ngo, Hernando Ombao, Ying Sun, Marc G. Genton, Jennifer Wu, Ramesh Srinivasan, and Steven Cramer. An exploratory data analysis of electroencephalograms using the functional boxplots approach. *Frontiers in Neuroscience*, 9, 2015. ISSN 1662-453X. doi: 10.3389/fnins.2015.00282.
- [7] Agencia Estatal de Meteorología. Meteorological data from spanish weather stations, 2009. URL <https://www.aemet.es>. Dataset. Accessed: 10 July 2026.
- [8] Carlos Ramos-Carreño, José Luis Torrecilla, Miguel Carbajo-Berrocal, Pablo Marcos, and Alberto Suárez. scikit-fda: A python package for functional data

- analysis. *Journal of Statistical Software*, 109(2):1–37, 2024. doi: 10.18637/jss.v109.i02. URL <https://www.jstatsoft.org/index.php/jss/article/view/v109i02>.
- [9] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32, pages 11895–11907. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/3001ef257407d5a371a96dcd947c7d93-Paper.pdf.
- [10] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=PxTIG12RRHS>. Accessed: 10 July 2026.
- [11] Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.
- [12] Severi Rissanen, Markus Heinonen, and Arno Solin. Generative modelling with inverse heat dissipation. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=4PJUBT9f201>. Accessed: 10 July 2026.
- [13] Emiel Hoogeboom and Tim Salimans. Blurring diffusion models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=0jDkC57x5sz>. Accessed: 10 July 2026.
- [14] Subham Sekhar Sahoo. Diffusion models with learned adaptive noise processes. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://openreview.net/forum?id=8gZtt8nrpI>. Accessed: 10 July 2026.
- [15] Xingchang Huang, Corentin Salaun, Cristina Vasconcelos, Christian Theobalt, Cengiz Oztireli, and Gurprit Singh. Blue noise for diffusion models. In *ACM SIGGRAPH 2024 Conference Papers*, SIGGRAPH '24, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705250. doi: 10.1145/3641519.3657435.
- [16] Xi Yu, Xiang Gu, Xin Hu, and Jian Sun. Engdm: Enhanced non-isotropic gaussian diffusion model for progressive image editing. *Sensors*, 25(10):2970, 2025. doi: 10.3390/s25102970.
- [17] Junwei Shu, Wenjie Liu, Changgu Chen, Hantang Liu, Yang Li, and Changbo Wang. Bridging physically based rendering and diffusion models with stochastic differential equation, 2026. URL <https://arxiv.org/abs/2602.20725>. Accessed: 10 July 2026.

- [18] Matthew Tivnan, Jacopo Teneggi, Tzu-Cheng Lee, Ruoqiao Zhang, Kirsten Boedeker, Liang Cai, Grace J. Gang, Jeremias Sulam, and J. Webster Stayman. Fourier diffusion models: A method to control MTF and NPS in score-based stochastic image generation. *IEEE Transactions on Medical Imaging*, 44(9):3694–3704, 2025. doi: 10.1109/TMI.2025.3553805.
- [19] Luca Scimeca, Thomas Jiralerspong, Berton Earnshaw, Jason Hartford, and Yoshua Bengio. Learning what matters: Steering diffusion via spectrally anisotropic forward noise, 2025. URL <https://arxiv.org/abs/2510.09660>. Accessed: 10 July 2026.
- [20] C. R. Dietrich and G. N. Newsam. Fast and exact simulation of stationary gaussian processes through circulant embedding of the covariance matrix. *SIAM Journal on Scientific Computing*, 18(4):1088–1107, 1997. doi: 10.1137/S1064827592240555.
- [21] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>. Accessed: 10 July 2026.
- [22] Gavin Kerrigan, Giosue Migliorini, and Padhraic Smyth. Functional flow matching. In *International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 3934–3942. PMLR, 2024. URL <https://proceedings.mlr.press/v238/kerrigan24a.html>. Accessed: 10 July 2026.
- [23] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer-Verlag, Heidelberg; New York, 5th edition, 2003. ISBN 978-3540637202.
- [24] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.
- [25] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data*, 7(1):154, May 2020. ISSN 2052-4463. doi: 10.1038/s41597-020-0495-6.
- [26] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Computer Vision and Pattern Recognition (CVPR)*, pages 10674–10685, Los Alamitos, CA, USA, June 2022. IEEE Computer Society. doi: 10.1109/CVPR52688.2022.01042.

- [27] Sifan Wang, Zehao Dou, Siming Shan, Tong-Rui Liu, and Lu Lu. Fundiff: diffusion models over function spaces for physics-informed generative modeling. *Nature Communications*, 17(1):5749, Apr 2026. ISSN 2041-1723. doi: 10.1038/s41467-026-72292-0.
- [28] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, UK, 2nd edition, 2012. ISBN 978-0521839402.

APPENDIX A

Circulant Matrices

Circulant matrices form a special class of structured matrices characterised by cyclic invariance and play an important role in many areas of applied mathematics. In particular, they admit an explicit spectral decomposition in terms of the Discrete Fourier Transform, which makes them especially interesting in analytical computations. Those matrices naturally appear in systems with a periodic structure. Particularly in this work, they are used to analyse Ornstein–Uhlenbeck systems with circulant drift and diffusion coefficients.

In this chapter, circulant matrices are first defined, and their fundamental structural properties are presented. Their spectral decomposition is then derived in detail, providing explicit expressions for eigenvalues and eigenvectors. Afterward, several useful properties of real and symmetric circulant matrices are collected. Finally, a change of notation related to Fourier frequencies is introduced for later use in circulant Ornstein–Uhlenbeck systems.

A.1. Definition

Let $D \in \mathbb{N}$ denote the dimension. A circulant matrix $\mathbf{C} \in \mathbb{C}^{D \times D}$ is completely determined by its first row, and it is characterised by the property that each row is obtained as a cyclic right shift of the previous one. Let $\mathbf{c} = (c_0, c_1, \dots, c_{D-1}) \in \mathbb{C}^D$ be the circulant matrix generated by $\mathbf{c}$. It is denoted as

$$\mathbf{C} := \text{circ}(c_0, c_1, \dots, c_{D-1})$$

and it is defined as

$$[\mathbf{C}]_{mn} = \mathbf{c}_{(n-m) \bmod D} \quad \text{for } m, n \in \{0, \dots, D-1\}.$$

This definition leads to a matrix with the following structure:

$$\mathbf{C} = \begin{pmatrix} c_0 & c_1 & c_2 & \dots & c_{D-1} \\ c_{D-1} & c_0 & c_1 & \dots & c_{D-2} \\ \vdots & \vdots & \vdots & & \vdots \\ c_1 & c_2 & c_3 & \dots & c_0 \end{pmatrix}.$$

A.2. Spectral Decomposition

The spectral properties of circulant matrices presented in this section follow the treatment in Chapter 3 of [4]. In this section, the eigenvalues and eigenvectors of circulant matrices are determined.

Let $\mathbf{C} \in \mathbb{C}^{D \times D}$ be a circulant matrix. Let λ be an eigenvalue of $\mathbf{C}$ with the corresponding eigenvector $\mathbf{y} := (y_0, \dots, y_{D-1})^\top \in \mathbb{C}^D$. Then, the following characteristic equation must be satisfied:

$$\mathbf{C}\mathbf{y} = \lambda\mathbf{y},$$

which is equivalent to

$$[\mathbf{C}\mathbf{y}]_m = \lambda y_m, \quad m = 0, \dots, D-1.$$

By expanding the left side of the equality, substituting the definition of $\mathbf{C}_{mn}$, and splitting the sum to handle the modulus operation, one obtains

$$\begin{aligned} [\mathbf{C}\mathbf{y}]_m &= \sum_{n=0}^{D-1} \mathbf{C}_{mn} y_n = \sum_{n=0}^{D-1} c_{(n-m) \pmod{D}} y_n = \sum_{n=0}^{m-1} c_{D+n-m} y_n + \sum_{n=m}^{D-1} c_{n-m} y_n \\ &= \sum_{\ell=D-m}^{D-1} c_\ell y_{\ell+m-D} + \sum_{\ell=0}^{D-(m+1)} c_\ell y_{\ell+m}. \end{aligned}$$

Then, for $m = 0, \dots, D-1$,

$$[\mathbf{C}\mathbf{y}]_m = \lambda y_m \iff \sum_{\ell=D-m}^{D-1} c_\ell y_{\ell+m-D} + \sum_{\ell=0}^{D-(m+1)} c_\ell y_{\ell+m} = \lambda y_m.$$

A solution is sought in the form $y_m = \rho^m$, where $\rho \in \mathbb{C}$ satisfies $\rho^D = 1$. Substituting into the previous expression yields

$$\begin{aligned} [\mathbf{C}\mathbf{y}]_m &= \sum_{\ell=D-m}^{D-1} c_\ell \rho^{\ell+m-D} + \sum_{\ell=0}^{D-(m+1)} c_\ell \rho^{\ell+m} \\ &= \rho^{m-D} \sum_{\ell=D-m}^{D-1} c_\ell \rho^\ell + \rho^m \sum_{\ell=0}^{D-(m+1)} c_\ell \rho^\ell \\ &= \rho^m \left(\rho^{-D} \sum_{\ell=D-m}^{D-1} c_\ell \rho^\ell + \sum_{\ell=0}^{D-(m+1)} c_\ell \rho^\ell \right). \end{aligned}$$

Since $\rho^D = 1$, it follows that

$$[\mathbf{C}\mathbf{y}]_m = \lambda y_m \iff \rho^m \sum_{\ell=0}^{D-1} c_\ell \rho^\ell = \lambda \rho^m.$$

In summary, if $\rho \in \mathbb{C}$ satisfies $\rho^D = 1$ and the eigenvalue λ and the eigenvector $\mathbf{y}$ are defined as:

$$\mathbf{y} = (1, \rho, \rho^2, \dots, \rho^{D-1})^\top, \quad \lambda = \sum_{\ell=0}^{D-1} c_\ell \rho^\ell,$$

then it is true that $\mathbf{C}\mathbf{y} = \lambda\mathbf{y}$.

There are several values of ρ that satisfy this equation. More concretely, all the D -th roots of unity, which are defined as

$$\rho_m := e^{-\frac{2\pi i}{D}m}, \quad \text{for } m = 0, \dots, D-1.$$

In conclusion, the eigenvalues $\{\lambda_m\}_{m=0}^{D-1}$ and the corresponding normalised eigenvectors $\{\mathbf{y}_m\}_{m=0}^{D-1}$ are given by:

$$\lambda_m = \sum_{\ell=0}^{D-1} c_\ell \omega^{m\ell}, \quad \mathbf{y}_m = \frac{1}{\sqrt{D}} (1, \omega^m, \omega^{2m}, \dots, \omega^{(D-1)m})^\top,$$

where $\omega := e^{-\frac{2\pi i}{D}}$.

Define the modal matrix $\mathbf{U}$ (containing eigenvectors as columns) and the diagonal matrix $\mathbf{\Lambda}$ as:

$$\mathbf{U} = (\mathbf{y}_0 \mid \dots \mid \mathbf{y}_{D-1}) = \left[\frac{1}{\sqrt{D}} e^{-\frac{2\pi i}{D}mn} \right]_{n,m=0}^{D-1}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_0, \dots, \lambda_{D-1}).$$

It follows that $\mathbf{C}\mathbf{U} = \mathbf{U}\mathbf{\Lambda}$, which implies the spectral decomposition

$$\mathbf{C} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\dagger,$$

where $\mathbf{U}^\dagger$ denotes the conjugate transpose of $\mathbf{U}$.

A.3. Circulant Matrices Properties

In this section, two fundamental properties of circulant matrices are established. Specifically, the structure of both real and Hermitian circulant matrices and their spectral decomposition is derived.

A.3.1. Real Case

Real circulant matrices arise frequently in applications such as signal processing. It is therefore of interest to characterise when a circulant matrix is real in terms of its eigenvalues.

Theorem A.1 (Characterisation of Real Circulant Matrices). *Let $\mathbf{C} \in \mathbb{C}^{D \times D}$ be a circulant matrix, and let $\{\lambda_m\}_{m=0}^{D-1}$ be its eigenvalues.*

Then,

$$\mathbf{C} \text{ is real} \iff \begin{cases} i) \lambda_0 \in \mathbb{R} \quad (\text{and } \lambda_{\frac{D}{2}} \in \mathbb{R} \text{ if } D \text{ is even}), \\ ii) \lambda_m = \overline{\lambda_{D-m}}, \quad \forall m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor. \end{cases}$$

Proof. Let $\mathbf{c} = (c_0, c_1, \dots, c_{D-1}) \in \mathbb{C}^D$ be the first row of the circulant matrix $\mathbf{C}$. Then, as shown in Section A.2, the eigenvalues of a circulant matrix whose first row is $\mathbf{c}$ can be expressed as:

$$\lambda_m = \sum_{\ell=0}^{D-1} c_\ell \omega^{m\ell}, \quad m = 0, \dots, D-1,$$

where ω is a D -th root of unity, that is $\omega = e^{-\frac{2\pi i}{D}}$.

($\Rightarrow$) On the one hand, assuming that $\mathbf{c} \in \mathbb{R}^D$, which means $c_\ell \in \mathbb{R} \forall \ell = 0, \dots, D-1$, it follows that $\overline{c_\ell} = c_\ell \forall \ell = 0, \dots, D-1$.

On the other hand, since $\omega^D = 1$, then $\omega^{-m} = \omega^{D-m}$. Therefore, it is obtained that

$$\overline{\omega^{m\ell}} = \omega^{-m\ell} = \omega^{\ell(D-m)}.$$

Taken together, these identities imply that, for each $m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor$,

$$\overline{\lambda_m} = \sum_{\ell=0}^{D-1} \overline{c_\ell \omega^{m\ell}} = \sum_{\ell=0}^{D-1} c_\ell \omega^{\ell(D-m)} = \lambda_{D-m}.$$

Taking the conjugate of both sides of the equality yields the desired relation.

There is left to prove that $\lambda_0 \in \mathbb{R}$, which is immediate after realising that

$$\lambda_0 = \sum_{\ell=0}^{D-1} c_\ell,$$

which is a sum of real numbers. The same happens for $\lambda_{\frac{D}{2}}$ when D is even, since

$$\lambda_{\frac{D}{2}} = \sum_{\ell=0}^{D-1} c_\ell \omega^{\frac{D}{2}\ell} = \sum_{\ell=0}^{D-1} c_\ell e^{-\ell\pi i} = \sum_{\ell=0}^{D-1} (-1)^\ell c_\ell.$$

($\Leftarrow$) Assume that

$$\begin{cases} \text{i) } \lambda_0 \in \mathbb{R} \quad (\text{and } \lambda_{\frac{D}{2}} \in \mathbb{R} \text{ if } D \text{ is even}), \\ \text{ii) } \lambda_m = \overline{\lambda_{D-m}}, \quad \forall m = 1, \dots, \lfloor \frac{D-1}{2} \rfloor. \end{cases}$$

These two conditions imply the following one:

$$\lambda_m = \overline{\lambda_{D-m}}, \quad \forall m = 0, \dots, D-1.$$

The proof relies on two identities, which are stated and derived as follows:

Identity 1. Orthogonality identity of roots of unity.

$$(A.1) \quad \sum_{n=0}^{D-1} \omega^{mn} = D \delta_{m,0} = \begin{cases} D & \text{if } m = 0 \\ 0 & \text{if } m \neq 0 \end{cases},$$

where $\omega = e^{-\frac{2\pi i}{D}}$.

To prove this, consider the geometric series

$$S_m = \sum_{\ell=0}^{D-1} \omega^{m\ell} = \sum_{\ell=0}^{D-1} (\omega^m)^\ell \quad \text{for } m = 0, \dots, D-1.$$

Two cases are distinguished.

Case 1. If $m = 0$, then $(\omega^0)^\ell = 1 \forall \ell = 0, \dots, D-1$. Therefore, $S_0 = D$.

Case 2. If $m \neq 0$, then $S_m = \frac{1-\omega^{mD}}{1-\omega^m}$, but $\omega^{mD} = 1$. Thus, $S_m = 0$.

Identity 2. Inverse Discrete Fourier Transform. For $\ell = 0, \dots, D-1$,

$$D c_\ell = \sum_{m=0}^{D-1} \lambda_m \omega^{-m\ell}$$

In this proof, the identity 1 will be applied.

$$\begin{aligned} \sum_{m=0}^{D-1} \lambda_m \omega^{-m\ell} &= \sum_{m=0}^{D-1} \left(\sum_{n=0}^{D-1} c_n \omega^{mn} \right) \omega^{-m\ell} = \sum_{n=0}^{D-1} c_n \sum_{m=0}^{D-1} \omega^{m(n-\ell)} \\ &= \sum_{n=0}^{D-1} c_n D \delta_{n-\ell,0} = \sum_{n=0}^{D-1} c_n D \delta_{n,\ell} = D c_\ell. \end{aligned}$$

Having established both identities, remember that it had been deduced earlier (thanks to assumptions), that

$$\lambda_m = \overline{\lambda_{D-m}}, \quad \forall m = 0, \dots, D-1.$$

Applying these equalities to identity 2, for $\ell = 0, \dots, D-1$,

$$\begin{aligned} D c_\ell &= \sum_{m=0}^{D-1} \lambda_m \omega^{-m\ell} = \sum_{m=0}^{D-1} \overline{\lambda_{D-m}} \omega^{-m\ell} \\ &= \sum_{m=0}^{D-1} \left(\sum_{n=0}^{D-1} \overline{c_n \omega^{n(D-m)}} \right) \omega^{-m\ell} \\ &= \sum_{m=0}^{D-1} \left(\sum_{n=0}^{D-1} \overline{c_n} \omega^{mn} \right) \omega^{-m\ell} \\ &= \sum_{n=0}^{D-1} \overline{c_n} \sum_{m=0}^{D-1} \omega^{m(n-\ell)} = \sum_{n=0}^{D-1} \overline{c_n} D \delta_{\ell,n} = D \overline{c_\ell}. \end{aligned}$$

Comparing both expressions yields

$$c_\ell = \overline{c_\ell} \quad \forall \ell = 0, \dots, D-1.$$

Thus, it is immediate that $c_\ell \in \mathbb{R} \forall \ell$, which means that the circulant matrix C is real. $\square$

A.3.2. Hermitian Case

The following result provides a characterisation of Hermitian circulant matrices in terms of their eigenvalues.

Theorem A.2 (Characterisation of Hermitian Circulant Matrices). *Let $\mathbf{C} \in \mathbb{C}^{D \times D}$ be a circulant matrix, and let $\{\lambda_m\}_{m=0}^{D-1}$ be its eigenvalues.*

Then,

$$\mathbf{C} \text{ is Hermitian} \iff \lambda_m \in \mathbb{R} \quad \forall m = 0, \dots, D-1.$$

Proof. For a circulant matrix, as shown in Section A.2, the eigenvalues $\{\lambda_m\}_{m=0}^{D-1}$ and eigenvectors $\{\mathbf{y}_m\}_{m=0}^{D-1}$ are given by

$$\lambda_m = \sum_{\ell=0}^{D-1} c_\ell \omega^{m\ell}, \quad \mathbf{y}_m = \frac{1}{\sqrt{D}} (1, \omega^m, \omega^{2m}, \dots, \omega^{(D-1)m})^\top,$$

where $\omega := e^{-\frac{2\pi i}{D}}$.

The diagonalisation of $\mathbf{C}$ via the Discrete Fourier Transform (DFT) was also derived in that section:

$$\mathbf{C} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\dagger,$$

where

$$\mathbf{U} = (\mathbf{y}_0 \mid \dots \mid \mathbf{y}_{D-1}), \quad \mathbf{\Lambda} = \text{diag}(\lambda_0, \dots, \lambda_{D-1}).$$

The matrix $\mathbf{U}$ is unitary, i.e.

$$\mathbf{U}^\dagger \mathbf{U} = \mathbf{U} \mathbf{U}^\dagger = I_D.$$

($\Rightarrow$) If $\mathbf{C}$ is Hermitian, then by the spectral theorem (see [28, Section 4.1]), all its eigenvalues are real. Hence $\lambda_m \in \mathbb{R}$ for all $m = 0, \dots, D-1$.

($\Leftarrow$) Assume that $\lambda_m \in \mathbb{R}$ for all m . Then $\mathbf{\Lambda}^\dagger = \mathbf{\Lambda}$. Taking adjoints,

$$\mathbf{C}^\dagger = (\mathbf{U} \mathbf{\Lambda} \mathbf{U}^\dagger)^\dagger = \mathbf{U} \mathbf{\Lambda}^\dagger \mathbf{U}^\dagger.$$

Hence,

$$\mathbf{C}^\dagger = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\dagger = \mathbf{C},$$

and therefore $\mathbf{C}$ is Hermitian. $\square$

A.4. Conjugate-paired DFT Matrix

In this section, an alternative notation for the diagonalisation of circulant matrices is introduced. This notation is equivalent to the one presented in Section A.2, but it is better suited for distinguishing frequency components.

A.4.1. Theoretical Construction

On the basis of the notation in Section A.2, the construction proceeds in two steps. The eigenvalues $\{\lambda_m\}_{m=0}^{D-1}$ and the corresponding eigenvectors $\{\mathbf{y}_m\}_{m=0}^{D-1}$ are recalled:

$$\lambda_m = \sum_{\ell=0}^{D-1} c_\ell \omega^{m\ell}, \quad \mathbf{y}_m = \frac{1}{\sqrt{D}} (1, \omega^m, \omega^{2m}, \dots, \omega^{(D-1)m})^\top,$$

where $\omega := e^{-\frac{2\pi i}{D}}$.

Step 1. Reordering the Fourier Basis. In the standard ordering,

$$\mathbf{C} = \mathbf{U} \tilde{\mathbf{\Lambda}} \mathbf{U}^\dagger,$$

where eigenvectors and eigenvalues are indexed from 0 to $D-1$:

$$\mathbf{U} = (\mathbf{y}_0 \mid \dots \mid \mathbf{y}_{D-1}), \quad \tilde{\mathbf{\Lambda}} = \text{diag}(\lambda_0, \dots, \lambda_{D-1}).$$

Note that the notation for the eigenvalue diagonal matrix has changed to $\tilde{\mathbf{\Lambda}}$, as the notation $\mathbf{\Lambda}$ is kept for the conjugate-paired eigenvalue diagonal matrix.

This ordering mixes low and high frequencies. The first indices correspond to increasing positive frequencies, followed by frequencies that can be interpreted as negative. For the purposes of this work, it is preferable to group frequencies into pairs of positive and negative components. This is achieved by reordering the Fourier basis.

Two special frequencies must be distinguished, as they do not have a corresponding negative counterpart.

The Direct Current (DC) mode λ_0 always exists and corresponds to the constant eigenvector $\mathbf{y}_0 = (1, \dots, 1)^\top$. It represents the mean component of the system and is associated with the lowest frequency (zero-frequency). This mode is placed at the beginning of the Fourier basis.

When D is even, the Nyquist mode $\lambda_{\frac{D}{2}}$ corresponds to the highest frequency. Its associated eigenvector alternates between 1 and -1 , i.e., $\mathbf{y}_{\frac{D}{2}} = (1, -1, 1, -1, \dots)^\top$. This mode is placed at the end of the Fourier basis.

Thus, the Fourier basis is reordered by grouping frequencies $\pm m$ and arranging them from low to high. Mathematically, this means reordering columns of $\mathbf{U}$.

Case 1. If D is even, the Nyquist mode exists, and

$$\mathbf{U} = \left(\mathbf{y}_0 \mid \mathbf{y}_1 \mid \mathbf{y}_{-1} \mid \mathbf{y}_2 \mid \mathbf{y}_{-2} \mid \dots \mid \mathbf{y}_{\frac{D}{2}-1} \mid \mathbf{y}_{-(\frac{D}{2}-1)} \mid \mathbf{y}_{\frac{D}{2}} \right),$$

where $\mathbf{y}_{-m} = \mathbf{y}_{D-m}$. The corresponding diagonal matrix of eigenvalues is

$$\mathbf{\Lambda} = \text{diag} \left(\lambda_0 \mid \lambda_1 \mid \lambda_{-1} \mid \lambda_2 \mid \lambda_{-2} \mid \cdots \mid \lambda_{\frac{D}{2}-1} \mid \lambda_{-(\frac{D}{2}-1)} \mid \lambda_{\frac{D}{2}} \right),$$

where $\lambda_{-m} = \lambda_{D-m}$.

Case 2. If D is odd, the Nyquist mode does not exist, and

$$\mathbf{U} = \left(\mathbf{y}_0 \mid \mathbf{y}_1 \mid \mathbf{y}_{-1} \mid \mathbf{y}_2 \mid \mathbf{y}_{-2} \mid \cdots \mid \mathbf{y}_{\frac{D-1}{2}} \mid \mathbf{y}_{-\frac{D-1}{2}} \right),$$

where $\mathbf{y}_{-m} = \mathbf{y}_{D-m}$. The corresponding diagonal matrix of eigenvalues is

$$\mathbf{\Lambda} = \text{diag} \left(\lambda_0 \mid \lambda_1 \mid \lambda_{-1} \mid \lambda_2 \mid \lambda_{-2} \mid \cdots \mid \lambda_{\frac{D-1}{2}} \mid \lambda_{-\frac{D-1}{2}} \right),$$

where $\lambda_{-m} = \lambda_{D-m}$.

The circulant matrix $\mathbf{C}$ is now diagonalised as

$$\mathbf{C} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\dagger.$$

Step 2. Definition of the Fourier Matrix. Define the matrix $\mathbf{F} := \mathbf{U}^\dagger$. With this new notation, the diagonalisation of $\mathbf{C}$ can be written as

$$\mathbf{C} = \mathbf{F}^\dagger \mathbf{\Lambda} \mathbf{F}.$$

A.4.2. Matrix-Based Construction of Fourier Basis

Equivalently, the construction of $\mathbf{F}$ can be achieved by first defining

$$\left[\tilde{\mathbf{F}} \right]_{n,m=0}^{D-1} = \left[\frac{1}{\sqrt{D}} e^{\frac{2\pi i}{D} mn} \right]_{n,m=0}^{D-1}, \quad \tilde{\mathbf{\Lambda}} = \text{diag}(\lambda_0, \dots, \lambda_{D-1}),$$

which implies the spectral decomposition of a circulant matrix $\mathbf{C}$ as $\mathbf{C} = (\tilde{\mathbf{F}})^\dagger \tilde{\mathbf{\Lambda}} \tilde{\mathbf{F}}$,

Secondly, reorder the Fourier basis, which is equivalent to reordering $\tilde{\mathbf{F}}$ rows.

Case 1: D even. Define the permutation $\pi : \{0, \dots, D-1\} \rightarrow \{0, \dots, D-1\}$, as:

$$\pi(0) = 0, \quad \pi(2m-1) = m, \quad \pi(2m) = D-m, \quad \text{for } m = 1, \dots, \frac{D}{2} - 1$$

and the final index:

$$\pi(D-1) = \frac{D}{2}.$$

Equivalently, in vector form:

$$\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D}{2}-1, \frac{D}{2}+1, \frac{D}{2}).$$

Case 2: D odd. Define the permutation $\pi : \{0, \dots, D-1\} \rightarrow \{0, \dots, D-1\}$, as:

$$\pi(0) = 0, \quad \pi(2m-1) = m, \quad \pi(2m) = D-m \quad \text{for } m = 1, \dots, \frac{D-1}{2}.$$

Equivalently, in vector form:

$$\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D-1}{2}, \frac{D+1}{2}).$$

Let $\mathbf{P} \in \mathbb{R}^{D \times D}$ be the permutation matrix associated with the permutation

$$\pi : \{0, \dots, D-1\} \rightarrow \{0, \dots, D-1\},$$

defined by

$$[\mathbf{P}]_{i,j} = \delta_{j,\pi(i)} \quad i, j = 0, \dots, D-1.$$

This means that the i -th row of $\mathbf{P}$ selects the $\pi(i)$ -th row of a matrix.

The conjugate-paired DFT matrix $\mathbf{F}$ is constructed as

$$\mathbf{F} := \mathbf{P} \tilde{\mathbf{F}},$$

and the conjugate-paired eigenvalues diagonal matrix as

$$\mathbf{\Lambda} = \mathbf{P} \tilde{\mathbf{\Lambda}} \mathbf{P}^\dagger.$$

The following results will be used later in Example B.24 and subsequently in Chapter 3.

Theorem A.3. *Define the conjugate-paired Discrete Fourier Transform matrix*

$$\mathbf{F} = \mathbf{P} \tilde{\mathbf{F}},$$

where

$$[\tilde{\mathbf{F}}]_{n,m=0}^{D-1} = \left[\frac{1}{\sqrt{D}} e^{\frac{2\pi i}{D} mn} \right]_{n,m=0}^{D-1},$$

and

$$[\mathbf{P}]_{i,j} = \delta_{j,\pi(i)} \quad i, j = 0, \dots, D-1,$$

for the permutation

- D even: $\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D}{2}-1, \frac{D}{2}+1, \frac{D}{2}).$
- D odd: $\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D-1}{2}, \frac{D+1}{2}).$

Define

- D even: $\mathbf{\Lambda} = \text{diag} \left(\lambda_0 \mid \lambda_1 \mid \lambda_{-1} \mid \lambda_2 \mid \lambda_{-2} \mid \dots \mid \lambda_{\frac{D}{2}-1} \mid \lambda_{-(\frac{D}{2}-1)} \mid \lambda_{\frac{D}{2}} \right),$
- D odd: $\mathbf{\Lambda} = \text{diag} \left(\lambda_0 \mid \lambda_1 \mid \lambda_{-1} \mid \lambda_2 \mid \lambda_{-2} \mid \dots \mid \lambda_{\frac{D-1}{2}} \mid \lambda_{-\frac{D-1}{2}} \right),$

Then,

$$\mathbf{C} = \mathbf{F}^\dagger \mathbf{\Lambda} \mathbf{F}.$$

Proof. One identifies that, using the notation above,

$$\mathbf{\Lambda} = \mathbf{P} \tilde{\mathbf{\Lambda}} \mathbf{P}^\dagger,$$

where $\tilde{\mathbf{\Lambda}} = (\lambda_0, \dots, \lambda_{D-1})$.

Then, it follows that

$$\mathbf{F}^\dagger \mathbf{\Lambda} \mathbf{F} = (\mathbf{P} \tilde{\mathbf{F}})^\dagger (\mathbf{P} \tilde{\mathbf{\Lambda}} \mathbf{P}^\dagger) (\mathbf{P} \tilde{\mathbf{F}}).$$

Since $\mathbf{P}$ is a permutation, it is unitary, then

$$\mathbf{F}^\dagger \mathbf{\Lambda} \mathbf{F} = \tilde{\mathbf{F}}^\dagger \tilde{\mathbf{\Lambda}} \tilde{\mathbf{F}} = \mathbf{C}$$

□

Theorem A.4. Define the conjugate-paired Discrete Fourier Transform matrix

$$\mathbf{F} = \mathbf{P} \tilde{\mathbf{F}},$$

where

$$\left[\tilde{\mathbf{F}} \right]_{n,m=0}^{D-1} = \left[\frac{1}{\sqrt{D}} e^{\frac{2\pi i}{D} mn} \right]_{n,m=0}^{D-1},$$

and

$$\mathbf{P}_{i,j} = \delta_{j,\pi(i)} \quad i, j = 0, \dots, D-1,$$

for the permutation

- D even: $\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D}{2}-1, \frac{D}{2}+1, \frac{D}{2})$.
- D odd: $\pi = (0, 1, D-1, 2, D-2, \dots, \frac{D-1}{2}, \frac{D+1}{2})$.

Then,

$$\mathbf{F} \mathbf{F}^\top = \text{diag} \left(1, \underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}}_{\text{pairs } (k, -k)}, \epsilon \right),$$

where:

- the first entry corresponds to the DC mode $k = 0$;
- each 2×2 block corresponds to a conjugate pair $(k, -k)$;
- $\epsilon = 1$ if D is even (Nyquist mode), and is absent if D is odd.

Proof. Expanding the product

$$\mathbf{F}\mathbf{F}^\top = (\mathbf{P}\tilde{\mathbf{F}})(\mathbf{P}\tilde{\mathbf{F}})^\top = \mathbf{P}(\tilde{\mathbf{F}}\tilde{\mathbf{F}}^\top)\mathbf{P}^\top = \mathbf{P}\Pi\mathbf{P}^\top,$$

where

$$[\Pi]_{m,n} := [\tilde{\mathbf{F}}\tilde{\mathbf{F}}^\top]_{m,n} = \sum_{k=0}^{D-1} \tilde{\mathbf{F}}_{m,k} \tilde{\mathbf{F}}_{n,k}^* = \frac{1}{D} \sum_{k=0}^{D-1} e^{\frac{2\pi i}{D}(m+n)k},$$

which means that, using the already proven Identity (A.1)

$$[\Pi]_{m,n} = [\tilde{\mathbf{F}}\tilde{\mathbf{F}}^\top]_{m,n} = \begin{cases} 1, & n \equiv -m \pmod{D}, \\ 0, & \text{otherwise.} \end{cases}$$

Using the definition of matrix multiplication:

$$[\mathbf{P}\Pi\mathbf{P}^\top]_{i,j} = \sum_{m,n=0}^{D-1} [\mathbf{P}]_{i,m} [\Pi]_{m,n} [\mathbf{P}]_{j,n}^* = \sum_{m,n=0}^{D-1} \delta_{m,\pi(i)} [\Pi]_{m,n} \delta_{n,\pi(j)} = [\Pi]_{\pi(i),\pi(j)}.$$

Therefore,

$$[\mathbf{P}\Pi\mathbf{P}^\top]_{i,j} = \begin{cases} 1, & \pi(j) \equiv -\pi(i) \pmod{D}, \\ 0, & \text{otherwise.} \end{cases}$$

Since

- $\pi(0) = 0$, and $0 \equiv -0 \pmod{D}$,
- if D is even, $\pi(D-1) = \frac{D}{2}$, and $\frac{D}{2} \equiv -\frac{D}{2} \pmod{D}$,
- $\pi(2m-1) = m$, $\pi(2m) = D-m$, and $\pi(2m) \equiv -\pi(2m-1) \pmod{D}$,

it follows that $\mathbf{P}\Pi\mathbf{P}^\top$ has the block-diagonal form:

$$\mathbf{P}\Pi\mathbf{P}^\top = \text{diag} \left(1, \underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}}_{\text{pairs } (k, -k)}, \epsilon \right),$$

where:

- the first entry corresponds to the DC mode $k = 0$;
- each 2×2 block corresponds to a conjugate pair $(k, -k)$;
- $\epsilon = 1$ if D is even (Nyquist mode), and is absent if D is odd.

□

APPENDIX B

Complex Distributions

This chapter introduces the main concepts related to complex-valued distributions, with particular emphasis on complex Gaussian distributions. These results are required for the derivation of the circulant Ornstein–Uhlenbeck process in Chapter 3.

B.1. Scalar Case

The exposition begins with the scalar case. This section aims to fully characterise the distribution of a complex random variable by giving the definition and fundamental properties of its parameters: expectation, covariance, and pseudocovariance. Finally, a direct relationship between a complex random variable and a two-dimensional real random vector is established.

Definition B.1 (Complex Random Variable). A complex random variable z is a random variable that can be written as

$$z = x + iy,$$

where both x and y are real-valued random variables.

The complex conjugate of z is defined by

$$\bar{z} := x - iy.$$

A complex random variable can be equivalently viewed as a two-dimensional real random vector (x, y) . However, it is often convenient to work directly in the complex setting.

B.1.1. Complex Expectation

The expectation of a complex random variable is defined in a natural way, analogous to the real case.

Definition B.2 (Expectation of a Complex Random Variable). Let $z = x + iy$ be a complex random variable such that $\mathbb{E}[|z|] < \infty$. The expectation of z is defined as

$$\mathbb{E}[z] = \mathbb{E}[x] + i \mathbb{E}[y],$$

where $\mathbb{E}[x]$ and $\mathbb{E}[y]$ denote the expectations of the real-valued random variables x and y , respectively.

It follows immediately that

$$\mathbb{E}[x] = \operatorname{Re}(\mathbb{E}[z]), \quad \mathbb{E}[y] = \operatorname{Im}(\mathbb{E}[z]).$$

The linearity property of the real-expectation case extends naturally to the complex setting.

Theorem B.3 (Linearity of Complex Expectation). *Let z and w be complex random variables such that $\mathbb{E}[|z|] < \infty$ and $\mathbb{E}[|w|] < \infty$. Then, for all $a, b \in \mathbb{C}$,*

$$\mathbb{E}[az + bw] = a \mathbb{E}[z] + b \mathbb{E}[w] \quad \text{for all } a, b \in \mathbb{C}.$$

Moreover, the expectation operator also commutes with complex conjugation.

Theorem B.4 (Conjugation and Expectation). *Let z be a complex random variable such that $\mathbb{E}[|z|] < \infty$. Then,*

$$\mathbb{E}[\bar{z}] = \overline{\mathbb{E}[z]}.$$

Both properties follow directly from the definition of z and the linearity of the real expectation.

B.1.2. Complex Variance

The relationship between the complex expectation $\mathbb{E}[z]$ and the real expectations $\mathbb{E}[\operatorname{Re}(z)]$ and $\mathbb{E}[\operatorname{Im}(z)]$ is one-to-one. The situation changes in the case of second-order moments, as shown below.

Definition B.5 (Variance of a Complex Random Variable). Let z be a complex random variable such that $\mathbb{E}[|z|^2] < \infty$.

The variance of z is defined by

$$K_{zz} := \operatorname{Var}(z) := \mathbb{E}[|z - \mathbb{E}[z]|^2],$$

where $|\cdot|$ denotes the complex modulus.

Similarly to the real case, the variance admits an equivalent expression

$$K_{zz} = \operatorname{Var}(z) = \mathbb{E}[|z|^2] - |\mathbb{E}[z]|^2.$$

Although z takes values in $\mathbb{C}$, its variance is real-valued since it is defined as the expectation of a non-negative real quantity.

The following result relates the complex variance to the variances of the real and imaginary parts. It gives an expression for the decomposition of the complex variance.

Theorem B.6 (Complex Variance Decomposition). *Let z be a complex random variable such that $\mathbb{E}[|z|^2] < \infty$. Then,*

$$K_{zz} = \text{Var}(z) = \text{Var}(\text{Re}(z)) + \text{Var}(\text{Im}(z)).$$

This identity follows from a direct expansion using linear algebraic operations of $|z - \mathbb{E}[z]|^2$.

The result does not establish a one-to-one correspondence between $\text{Var}(z)$ and the pair $(\text{Var}(\text{Re}(z)), \text{Var}(\text{Im}(z)))$. Therefore, additional quantities are required to fully describe second-order properties.

The concept of variance naturally extends to two complex random variables through covariance, in the same way as in the real case. Indeed, variance appears as a particular case of covariance when the two variables coincide.

Definition B.7 (Covariance Between Complex Random Variables). *Let z and w be complex random variables such that $\mathbb{E}[|z|^2] < \infty$ and $\mathbb{E}[|w|^2] < \infty$.*

The covariance between z and w is defined as

$$K_{zw} := \text{Cov}(z, w) := \mathbb{E}[(z - \mathbb{E}[z])(\overline{w - \mathbb{E}[w]})].$$

Equivalently, expanding the product,

$$K_{zw} = \text{Cov}(z, w) = \mathbb{E}[z \overline{w}] - \mathbb{E}[z] \overline{\mathbb{E}[w]}.$$

Theorem B.8 (Properties of Complex Covariance). *Let z, w, z_1, z_2, w_1, w_2 be complex random variables with finite second moments. Then:*

- i) $\text{Cov}(z, z) = \text{Var}(z)$.
- ii) $\text{Cov}(z, w) = \overline{\text{Cov}(w, z)}$.
- iii) $\text{Cov}(\alpha_1 z_1 + \alpha_2 z_2, w) = \alpha_1 \text{Cov}(z_1, w) + \alpha_2 \text{Cov}(z_2, w), \quad \forall \alpha_1, \alpha_2 \in \mathbb{C}$.
- iv) $\text{Cov}(z, \beta_1 w_1 + \beta_2 w_2) = \overline{\beta_1} \text{Cov}(z, w_1) + \overline{\beta_2} \text{Cov}(z, w_2), \quad \forall \beta_1, \beta_2 \in \mathbb{C}$.

These properties follow directly from the definition of covariance and the linearity of the expectation, by straightforward algebraic manipulations.

B.1.3. Pseudo-Variance

As explained above, unlike the real case, covariance alone is not sufficient to fully characterise the second-order properties of a complex random variable. An additional quantity, called the pseudo-variance, is required. The following is a brief explanation of the concept of pseudo-variance in complex random variables.

Definition B.9 (Pseudo-Variance of a Complex Random Variable). *Let z be a complex random variable such that $\mathbb{E}[|z|^2] < \infty$.*

The pseudo-variance of z is defined by

$$J_{zz} := \mathbb{E}[(z - \mathbb{E}[z])^2],$$

where $(\cdot)^2$ denotes the square operation in complex values.

Similarly to the variance, an equivalent expression, derived by expanding the definition, is given by

$$J_{zz} = \mathbb{E}[z^2] - \mathbb{E}[z]^2.$$

In contrast to the variance, the pseudo-variance may take complex values.

The following result relates the pseudo-variance to the real and imaginary parts of z .

Theorem B.10 (Pseudo-Variance Decomposition). *Let z be a complex random variable such that $\mathbb{E}[|z|^2] < \infty$. Then,*

$$J_{zz} = \text{Var}(\text{Re}(z)) - \text{Var}(\text{Im}(z)) + 2i \text{Cov}(\text{Re}(z), \text{Im}(z)).$$

This identity follows from a direct expansion of $(z - \mathbb{E}[z])^2$.

As in the variance case, the pseudo-variance arises as a particular case of a more general definition.

Definition B.11 (Complex Pseudo-Covariance). Let z and w be complex random variables such that $\mathbb{E}[|z|^2] < \infty$ and $\mathbb{E}[|w|^2] < \infty$.

The pseudo-covariance between z and w is defined as

$$J_{zw} := \mathbb{E}[(z - \mathbb{E}[z])(w - \mathbb{E}[w])].$$

Equivalently, by expanding the product,

$$J_{zw} = \mathbb{E}[zw] - \mathbb{E}[z]\mathbb{E}[w].$$

In particular, as mentioned before, when $z = w$ the pseudo-covariance reduces to the pseudo-variance:

$$J_{zz} = \mathbb{E}[(z - \mathbb{E}[z])^2].$$

The pair $(\text{Var}(z), J_{zz})$ fully characterises the second-order structure of a complex random variable. In particular, the pseudo-variance captures the interaction between the real and imaginary parts that is not reflected in the variance alone.

A complex random variable is said to be *proper* (or *circular*) if $J_{zz} = 0$.

B.1.4. Real Representation

The previous results allow a complex random variable to be represented equivalently as a two-dimensional real random vector.

Theorem B.12 (Real Representation of a Complex Random Variable). *Let z be a complex random variable such that $\mathbb{E}[|z|^2] < \infty$, and write*

$$z = x + iy, \quad \text{where } x = \operatorname{Re}(z), \ y = \operatorname{Im}(z).$$

Then:

$$\begin{aligned} \mathbb{E}[z] &= \mathbb{E}[x] + i \mathbb{E}[y], \\ K_{zz} &= \operatorname{Var}(x) + \operatorname{Var}(y), \\ J_{zz} &= \operatorname{Var}(x) - \operatorname{Var}(y) + 2i \operatorname{Cov}(x, y). \end{aligned}$$

Conversely, define the real random vector

$$\mathbf{u} := \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2.$$

Then:

$$\begin{aligned} \mathbb{E}[\mathbf{u}] &= \begin{pmatrix} \operatorname{Re}(\mathbb{E}[z]) \\ \operatorname{Im}(\mathbb{E}[z]) \end{pmatrix}, \\ \operatorname{Cov}(\mathbf{u}) &= \frac{1}{2} \begin{pmatrix} K_{zz} + \operatorname{Re}(J_{zz}) & \operatorname{Im}(J_{zz}) \\ \operatorname{Im}(J_{zz}) & K_{zz} - \operatorname{Re}(J_{zz}) \end{pmatrix}. \end{aligned}$$

This result establishes a correspondence between the second-order structure of a complex random variable, described by $(\mathbb{E}[z], K_{zz}, J_{zz})$, and the second-order structure of the associated real vector $\mathbf{u}$, described by $(\mathbb{E}[\mathbf{u}], \operatorname{Cov}(\mathbf{u}))$.

To conclude, the preceding theorem is illustrated through the following example involving a complex random variable.

Example B.13 (Standard Complex Gaussian Random Variable). *Let $z = x + iy$, where*

$$x \sim \mathcal{N}(0, \tfrac{1}{2}), \quad y \sim \mathcal{N}(0, \tfrac{1}{2}),$$

and x and y are independent.

The associated real vector $\mathbf{u} = (x, y)^\top$ has covariance matrix

$$\operatorname{Cov}(\mathbf{u}) = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}.$$

The random variable z is called a standard complex Gaussian random variable, and is denoted by

$$z \sim \mathcal{CN}(0, 1).$$

It is easy to check that:

$$\begin{aligned} \mathbb{E}[z] &= 0, \\ K_{zz} &= \operatorname{Var}(x) + \operatorname{Var}(y) = \frac{1}{2} + \frac{1}{2} = 1, \\ J_{zz} &= \operatorname{Var}(x) - \operatorname{Var}(y) + 2i \operatorname{Cov}(x, y) = 0. \end{aligned}$$

Since $J_{zz} = 0$, the random variable z is proper (or circular). ♣

Example B.14 (General Complex Gaussian Random Variable). Let $z = x + iy$ be such that the real vector

$$\mathbf{u} = \begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

where

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix}.$$

Then z is called a *complex Gaussian random variable*, denoted by

$$z \sim \mathcal{CN}(\mu_z, K_{zz}, J_{zz}),$$

where

$$\begin{aligned} \mu_z &= \mu_x + i\mu_y, \\ K_{zz} &= \sigma_x^2 + \sigma_y^2, \\ J_{zz} &= \sigma_x^2 - \sigma_y^2 + 2i\sigma_{xy}. \end{aligned}$$



B.2. Multivariate Case

This section outlines the fundamental principles of complex multivariate distributions, extending the concepts given in Section B.1 to the multivariate case. The exposition begins with the definition of a complex random vector, which naturally is an extension of the Definition B.1.

Definition B.15 (Complex random vector). Let $z_1, \dots, z_D$ be complex random variables. The vector

$$\mathbf{z} := (z_1, \dots, z_D)^\top$$

is called a complex random vector in $\mathbb{C}^D$.

Following the same structure as in the previous section, several quantities are required to characterise the distribution of a complex random vector.

B.2.1. Complex Multivariate Expectation

As in the real multivariate case Definition B.2, the expectation of a complex random vector is defined component-wise.

Definition B.16 (Expectation of a Complex Random Vector). Let $\mathbf{z} = (z_1, \dots, z_D)^\top$ be a complex random vector such that $\mathbb{E}[|z_i|] < \infty$ for all $i = 1, \dots, D$.

The expectation of $\mathbf{z}$ is defined as

$$\mathbb{E}[\mathbf{z}] = (\mathbb{E}[z_1], \dots, \mathbb{E}[z_D])^\top.$$

Results such as Theorems B.3 and B.4 can be naturally extended component-wise to the multivariate case.

B.2.2. Complex Covariance

In analogy with the real multivariate case Definition B.5, the covariance matrix describes the covariances between each pair of components of a complex random vector, defined in Definition B.7.

Definition B.17 (Covariance of a Complex Random Vector). Let $\mathbf{z} = (z_1, \dots, z_D)^\top$ be a complex random vector such that $\mathbb{E}[|z|^2] < \infty$ and denote its mean vector by $\boldsymbol{\mu} := \mathbb{E}[\mathbf{z}]$.

The *covariance matrix* of $\mathbf{z}$ is defined as

$$\mathbf{K}_{\mathbf{zz}} := \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger],$$

where $(\cdot)^\dagger$ denotes the Hermitian transpose.

Equivalently to the real case,

$$\mathbf{K}_{\mathbf{zz}} = \mathbb{E}[\mathbf{z}\mathbf{z}^\dagger] - \boldsymbol{\mu}\boldsymbol{\mu}^\dagger.$$

Unlike the real-valued or scalar complex cases, the covariance matrix $\mathbf{K}_{\mathbf{zz}}$ is generally complex-valued. However, its diagonal entries are real and non-negative, as they correspond to the variance of each complex random variable separately.

Theorem B.18 (Properties of the Covariance Matrix). Let $\mathbf{z} \in \mathbb{C}^D$ be a complex random vector with mean $\boldsymbol{\mu} = \mathbb{E}[\mathbf{z}]$ and covariance matrix

$$\mathbf{K}_{\mathbf{zz}} := \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger].$$

Then:

i) $\mathbf{K}_{\mathbf{zz}}$ is Hermitian:

$$\mathbf{K}_{\mathbf{zz}} = \mathbf{K}_{\mathbf{zz}}^\dagger.$$

ii) $\mathbf{K}_{\mathbf{zz}}$ is positive semidefinite (PSD):

$$\mathbf{x}^\dagger \mathbf{K}_{\mathbf{zz}} \mathbf{x} \geq 0, \quad \forall \mathbf{x} \in \mathbb{C}^D.$$

iii) The diagonal entries are the variances of the components:

$$(\mathbf{K}_{\mathbf{zz}})_{kk} = \text{Var}(z_k) \quad \text{for } k = 1, \dots, D.$$

Proof. i) $\mathbf{K}_{\mathbf{zz}}$ is Hermitian:

$$\mathbf{K}_{\mathbf{zz}}^\dagger = (\mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger])^\dagger.$$

Using $(\mathbf{AB})^\dagger = \mathbf{B}^\dagger \mathbf{A}^\dagger$ and linearity of expectation,

$$\mathbf{K}_{\mathbf{zz}}^\dagger = \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger]^\dagger.$$

But

$$((\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger)^\dagger = (\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger.$$

Hence

$$\mathbf{K}_{\mathbf{z}\mathbf{z}}^\dagger = \mathbf{K}_{\mathbf{z}\mathbf{z}}.$$

ii) $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ is positive semidefinite:

For any $\mathbf{x} \in \mathbb{C}^D$,

$$\mathbf{x}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{x} = \mathbf{x}^\dagger \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger] \mathbf{x}.$$

By linearity of expectation,

$$\mathbf{x}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{x} = \mathbb{E}[\mathbf{x}^\dagger (\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger \mathbf{x}].$$

Note that

$$\mathbf{x}^\dagger (\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\dagger \mathbf{x} = |(\mathbf{z} - \boldsymbol{\mu})^\dagger \mathbf{x}|^2.$$

Since the squared modulus is non-negative, it follows that

$$\mathbf{x}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{x} = \mathbb{E}[|(\mathbf{z} - \boldsymbol{\mu})^\dagger \mathbf{x}|^2] \geq 0.$$

Thus $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ is positive semidefinite.

iii) The diagonal entries are the variances of the components:

The (k, k) -entry of $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ is

$$(\mathbf{K}_{\mathbf{z}\mathbf{z}})_{kk} = \mathbb{E}[(z_k - \mu_k)(\overline{z_k - \mu_k})].$$

Hence

$$(\mathbf{K}_{\mathbf{z}\mathbf{z}})_{kk} = \mathbb{E}[|z_k - \mu_k|^2] = \text{Var}(z_k).$$

□

Just as in the scalar case, the covariance matrix $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ is not sufficient to fully characterise the second-order properties of a complex random vector. In general, the pseudo-covariance matrix must also be considered.

B.2.3. Pseudo-Covariance

In addition to the covariance matrix, a second matrix is required to fully describe the second-order properties of a complex random vector. That matrix will be the *pseudo-covariance matrix*.

Similarly to the covariance matrix, given a complex random vector $\mathbf{z} \in \mathbb{C}^D$, its pseudo-covariance matrix represents at its position (i, j) the pseudo-covariance between the pair (z_i, z_j) .

Remember that the pseudo-covariance of a pair of complex random variables was defined in Definition B.11

Definition B.19 (Pseudo-Covariance of a Complex Random Vector). Consider a complex random vector $\mathbf{z} = (z_1, \dots, z_D)^\top$ such that $\mathbb{E}[|\mathbf{z}|^2] < \infty$, and denote its mean vector by $\boldsymbol{\mu} := \mathbb{E}[\mathbf{z}]$.

The *pseudo-covariance matrix* of $\mathbf{z}$ is defined as

$$\mathbf{J}_{\mathbf{zz}} := \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\top],$$

where $(\cdot)^\top$ denotes the transpose.

Similarly to the covariance case Definition B.17, but using the transpose operator, and extending the scalar case, it follows that

$$\mathbf{J}_{\mathbf{zz}} = \mathbb{E}[\mathbf{zz}^\top] - \boldsymbol{\mu}\boldsymbol{\mu}^\top.$$

The entries of $\mathbf{J}_{\mathbf{zz}}$ are given by

$$(\mathbf{J}_{\mathbf{zz}})_{ij} = \mathbb{E}[(z_i - \mu_i)(z_j - \mu_j)] \quad \text{for } i, j = 1, \dots, D.$$

In contrast to the covariance properties given in Theorem B.18, the derived pseudo-covariance matrix is generally not positive semidefinite. However, it still presents an important property.

Theorem B.20 (Property of the Pseudo-Covariance Matrix). *Let $\mathbf{z} \in \mathbb{C}^D$ be a complex random vector with mean $\boldsymbol{\mu} = \mathbb{E}[\mathbf{z}]$ and pseudo-covariance matrix*

$$\mathbf{J}_{\mathbf{zz}} := \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\top].$$

Then $\mathbf{J}_{\mathbf{zz}}$ is symmetric:

$$\mathbf{J}_{\mathbf{zz}} = \mathbf{J}_{\mathbf{zz}}^\top,$$

Proof. Using the identity $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ and the linearity of the expectation,

$$\mathbf{J}_{\mathbf{zz}}^\top = \mathbb{E}\left[\left((\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\top\right)^\top\right] = \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^\top] = \mathbf{J}_{\mathbf{zz}}.$$

□

The pair $(\mathbf{K}_{\mathbf{zz}}, \mathbf{J}_{\mathbf{zz}})$ completely characterises the second-order structure of a complex random vector.

A complex random vector is said to be *proper* (or *circular*) if

$$\mathbf{J}_{\mathbf{zz}} = \mathbf{0}.$$

B.2.4. Real Representation

Looking at Theorem B.12, there is correspondence between a complex random variable and its associated bidimensional real random vector containing real and imaginary parts. Therefore, this motivates identifying a complex-valued random vector in $\mathbb{C}^D$ with a real-valued random vector in $\mathbb{R}^{2D}$.

Theorem B.21 (Real Representation of a Complex Random Vector). *Let $\mathbf{z} \in \mathbb{C}^D$ be a complex random vector such that $\mathbb{E}[|\mathbf{z}|^2] < \infty$, and let $\boldsymbol{\mu}, \mathbf{K}_{\mathbf{zz}}, \mathbf{J}_{\mathbf{zz}}$ be its mean, covariance matrix and pseudo-covariance matrix, respectively.*

Write

$$\mathbf{z} = \mathbf{x} + i\mathbf{y}, \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^D.$$

Then the following identities hold.

$$\mathbb{E}[\mathbf{z}] = \mathbb{E}[\mathbf{x}] + i\mathbb{E}[\mathbf{y}],$$

$$\mathbf{K}_{\mathbf{zz}} = \text{Cov}(\mathbf{x}) + \text{Cov}(\mathbf{y}) + i(\text{Cov}(\mathbf{y}, \mathbf{x}) - \text{Cov}(\mathbf{x}, \mathbf{y})),$$

$$\mathbf{J}_{\mathbf{zz}} = \text{Cov}(\mathbf{x}) - \text{Cov}(\mathbf{y}) + i(\text{Cov}(\mathbf{y}, \mathbf{y}) + \text{Cov}(\mathbf{x}, \mathbf{y})).$$

Conversely, define a real random vector in $\mathbb{R}^{2D}$ such as

$$\mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \in \mathbb{R}^{2D}.$$

Then the following identities hold.

$$\mathbb{E}[\mathbf{u}] = \begin{pmatrix} \text{Re}(\mathbb{E}[\mathbf{z}]) \\ \text{Im}(\mathbb{E}[\mathbf{z}]) \end{pmatrix},$$

$$\text{Cov}(\mathbf{u}) = \frac{1}{2} \begin{pmatrix} \text{Re}(\mathbf{K}_{\mathbf{zz}} + \mathbf{J}_{\mathbf{zz}}) & -\text{Im}(\mathbf{K}_{\mathbf{zz}} - \mathbf{J}_{\mathbf{zz}}) \\ \text{Im}(\mathbf{K}_{\mathbf{zz}} + \mathbf{J}_{\mathbf{zz}}) & \text{Re}(\mathbf{K}_{\mathbf{zz}} - \mathbf{J}_{\mathbf{zz}}) \end{pmatrix}.$$

Substituting $\mathbf{z} = \mathbf{x} + i\mathbf{y}$ into the definitions of covariance and pseudo-covariance, using the linearity of expectation, expanding and separating real and imaginary parts, the converse representation follows by block matrix identification.

In the same way as in the previous Section B.1, an example of a complex random vector illustrates the last formulated theorem.

Example B.22 (General Multivariate Complex Gaussian Random Vector). *Let $\mathbf{z} = \mathbf{x} + i\mathbf{y} \in \mathbb{C}^D$ be a complex random vector such that the associated real vector*

$$\mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \in \mathbb{R}^{2D}$$

satisfies

$$\mathbf{u} \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{u}}, \boldsymbol{\Sigma}_{\mathbf{u}}),$$

where

$$\boldsymbol{\mu}_{\mathbf{u}} = \begin{pmatrix} \boldsymbol{\mu}_{\mathbf{x}} \\ \boldsymbol{\mu}_{\mathbf{y}} \end{pmatrix}, \quad \boldsymbol{\Sigma}_{\mathbf{u}} = \begin{pmatrix} \boldsymbol{\Sigma}_{\mathbf{xx}} & \boldsymbol{\Sigma}_{\mathbf{xy}} \\ \boldsymbol{\Sigma}_{\mathbf{yx}} & \boldsymbol{\Sigma}_{\mathbf{yy}} \end{pmatrix},$$

with $\boldsymbol{\mu}_x, \boldsymbol{\mu}_y \in \mathbb{R}^D$, $\boldsymbol{\Sigma}_{xx}, \boldsymbol{\Sigma}_{yy} \in \mathbb{R}^{D \times D}$ symmetric and positive semidefinite and $\boldsymbol{\Sigma}_{xy} = \boldsymbol{\Sigma}_{yx}^\top$.

Then $\mathbf{z}$ is called a **multivariate complex Gaussian random vector**, denoted by

$$\mathbf{z} \sim \mathcal{CN}(\boldsymbol{\mu}_z, \mathbf{K}_{zz}, \mathbf{J}_{zz}),$$

where, using the expressions given by Theorem B.21,

$$\begin{aligned}\boldsymbol{\mu}_z &= \boldsymbol{\mu}_x + i\boldsymbol{\mu}_y, \\ \mathbf{K}_{zz} &= \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu}_z)(\mathbf{z} - \boldsymbol{\mu}_z)^\dagger] = \boldsymbol{\Sigma}_{xx} + \boldsymbol{\Sigma}_{yy} + i(\boldsymbol{\Sigma}_{yx} - \boldsymbol{\Sigma}_{xy}), \\ \mathbf{J}_{zz} &= \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu}_z)(\mathbf{z} - \boldsymbol{\mu}_z)^\top] = \boldsymbol{\Sigma}_{xx} - \boldsymbol{\Sigma}_{yy} + i(\boldsymbol{\Sigma}_{xy} + \boldsymbol{\Sigma}_{yx}).\end{aligned}$$

Theorem B.21 also shows that it is possible to describe the real vector $\mathbf{u}$ (real and imaginary parts of the complex vector) in terms of the complex distribution. Applying the formulas, if $\mathbf{z} \sim \mathcal{CN}_D(\boldsymbol{\mu}_z, \mathbf{K}_{zz}, \mathbf{J}_{zz})$, it follows that

$$\mathbf{u} \sim \mathcal{N}_{2D}(\boldsymbol{\mu}_u, \boldsymbol{\Sigma}_u),$$

where

$$\boldsymbol{\mu}_u = \begin{pmatrix} \text{Re}(\boldsymbol{\mu}_z) \\ \text{Im}(\boldsymbol{\mu}_z) \end{pmatrix}, \quad \boldsymbol{\Sigma}_u = \frac{1}{2} \begin{pmatrix} \text{Re}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) & -\text{Im}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \\ \text{Im}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) & \text{Re}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \end{pmatrix}.$$

♣

Example B.23 (Linear Transformation of a Complex Gaussian Vector). *Let*

$$\mathbf{z} \sim \mathcal{CN}(\boldsymbol{\mu}_z, \mathbf{K}_{zz}, \mathbf{J}_{zz})$$

be a complex Gaussian random vector in $\mathbb{C}^D$, and let $\mathbf{A} \in \mathbb{C}^{d \times D}$ and $\mathbf{b} \in \mathbb{C}^d$ be deterministic. Define $\mathbf{w} = \mathbf{A}\mathbf{z} + \mathbf{b}$. Then $\mathbf{w}$ is complex Gaussian and

$$\mathbf{w} \sim \mathcal{CN}(\boldsymbol{\mu}_w, \mathbf{K}_{ww}, \mathbf{J}_{ww}),$$

where

$$\begin{aligned}\boldsymbol{\mu}_w &= \mathbf{A}\boldsymbol{\mu}_z + \mathbf{b}, \\ \mathbf{K}_{ww} &= \mathbf{A}\mathbf{K}_{zz}\mathbf{A}^\dagger, \\ \mathbf{J}_{ww} &= \mathbf{A}\mathbf{J}_{zz}\mathbf{A}^\top.\end{aligned}$$

A brief explanation regarding the derivation of these formulas is provided below.

In the case of real-valued Gaussian vectors, if $\mathbf{T} \in \mathbb{R}^{2d \times 2D}$, $\mathbf{c} \in \mathbb{R}^{2d}$ and $\mathbf{u} \in \mathbb{R}^{2D}$ satisfies

$$\mathbf{u} \sim \mathcal{N}_{2D}(\boldsymbol{\mu}_u, \boldsymbol{\Sigma}_u),$$

then

$$(B.1) \quad \mathbf{T}\mathbf{u} + \mathbf{c} \sim \mathcal{N}_{2d}(\mathbf{T}\boldsymbol{\mu}_u + \mathbf{c}, \mathbf{T}\boldsymbol{\Sigma}_u\mathbf{T}^\top).$$

Therefore, by defining

$$\mathbf{u} = \begin{pmatrix} \text{Re}(\mathbf{z}) \\ \text{Im}(\mathbf{z}) \end{pmatrix} \in \mathbb{R}^{2D}, \quad \mathbf{T} = \begin{pmatrix} \text{Re}(\mathbf{A}) & -\text{Im}(\mathbf{A}) \\ \text{Im}(\mathbf{A}) & \text{Re}(\mathbf{A}) \end{pmatrix} \in \mathbb{R}^{2d \times 2D}, \quad \mathbf{c} = \begin{pmatrix} \text{Re}(\mathbf{b}) \\ \text{Im}(\mathbf{b}) \end{pmatrix} \in \mathbb{R}^{2d},$$

and applying the formulas (B.1), it follows that

$$\mathbf{T}\mathbf{u} + \mathbf{c} = \begin{pmatrix} \text{Re}(\mathbf{A}\mathbf{z} + \mathbf{b}) \\ \text{Im}(\mathbf{A}\mathbf{z} + \mathbf{b}) \end{pmatrix} \sim \mathcal{N}_{2d}(\mathbf{T}\boldsymbol{\mu}_{\mathbf{u}} + \mathbf{c}, \mathbf{T}\boldsymbol{\Sigma}_{\mathbf{u}}\mathbf{T}^\top).$$

where the mean is

$$\mathbf{T}\boldsymbol{\mu}_{\mathbf{u}} + \mathbf{c} = \begin{pmatrix} \text{Re}(\mathbf{A}) \text{Re}(\boldsymbol{\mu}_{\mathbf{z}}) - \text{Im}(\mathbf{A}) \text{Im}(\boldsymbol{\mu}_{\mathbf{z}}) + \text{Re}(b) \\ \text{Im}(\mathbf{A}) \text{Re}(\boldsymbol{\mu}_{\mathbf{z}}) + \text{Re}(\mathbf{A}) \text{Im}(\boldsymbol{\mu}_{\mathbf{z}}) + \text{Im}(b) \end{pmatrix} = \begin{pmatrix} \text{Re}(A\boldsymbol{\mu}_{\mathbf{z}} + b) \\ \text{Im}(A\boldsymbol{\mu}_{\mathbf{z}} + b) \end{pmatrix},$$

and the covariance matrix is obtained by expanding the product $\mathbf{T}\boldsymbol{\Sigma}_{\mathbf{u}}\mathbf{T}^\top$.

Since $\mathbf{w} = \mathbf{A}\mathbf{z} + \mathbf{b}$, the expressions of $\boldsymbol{\mu}_{\mathbf{w}}, \mathbf{K}_{\mathbf{ww}}, \mathbf{J}_{\mathbf{ww}}$ can be explicitly derived by applying the formulas given in Theorem B.21. One identifies the corresponding mean, covariance and pseudo-covariance derived expressions as the formulas

$$\boldsymbol{\mu}_{\mathbf{w}} = \mathbf{A}\boldsymbol{\mu}_{\mathbf{z}} + \mathbf{b}, \quad \mathbf{K}_{\mathbf{ww}} = \mathbf{A}\mathbf{K}_{\mathbf{zz}}\mathbf{A}^\dagger, \quad \mathbf{J}_{\mathbf{ww}} = \mathbf{A}\mathbf{J}_{\mathbf{zz}}\mathbf{A}^\top.$$

The resulting expressions of $\boldsymbol{\mu}_{\mathbf{w}}, \mathbf{K}_{\mathbf{ww}}, \mathbf{J}_{\mathbf{ww}}$ coincide with the formulas at the statement of the example. 

Before presenting the last theorem of the section, an application of the previous Example B.23 is shown below.

Example B.24 (Complex-Valued Wiener Process). *Let $\mathbf{F}$ be the conjugate-paired DFT matrix defined in Section A.4, and let $\mathbf{W}(\tau)$ be a D -dimensional real Wiener process for $\tau \in [0, T] \subset \mathbb{R}$.*

Define $\widetilde{\mathbf{W}}(\tau) := \mathbf{F}\mathbf{W}(\tau)$. The mean, covariance and pseudo-covariance of the distribution $\widetilde{\mathbf{W}}(\tau)$ are derived below.

To use the expressions given in Example B.23, it is first determined the complex distribution of the Wiener process $\mathbf{W}(\tau) \in \mathbb{R}^D$.

Even though $\mathbf{W}(\tau)$ is a real Wiener process, it can be embedded into the complex framework by identifying it with a complex random vector with zero imaginary part. This follows directly from the fact that a real distribution is generalised to a complex distribution by choosing complex-valued mean equal to real-valued mean and complex-valued covariance and pseudo-covariance matrices both equal to real-valued covariance matrix. Theorem B.21 gives an easy proof of this fact. Therefore,

$$\mathbf{W}(\tau) \sim \mathcal{N}_D(\mathbf{0}, \tau I_D) \sim \mathcal{CN}_D(\mathbf{0}, \tau I_D, \tau I_D)$$

Now, the formulas obtained in the Example B.23 are applied over $\widetilde{\mathbf{W}}(\tau) = \mathbf{F}\mathbf{W}(\tau)$:

$$\widetilde{\mathbf{W}}(\tau) = \mathbf{F}\mathbf{W}(\tau) \sim \mathcal{CN}_D(0, \tau I_D, \tau \mathbf{F}\mathbf{F}^\top).$$

The problem has been reduced to the calculation of the pseudo-covariance matrix $\mathbf{F} \mathbf{F}^\top$. The result of this matrix operation is explained in Theorem A.4, which states that

$$\mathbf{F} \mathbf{F}^\top = \text{diag} \left(1, \underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}}_{\text{pairs } (k, -k)}, \epsilon \right),$$

where:

- the first entry corresponds to the DC mode $k = 0$,
- each 2×2 block corresponds to a conjugate pair $(k, -k)$,
- $\epsilon = 1$ if D is even (Nyquist mode), and is absent if D is odd.

Moreover, by defining

$$U := \begin{pmatrix} \text{Re}(\mathbf{F} \mathbf{W}(\tau)) \\ \text{Im}(\mathbf{F} \mathbf{W}(\tau)) \end{pmatrix},$$

and using the formula given by Theorem B.21,

$$\mathbf{u}(\tau) \sim \mathcal{N}_{2D}(\mathbf{0}, \mathbf{\Sigma}(\tau)),$$

where

$$\mathbf{\Sigma}(\tau) = \frac{\tau}{2} \begin{pmatrix} I_D + \mathbf{F} \mathbf{F}^\top & 0 \\ 0 & I_D - \mathbf{F} \mathbf{F}^\top \end{pmatrix}.$$

The variance of the Wiener process is preserved, while the pseudo-covariance matrix captures the dependence structure introduced by the Fourier transform. In particular, although the real and imaginary parts are uncorrelated, the pseudo-covariance matrix is non-zero. Therefore, $\widetilde{\mathbf{W}}(\tau)$ is not a proper complex Gaussian process. Consequently, $\widetilde{\mathbf{W}}(\tau)$ provides an example of a non-proper complex-valued Wiener process. ♣

To conclude this section, the following theorem summarises the necessary conditions for the parameters of a complex distribution to be well defined. For a real random vector, the covariance matrix must be symmetric and positive semidefinite. The next result provides the equivalent characterisation in the complex setting.

Theorem B.25 (Equivalence of Real and Augmented Covariance). *Let $\mathbf{z} \in \mathbb{C}^D$ be a complex random vector with finite second moments, and let*

$$\mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \in \mathbb{R}^{2D}$$

be its associated real representation.

Then, the following statements are equivalent:

1. Real covariance matrix $\text{Cov}(\mathbf{u})$ is symmetric and positive semidefinite (PSD).

2. Augmented complex covariance matrix

$$\mathcal{C}_{\mathbf{z}} = \begin{pmatrix} \mathbf{K}_{\mathbf{z}\mathbf{z}} & \mathbf{J}_{\mathbf{z}\mathbf{z}} \\ \overline{\mathbf{J}_{\mathbf{z}\mathbf{z}}} & \overline{\mathbf{K}_{\mathbf{z}\mathbf{z}}} \end{pmatrix}$$

is Hermitian and positive semidefinite.

Proof. (i) **Real covariance matrix symmetric iff augmented covariance matrix Hermitian.**

By matrix block identification, the matrix $\mathcal{C}_{\mathbf{z}}$ is Hermitian if and only if

$$\mathbf{K}_{\mathbf{z}\mathbf{z}} = \mathbf{K}_{\mathbf{z}\mathbf{z}}^\dagger \quad \text{and} \quad \mathbf{J}_{\mathbf{z}\mathbf{z}} = \mathbf{J}_{\mathbf{z}\mathbf{z}}^\top.$$

Using the representation of the real covariance matrix $\text{Cov}(\mathbf{u})$ in terms of $\mathbf{J}_{\mathbf{z}\mathbf{z}}$ and $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ given in Theorem B.21, one verifies that $\text{Cov}(\mathbf{u})$ is symmetric if and only if $\mathbf{K}_{\mathbf{z}\mathbf{z}}$ is Hermitian and $\mathbf{J}_{\mathbf{z}\mathbf{z}}$ is symmetric, which proves the equivalence.

(ii) **Real covariance matrix PSD iff augmented complex covariance matrix PSD.**

Let $\mathbf{v} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \in \mathbb{R}^{2D}$ and let $\mathbf{z} = \mathbf{x} + i\mathbf{y} \in \mathbb{C}^D$.

The following identity holds:

$$(B.2) \quad \mathbf{v}^\top \text{Cov}(\mathbf{u}) \mathbf{v} = \frac{1}{2} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix}^\dagger \mathcal{C}_{\mathbf{z}} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix}.$$

After that, the equivalence is immediate.

Expanding the right-hand side,

$$\begin{aligned} \frac{1}{2} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix}^\dagger \mathcal{C}_{\mathbf{z}} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix} &= \frac{1}{2} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix}^\dagger \begin{pmatrix} \mathbf{K}_{\mathbf{z}\mathbf{z}} & \mathbf{J}_{\mathbf{z}\mathbf{z}} \\ \overline{\mathbf{J}_{\mathbf{z}\mathbf{z}}} & \overline{\mathbf{K}_{\mathbf{z}\mathbf{z}}} \end{pmatrix} \begin{pmatrix} \mathbf{z} \\ \overline{\mathbf{z}} \end{pmatrix} \\ &= \frac{1}{2} \left(\mathbf{z}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{z} + \mathbf{z}^\dagger \mathbf{J}_{\mathbf{z}\mathbf{z}} \overline{\mathbf{z}} + \mathbf{z}^\top \overline{\mathbf{J}_{\mathbf{z}\mathbf{z}}} \mathbf{z} + \mathbf{z}^\top \overline{\mathbf{K}_{\mathbf{z}\mathbf{z}}} \overline{\mathbf{z}} \right) \\ &= \frac{1}{2} \left(\mathbf{z}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{z} + \mathbf{z}^\dagger \mathbf{J}_{\mathbf{z}\mathbf{z}} \overline{\mathbf{z}} + \overline{\mathbf{z}^\dagger \mathbf{J}_{\mathbf{z}\mathbf{z}} \mathbf{z}} + \overline{\mathbf{z}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \overline{\mathbf{z}}} \right) \\ &= \text{Re}(\mathbf{z}^\dagger \mathbf{K}_{\mathbf{z}\mathbf{z}} \mathbf{z}) + \text{Re}(\mathbf{z}^\dagger \mathbf{J}_{\mathbf{z}\mathbf{z}} \overline{\mathbf{z}}) \end{aligned}$$

Expanding the left-hand side using the block expression of $\text{Cov}(\mathbf{u})$ given by Theorem B.21,

$$\begin{aligned}
\mathbf{v}^\top \text{Cov}(\mathbf{u}) \mathbf{v} &= \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}^\top \frac{1}{2} \begin{pmatrix} \text{Re}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) & -\text{Im}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \\ \text{Im}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) & \text{Re}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \end{pmatrix} \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \\
&= \frac{1}{2} \left(\mathbf{x}^\top \text{Re}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) \mathbf{x} - \mathbf{x}^\top \text{Im}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \mathbf{y} \right) \\
&\quad + \frac{1}{2} \left(\mathbf{y}^\top \text{Im}(\mathbf{K}_{zz} + \mathbf{J}_{zz}) \mathbf{x} + \mathbf{y}^\top \text{Re}(\mathbf{K}_{zz} - \mathbf{J}_{zz}) \mathbf{y} \right) \\
&= \frac{1}{2} \left(\mathbf{x}^\top \text{Re}(\mathbf{K}_{zz}) \mathbf{x} + \mathbf{y}^\top \text{Re}(\mathbf{K}_{zz}) \mathbf{y} - \mathbf{x}^\top \text{Im}(\mathbf{K}_{zz}) \mathbf{y} + \mathbf{y}^\top \text{Im}(\mathbf{K}_{zz}) \mathbf{x} \right) \\
&\quad + \frac{1}{2} \left(\mathbf{x}^\top \text{Re}(\mathbf{J}_{zz}) \mathbf{x} - \mathbf{y}^\top \text{Re}(\mathbf{J}_{zz}) \mathbf{y} + \mathbf{x}^\top \text{Im}(\mathbf{J}_{zz}) \mathbf{y} + \mathbf{y}^\top \text{Im}(\mathbf{J}_{zz}) \mathbf{x} \right) \\
&= \text{Re}(\mathbf{z}^\dagger \mathbf{K}_{zz} \mathbf{z}) + \text{Re}(\mathbf{z}^\dagger \mathbf{J}_{zz} \bar{\mathbf{z}}),
\end{aligned}$$

which proves (B.2). The equivalence of positive semidefiniteness follows immediately. $\square$

APPENDIX C

Closed-Form Solutions for Linear Noise Schedules

C.1. Isotropic Variance Preserving Diffusion Model

This section derives the explicit closed-form distribution for the time-dependent and isotropic VP process (3.1) under the linear drift schedule

$$\lambda(\tau) = \lambda_{\min} + \frac{\tau - t_0}{T - t_0} (\lambda_{\max} - \lambda_{\min}).$$

In this case, the integral form has an immediate closed-form solution, which is

$$\begin{aligned} \int_{t_0}^{\tau} \lambda(u) du &= \int_{t_0}^{\tau} \left(\lambda_{\min} + \frac{u - t_0}{T - t_0} (\lambda_{\max} - \lambda_{\min}) \right) du \\ &= \lambda_{\min} (\tau - t_0) + \frac{(\tau - t_0)^2}{2(T - t_0)} (\lambda_{\max} - \lambda_{\min}). \end{aligned}$$

Hence, by denoting $\Delta_{\lambda} := \lambda_{\max} - \lambda_{\min}$, the marginal Gaussian distribution solution (3.1) is

$$\mathbf{x}(\tau) \sim \mathcal{N}_D \left(e^{-\lambda_{\min}(\tau - t_0) - \frac{(\tau - t_0)^2}{2(T - t_0)} \Delta_{\lambda}} \mathbf{x}_0, \left(1 - e^{-2\lambda_{\min}(\tau - t_0) - \frac{(\tau - t_0)^2}{T - t_0} \Delta_{\lambda}} \right) I_D \right).$$

C.2. General Variance Preserving Diffusion Model

An important practical application of Section 3.3.3 is the case of a linear schedule for the time-dependent drift coefficient, where the diffusion coefficient must maintain the generalised Variance Preserving property.

Let $\mathbf{K}(\tau), \mathbf{S}(\tau) \in \mathbb{R}^{D \times D}$ be time-dependent matrices defined on $\tau \in [t_0, T]$, and assume they are both **circulant** and **symmetric**. Consider the stochastic process $\mathbf{x}(\tau) \in \mathbb{R}^D$ governed by the Ornstein–Uhlenbeck equation and assume in addition the following **Variance Preserving** condition

$$\mathbf{S}(\tau)^2 = 2\mathbf{K}(\tau), \quad \forall \tau \in [t_0, T].$$

Assume that the drift Fourier eigenvalue matrix follows the linear schedule

$$\mathbf{\Lambda}(\tau) = \mathbf{\Lambda}_{\min} + \frac{\tau - t_0}{T - t_0} (\mathbf{\Lambda}_{\max} - \mathbf{\Lambda}_{\min}),$$

where

$$\mathbf{\Lambda}_{\min}, \mathbf{\Lambda}_{\max}$$

are real-valued diagonal matrices satisfying

$$\mathbf{\Lambda}(\tau) \geq 0 \quad \text{for all } \tau \in [t_0, T].$$

Under the Variance Preserving property, the diffusion eigenvalues matrix must satisfy the following schedule

$$\mathbf{B}(\tau) = \sqrt{2\mathbf{\Lambda}(\tau)} = \sqrt{2 \left(\mathbf{\Lambda}_{\min} + \frac{\tau - t_0}{T - t_0} (\mathbf{\Lambda}_{\max} - \mathbf{\Lambda}_{\min}) \right)},$$

where the square root is understood diagonally. Define $\Delta_{\Lambda} := \mathbf{\Lambda}_{\max} - \mathbf{\Lambda}_{\min}$. Under these assumptions,

$$\int_{t_0}^{\tau} \mathbf{\Lambda}(u) du = (\tau - t_0) \mathbf{\Lambda}_{\min} + \frac{(\tau - t_0)^2}{2(T - t_0)} \Delta_{\Lambda}.$$

Therefore, applying the formulas derived in Section 3.3.3 of Chapter 3:

$$\boxed{\mathbf{y}(\tau) \sim \mathcal{CN}_D(\boldsymbol{\mu}_{\mathbf{y}}(\tau), \mathbf{C}_{\mathbf{y}}(\tau), \mathbf{P}_{\mathbf{y}}(\tau))}$$

where the mean simplifies to

$$\boldsymbol{\mu}_{\mathbf{y}}(\tau) = e^{\left(-(\tau - t_0) \mathbf{\Lambda}_{\min} - \frac{(\tau - t_0)^2}{2(T - t_0)} \Delta_{\Lambda} \right)} \mathbf{y}_0$$

and the covariance formula simplifies to

$$\mathbf{C}_{\mathbf{y}}(\tau) = I_D - e^{\left(-2(\tau - t_0) \mathbf{\Lambda}_{\min} - \frac{(\tau - t_0)^2}{T - t_0} \Delta_{\Lambda} \right)},$$

where all operations are understood element-wise due to the diagonal structure. The pseudo-covariance formula simplifies to

- **DC mode:**

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{0,0} = 1 - e^{\left(-2(\tau - t_0) \lambda_{0,\min} - \frac{(\tau - t_0)^2}{T - t_0} \Delta_{\lambda_0} \right)},$$

- **Conjugate pairs:** for $m = 1, \dots, \frac{D}{2} - 1$ if D is even, or $m = 1, \dots, \frac{D-1}{2}$ if D is odd,

$$[\mathbf{P}_{\mathbf{y}}(\tau)]_{2m-1, 2m} = 1 - e^{\left(-2(\tau - t_0) \lambda_{m,\min} - \frac{(\tau - t_0)^2}{T - t_0} \Delta_{\lambda_m} \right)},$$

- **Nyquist mode (if D is even):**

$$[\mathbf{P}_y(\tau)]_{D-1, D-1} = 1 - e^{\left(-2(\tau-t_0) \lambda_{\frac{D}{2}, \min} - \frac{(\tau-t_0)^2}{T-t_0} \Delta_{\lambda_{\frac{D}{2}}}\right)},$$

where $\Delta_{\lambda_m} := \lambda_{m, \max} - \lambda_{m, \min}$ for $m = 0, \dots, D-1$.

- Otherwise,

$$[\mathbf{P}_y(\tau)]_{j,k} = 0.$$

Moreover, the solution in physical space $\mathbf{x}(\tau)$ also simplifies to

$$\boxed{\mathbf{x}(\tau) \sim \mathcal{N}(\boldsymbol{\mu}_x(\tau), \boldsymbol{\Sigma}_x(\tau))}$$

where

$$\begin{aligned} \boldsymbol{\mu}_x(\tau) &= e^{\left(-(\tau-t_0) \mathbf{K}_{\min} - \frac{(\tau-t_0)^2}{2(T-t_0)} \Delta_{\mathbf{K}}\right)} \mathbf{x}_0, \\ \boldsymbol{\Sigma}_x(\tau) &= I_D - e^{\left(-2(\tau-t_0) \mathbf{K}_{\min} - \frac{(\tau-t_0)^2}{T-t_0} \Delta_{\mathbf{K}}\right)}, \end{aligned}$$

where $\Delta_{\mathbf{K}} := \mathbf{K}_{\max} - \mathbf{K}_{\min}$ with $\mathbf{K}_{\min} = \mathbf{F}^\dagger \Lambda_{\min} \mathbf{F}$ and $\mathbf{K}_{\max} = \mathbf{F}^\dagger \Lambda_{\max} \mathbf{F}$.

Since all circulant matrices commute, the matrix exponential is consistent with the diagonal Fourier representation, and the covariance inherits the generalised Variance Preserving form.

APPENDIX D

Mathematical Proofs

D.1. Lyapunov Time-Dependent Equation

Theorem D.1 (Differential Time-Dependent Lyapunov Equation). *Let $\mathbf{x}(\tau)$ satisfy the time-dependent Ornstein–Uhlenbeck SDE*

$$d\mathbf{x}(\tau) = -\mathbf{K}(\tau) \mathbf{x}(\tau) d\tau + \mathbf{S}(\tau) d\mathbf{W}(\tau).$$

Then the covariance matrix $\Sigma(\tau) := \text{Cov}(\mathbf{x}(\tau))$ satisfies the Lyapunov-type differential equation

$$(D.1) \quad \frac{d}{d\tau} \Sigma(\tau) = -\mathbf{K}(\tau) \Sigma(\tau) - \Sigma(\tau) \mathbf{K}(\tau)^\top + \mathbf{S}(\tau) \mathbf{S}(\tau)^\top.$$

Proof. Let $\boldsymbol{\mu}(\tau) := \mathbb{E}[\mathbf{x}(\tau)]$. The covariance matrix can be written as:

$$\Sigma(\tau) := \mathbb{E} \left[(\mathbf{x}(\tau) - \boldsymbol{\mu}(\tau)) (\mathbf{x}(\tau) - \boldsymbol{\mu}(\tau))^\top \right] = \mathbb{E}[\mathbf{x}(\tau) \mathbf{x}(\tau)^\top] - \boldsymbol{\mu}(\tau) \boldsymbol{\mu}(\tau)^\top.$$

Differentiating with respect to time,

$$(D.2) \quad \frac{d}{d\tau} \Sigma(\tau) = \frac{d}{d\tau} \mathbb{E}[\mathbf{x}(\tau) \mathbf{x}(\tau)^\top] - \dot{\boldsymbol{\mu}}(\tau) \boldsymbol{\mu}(\tau)^\top - \boldsymbol{\mu}(\tau) \dot{\boldsymbol{\mu}}(\tau)^\top.$$

Using the linearity of expectation and the SDE, the mean satisfies

$$d\boldsymbol{\mu}(\tau) = \mathbb{E}[d\mathbf{x}(\tau)] = -\mathbf{K}(\tau) \boldsymbol{\mu}(\tau) d\tau \implies \dot{\boldsymbol{\mu}}(\tau) = -\mathbf{K}(\tau) \boldsymbol{\mu}(\tau).$$

Applying Itô's product rule (see [23, Theorem 4.2.1]) for $\mathbf{x}(\tau) \mathbf{x}(\tau)^\top$:

$$d \left(\mathbf{x}(\tau) \mathbf{x}(\tau)^\top \right) = d\mathbf{x}(\tau) \mathbf{x}(\tau)^\top + \mathbf{x}(\tau) d\mathbf{x}(\tau)^\top + d\mathbf{x}(\tau) d\mathbf{x}(\tau)^\top.$$

For notational simplicity, the explicit time dependence is omitted in the following computations. Substituting the expression of $d\mathbf{x}(\tau)$ and using the Itô isometry rules (see [23, Theorems 4.1.2 and 4.2.1])

$$(d\tau)^2 = 0, \quad d\tau d\mathbf{W}(\tau) = 0, \quad d\mathbf{W}(\tau) d\mathbf{W}(\tau)^\top = I_D d\tau,$$

it follows that

$$\begin{aligned} d(\mathbf{x} \mathbf{x}^\top) &= (-\mathbf{K} \mathbf{x} d\tau + \mathbf{S} d\mathbf{W}) \mathbf{x}^\top + \mathbf{x} (-\mathbf{x}^\top \mathbf{K}^\top d\tau + d\mathbf{W}^\top \mathbf{S}^\top) + \mathbf{S} \mathbf{S}^\top d\tau \\ &= \left(-\mathbf{K} \mathbf{x} \mathbf{x}^\top - \mathbf{x} \mathbf{x}^\top \mathbf{K}^\top + \mathbf{S} \mathbf{S}^\top \right) d\tau + \mathbf{S} d\mathbf{W} \mathbf{x}^\top + \mathbf{x} d\mathbf{W}^\top \mathbf{S}^\top. \end{aligned}$$

Taking expectations and using that the stochastic integrals have zero mean (see [23, Theorem 3.2.1],

$$\frac{d}{d\tau} \mathbb{E}[\mathbf{x} \mathbf{x}^\top] = -\mathbf{K}(\tau) \mathbb{E}[\mathbf{x} \mathbf{x}^\top] - \mathbb{E}[\mathbf{x} \mathbf{x}^\top] \mathbf{K}(\tau)^\top + \mathbf{S}(\tau) \mathbf{S}(\tau)^\top.$$

Substituting into the covariance differential equation (D.2) and simplifying,

$$\begin{aligned} \frac{d}{d\tau} \Sigma(\tau) &= \left(-\mathbf{K} \mathbb{E}[\mathbf{x} \mathbf{x}^\top] - \mathbb{E}[\mathbf{x} \mathbf{x}^\top] \mathbf{K}^\top + \mathbf{S} \mathbf{S}^\top \right) - (-\mathbf{K} \boldsymbol{\mu}) \boldsymbol{\mu}^\top - \boldsymbol{\mu} (-\mathbf{K} \boldsymbol{\mu})^\top \\ &= -\mathbf{K} \left(\mathbb{E}[\mathbf{x} \mathbf{x}^\top] - \boldsymbol{\mu} \boldsymbol{\mu}^\top \right) - \left(\mathbb{E}[\mathbf{x} \mathbf{x}^\top] - \boldsymbol{\mu} \boldsymbol{\mu}^\top \right) \mathbf{K}^\top + \mathbf{S} \mathbf{S}^\top \\ &= -\mathbf{K}(\tau) \Sigma(\tau) - \Sigma(\tau) \mathbf{K}(\tau)^\top + \mathbf{S}(\tau) \mathbf{S}(\tau)^\top, \end{aligned}$$

which concludes the proof. $\square$

The time-independent result follows immediately by taking constant coefficients $\mathbf{K}(t) \equiv \mathbf{K}$ and $\mathbf{S}(t) \equiv \mathbf{S}$.

D.2. Consistency Check for the Circulant OU Process

This section verifies that the physical-space distribution derived via spectral integration in Section 3.3 is consistent with the result obtained by transforming the Fourier-space distribution directly back to physical coordinates.

Formulas from Example B.23 will be directly applied to the complex distribution established in (3.12).

Since $\mathbf{x}(\tau)$ is strictly real, the final covariance must coincide with the pseudo-covariance $\mathbf{C}_{\mathbf{x}}(\tau) = \mathbf{P}_{\mathbf{x}}(\tau)$. Indeed,

$$\mathbf{x}(\tau) = \mathbf{F}^\dagger \mathbf{y}(\tau) \sim \mathcal{CN}_D(\boldsymbol{\mu}_{\mathbf{x}}(\tau), \mathbf{C}_{\mathbf{x}}(\tau), \mathbf{P}_{\mathbf{x}}(\tau)).$$

The mean is given by

$$\begin{aligned} \boldsymbol{\mu}_{\mathbf{x}}(\tau) &= \mathbf{F}^\dagger \boldsymbol{\mu}_{\mathbf{y}}(\tau) \\ &= \mathbf{F}^\dagger e^{-\int_{t_0}^\tau \Lambda(u) du} \mathbf{y}_0, \\ &= e^{-\int_{t_0}^\tau \mathbf{K}(u) du} \mathbf{x}_0. \end{aligned}$$

The covariance matrix is

$$\begin{aligned}
\mathbf{C}_{\mathbf{x}}(\tau) &= \mathbf{F}^\dagger \mathbf{C}_{\mathbf{y}}(\tau) \mathbf{F} \\
&= \int_{t_0}^{\tau} \mathbf{F}^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r) dr} \mathbf{B}(u) \mathbf{B}(u)^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r)^\dagger dr} \mathbf{F} du \\
&= \int_{t_0}^{\tau} \mathbf{F}^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r) dr} \mathbf{F} \mathbf{F}^\dagger \mathbf{B}(u) \mathbf{F} \mathbf{F}^\dagger \mathbf{B}(u)^\dagger \mathbf{F} \mathbf{F}^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r)^\dagger dr} \mathbf{F} du \\
&= \int_{t_0}^{\tau} e^{-\int_u^\tau \mathbf{K}(r) dr} \mathbf{S}(u) \mathbf{S}(u)^\dagger e^{-\int_u^\tau \mathbf{K}(r)^\dagger dr} du \\
&= \int_{t_0}^{\tau} e^{-\int_u^\tau \mathbf{K}(r) dr} \mathbf{S}(u) \mathbf{S}(u)^\top e^{-\int_u^\tau \mathbf{K}(r)^\top dr} du.
\end{aligned}$$

The pseudo-covariance matrix is

$$\begin{aligned}
\mathbf{P}_{\mathbf{x}}(\tau) &= \mathbf{F}^\dagger \mathbf{P}_{\mathbf{y}}(\tau) (\mathbf{F}^\dagger)^\top \\
&= \mathbf{F}^\dagger \int_{t_0}^{\tau} e^{-\int_u^\tau \mathbf{\Lambda}(r) dr} \mathbf{B}(u) \mathbf{F} \mathbf{F}^\top \mathbf{B}(u)^\top e^{-\int_u^\tau \mathbf{\Lambda}(r)^\top dr} du (\mathbf{F}^\dagger)^\top \\
&= \int_{t_0}^{\tau} \mathbf{F}^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r) dr} \mathbf{B}(u) \mathbf{F} \mathbf{F}^\top \mathbf{B}(u)^\top e^{-\int_u^\tau \mathbf{\Lambda}(r)^\top dr} (\mathbf{F}^\dagger)^\top du, \\
&= \int_{t_0}^{\tau} \mathbf{F}^\dagger e^{-\int_u^\tau \mathbf{\Lambda}(r) dr} \mathbf{F} \mathbf{F}^\dagger \mathbf{B}(u) \mathbf{F} \mathbf{F}^\top \mathbf{B}(u)^\top (\mathbf{F} \mathbf{F}^\dagger)^\top e^{-\int_u^\tau \mathbf{\Lambda}(r)^\top dr} (\mathbf{F}^\dagger)^\top du, \\
&= \int_{t_0}^{\tau} e^{-\int_u^\tau \mathbf{K}(r) dr} \mathbf{S}(u) \mathbf{S}(u)^\top e^{-\int_u^\tau \mathbf{K}(r)^\top dr} du.
\end{aligned}$$

It was repeatedly used that $\mathbf{F} \mathbf{F}^\dagger = I_D$ to insert identity matrices. Note that $\mathbf{C}_{\mathbf{x}}(\tau) = \mathbf{P}_{\mathbf{x}}(\tau)$, which are both real valued. The formulas given in Theorem B.21 conclude that $\mathbf{x}(\tau)$ is real valued, as expected. In particular,

$$\mathbf{x}(\tau) \sim \mathcal{N} \left(e^{-\int_{t_0}^\tau \mathbf{K}(u) du} \mathbf{x}_0, \int_{t_0}^{\tau} e^{-\int_u^\tau \mathbf{K}(r) dr} \mathbf{S}(u) \mathbf{S}(u)^\top e^{-\int_u^\tau \mathbf{K}(r)^\top dr} du \right),$$

which after using the commutativity of circulant matrices coincides with the expression 3.15.

D.3. Denoising Score Matching Identity

Theorem D.2 (Denoising Score Matching Identity). *Let $\mathbf{x}_0 \sim p_0(\mathbf{x})$, and let $\mathbf{x}_\tau$ be generated by a conditional density $p(\mathbf{x}, \tau | \mathbf{x}_0, 0)$. Assume that the marginal density*

$$p(\mathbf{x}, \tau) = \int p(\mathbf{x}, \tau | \mathbf{x}_0) p_0(\mathbf{x}_0) d\mathbf{x}_0$$

is differentiable with respect to $\mathbf{x}$, and that differentiation can be exchanged with integration. Then the score of the marginal distribution satisfies

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) = \mathbb{E} \left[\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0) \mid \mathbf{x}_\tau \right].$$

Proof. Start from the definition of the marginal density:

$$p(\mathbf{x}, \tau) = \int p(\mathbf{x}, \tau | \mathbf{x}_0) p_0(\mathbf{x}_0) d\mathbf{x}_0.$$

Differentiating under the integral sign gives

$$\nabla_{\mathbf{x}} p(\mathbf{x}, \tau) = \int \nabla_{\mathbf{x}} p(\mathbf{x}, \tau | \mathbf{x}_0, 0) p_0(\mathbf{x}_0) d\mathbf{x}_0.$$

Using the identity

$$\nabla_{\mathbf{x}} p(\mathbf{x}, \tau | \mathbf{x}_0, 0) = p(\mathbf{x}, \tau | \mathbf{x}_0, 0) \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0),$$

it is obtained

$$\nabla_{\mathbf{x}} p(\mathbf{x}, \tau) = \int p(\mathbf{x}, \tau | \mathbf{x}_0, 0) \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0) p_0(\mathbf{x}_0) d\mathbf{x}_0.$$

Now divide both sides by $p(\mathbf{x}, \tau)$:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) = \frac{\int p(\mathbf{x}, \tau | \mathbf{x}_0, 0) \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0) p_0(\mathbf{x}_0) d\mathbf{x}_0}{p(\mathbf{x}, \tau)}.$$

Using Bayes' rule,

$$p(\mathbf{x}_0 | \mathbf{x}) = \frac{p(\mathbf{x}, \tau | \mathbf{x}_0) p_0(\mathbf{x}_0)}{p(\mathbf{x}, \tau)},$$

the expression is rewritten as

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) = \int \nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0) p(\mathbf{x}_0 | \mathbf{x}) d\mathbf{x}_0.$$

Recognising this as a conditional expectation yields

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau) = \mathbb{E} \left[\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau | \mathbf{x}_0, 0) \mid \mathbf{x}_\tau \right].$$

□

D.4. Score Function of a Gaussian Distribution

Theorem D.3 (Score Function of a Quadratic Form). *Let $\mathbf{A} \in \mathbb{R}^{D \times D}$ be a symmetric matrix. Let $\boldsymbol{\mu} \in \mathbb{R}^D$. Define the function*

$$f(\mathbf{x}) := (\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{A} (\mathbf{x} - \boldsymbol{\mu}), \quad \mathbf{x} \in \mathbb{R}^D.$$

Then,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 2 \mathbf{A} (\mathbf{x} - \boldsymbol{\mu}).$$

Proof. Define for simplicity the variable $\mathbf{y} \in \mathbb{R}^D$ as

$$\mathbf{y} := \mathbf{x} - \boldsymbol{\mu}.$$

Then,

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{y}^\top \mathbf{A} \mathbf{y} = \sum_{i=1}^D \sum_{j=1}^D y_i \mathbf{A}_{i,j} y_j \\ &= \sum_{j=1}^D y_j \mathbf{A}_{j,j} y_j + \sum_{\substack{i=1 \\ i \neq k}}^D y_i \mathbf{A}_{i,k} y_k + \sum_{\substack{i=1 \\ i \neq k}}^D \sum_{\substack{j=1 \\ j \neq k}}^D y_i \mathbf{A}_{i,j} y_j. \end{aligned}$$

Differentiating with respect to the component y_k , the last term vanishes, and it follows that

$$\frac{d}{dy_k} (\mathbf{y}^\top \mathbf{A} \mathbf{y}) = \sum_{j=1}^D \mathbf{A}_{k,j} y_j + \sum_{\substack{i=1 \\ i \neq k}}^D \mathbf{A}_{i,k} y_i = (\mathbf{A} \mathbf{y})_k + (\mathbf{A}^\top \mathbf{y})_k.$$

Since matrix $\mathbf{A}$ is assumed symmetric,

$$\frac{d}{dy_k} (\mathbf{y}^\top \mathbf{A} \mathbf{y}) = (2 \mathbf{A} \mathbf{y})_k, \quad k = 1, \dots, D.$$

Since $\nabla_{\mathbf{x}} f(\mathbf{x}) = \nabla_{\mathbf{y}} f(\mathbf{x})$,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \nabla_{\mathbf{y}} f(\mathbf{x}) = \nabla_{\mathbf{y}} (\mathbf{y}^\top \mathbf{A} \mathbf{y}) = 2 \mathbf{A} \mathbf{y} = 2 \mathbf{A} (\mathbf{x} - \boldsymbol{\mu}).$$

□

Theorem D.4 (Score Function of a Gaussian Distribution). *Let*

$$\mathbf{x} \sim \mathcal{N}_D(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

where $\boldsymbol{\mu} \in \mathbb{R}^D$ is the mean vector and $\boldsymbol{\Sigma} \in \mathbb{R}^{D \times D}$ is the covariance matrix (symmetric and positive definite). Let $p(\mathbf{x})$ be its probability density function.

Then, it follows that

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = -\boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}).$$

Proof. The probability density function of a Gaussian vector $\mathbf{x}$ is, for $\mathbf{x} \in \mathbb{R}^D$,

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{D}{2}} |\boldsymbol{\Sigma}|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right).$$

It is immediate that

$$\log p(\mathbf{x}) = C - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}).$$

Therefore, since $\boldsymbol{\Sigma}$ is symmetric, it follows from Theorem D.3 that

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = -\frac{1}{2} 2 \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = -\boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}).$$

□

D.5. Tweedie Denoising Estimator

In diffusion models, at time τ , the stochastic process satisfying the usual isotropic SDE is typically expressed as:

$$\mathbf{x}(\tau) = \alpha(\tau) \mathbf{x}_0 + \sigma(\tau) \mathbf{z},$$

where $\mathbf{z}$ follows a multivariate standard Gaussian distribution and $\alpha(\tau)$ and $\sigma(\tau)$ are scalar functions since the process is isotropic. This implies that

$$\mathbf{x}(\tau) \mid \mathbf{x}_0 \sim \mathcal{N}(\alpha(\tau) \mathbf{x}_0, \sigma^2(\tau) I_D).$$

However, during the denoising process, the objective is to achieve a synthetic sample $\hat{\mathbf{x}}_0$ starting from a noisy sample $\mathbf{x}(T)$. Therefore, the useful formula arises when considering a noisy sample as the known data and the synthetic sample as the inferred data. Let $\tau \in [T, 0]$ be fixed, consider the intermediate noised sample $\mathbf{x}(\tau)$, denoted as $\mathbf{x}_\tau$:

$$\mathbf{x}_\tau = \alpha_\tau \mathbf{x}_0 + \sigma_\tau \mathbf{z},$$

where $\mathbf{x}_\tau = \mathbf{x}(\tau)$, $\alpha_\tau = \alpha(\tau)$ and $\sigma_\tau = \sigma(\tau)$. Solving the unknown factor $\mathbf{x}_0$ yields the expression:

$$(D.3) \quad \mathbf{x}_0 = \alpha_\tau^{-1} (\mathbf{x}_\tau - \sigma_\tau \mathbf{z}).$$

Note that the noise $\mathbf{z}$ is completely unknown (if it was known, the reverse process would always be exact), it is necessary to replace it by something known. Using the approximated score factor and the Theorem D.4, it follows that

$$\begin{aligned} \mathbf{s}_\theta(\mathbf{x}, \tau) &\approx \mathbb{E} \left[\nabla_{\mathbf{x}} \log p(\mathbf{x}, \tau \mid \mathbf{x}_0, 0) \mid \mathbf{x}_\tau \right] \\ &= \mathbb{E} \left[-(\sigma_\tau^{-1} \mathbf{z}) \mid \mathbf{x}_\tau \right] \\ &= -\sigma_\tau^{-1} \mathbb{E} [\mathbf{z} \mid \mathbf{x}_\tau]. \end{aligned}$$

Therefore, the factor $\mathbb{E} [\mathbf{z} \mid \mathbf{x}_\tau]$ is approximated as $\mathbb{E} [\mathbf{z} \mid \mathbf{x}_\tau] \approx -\sigma_\tau \mathbf{s}_\theta(\mathbf{x}, \tau)$.

Taking conditional expectations to $\mathbf{x}_\tau$ on the expression (D.3) and using the estimated expression for the factor $\mathbb{E} [\mathbf{z} \mid \mathbf{x}_\tau]$ yields:

$$\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_\tau] = \alpha_\tau^{-1} (\mathbf{x}_\tau - \sigma_\tau \mathbb{E}[\mathbf{z} \mid \mathbf{x}_\tau]) \approx \alpha_\tau^{-1} (\mathbf{x}_\tau + \sigma_\tau^2 \mathbf{s}_\theta(\mathbf{x}, \tau))$$

Finally, the synthetic sample $\hat{\mathbf{x}}_0$ is obtained as an approximation of the posterior expected value of $\mathbf{x}_0$ conditioned on the observed noisy sample $\mathbf{x}_\tau$:

$$\hat{\mathbf{x}}_0 = \alpha_\tau^{-1} (\mathbf{x}_\tau + \sigma_\tau^2 \mathbf{s}_\theta(\mathbf{x}, \tau)) \approx \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_\tau],$$

which is known as Tweedie's formula (or the Tweedie denoising estimator) for isotropic diffusion processes.

In addition, an equivalent mathematical derivation for a general multivariate diffusion process yields the following Tweedie's formula:

$$\hat{\mathbf{x}}_0 = \mathbf{A}_\tau^{-1} (\mathbf{x}_\tau + \text{Cov}(\mathbf{x}_\tau) \mathbf{s}_\theta(\mathbf{x}, \tau)),$$

where $\mathbf{x}_\tau = \mathbf{A}_\tau \mathbf{x}_0 + \mathbf{L}_\tau \mathbf{z}$ and

$$\mathbf{x}_\tau = \mathbf{x}(\tau), \quad \mathbf{A}_\tau = \mathbf{A}(\tau), \quad \text{Cov}(\mathbf{x}_\tau) = \mathbf{L}_\tau \mathbf{L}_\tau^\top, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_D).$$

APPENDIX E

Frequency-Selective Noise Injection Schedules

As shown in Chapter 3.3, introducing circulant matrices as the drift and diffusion coefficients in a multivariate Ornstein–Uhlenbeck process allows the diffusion process to be studied in the frequency domain. This provides a natural framework for analysing function generation in terms of frequency content. This makes it possible to construct a structured diffusion model in the physical domain by defining frequency-dependent noise injection schedules.

In the standard diffusion model proposed in this work, which takes place over the time interval $[0, 1]$, the noise injection schedule is the linear one, which is given by two parameters: $\lambda_{\min}$ and $\lambda_{\max}$:

$$\lambda(\tau) = \lambda_{\min} + \tau (\lambda_{\max} - \lambda_{\min}).$$

However, this schedule is the same for all Fourier modes, without distinguishing low and high frequencies. A different perspective is therefore required to handle a different noise injection rate for low and high frequencies. A natural approach is to retain a linear schedule for both frequency groups while modifying the time scale to accelerate or delay the noise injection process.

Consider a diffusion model over the interval $[0, 1]$ where the Fourier modes are divided into two groups: low frequencies and high frequencies. Assume that the objective is to implement a noise injection schedule so that the modes associated with the low-frequency group experience a faster increase in noise level than those belonging to the high-frequency group. Let $\lambda(\tau)$ be the linear noise injection schedule introduced above. Define $\lambda_{\text{low}}(\tau)$ and $\lambda_{\text{high}}(\tau)$ as follows.

On the one hand, since the objective is for the low-frequency modes to decay more rapidly, scaling the time variable by a factor of 2 for those components yields a faster noise injection, as the schedule evolves twice as fast. This can be mathematically expressed as:

$$\lambda_{\text{low}}(\tau) := \lambda(2\tau), \quad \tau \in [0, 1],$$

where the linear schedule $\lambda(\tau)$ is naturally extended beyond the interval $[0, 1]$. This corresponds to a linear noise injection schedule ranging from $\lambda_{\min}$ to $2\lambda_{\max} - \lambda_{\min}$ over the interval $[0, 1]$.

On the other hand, the components associated with the high-frequency modes are intended to decay more slowly. One possibility would be to scale the time by a factor of 0.5, analogously to the accelerated schedule. However, since the most significant changes in the diffusion process occur at the beginning of the forward process, this would not sufficiently emphasise the difference between low and high frequencies. An alternative approach is to delay the increase in the noise injection level during the initial stage of the forward process. The idea is to keep the noise injection fixed at $\lambda_{\min}$ during the first half of the diffusion process and then increase it linearly so that $\lambda_{\max}$ is reached at the final diffusion time. Mathematically, this can be expressed as follows:

$$\lambda_{\text{high}}(\tau) = \begin{cases} \lambda(0) & \text{if } \tau \leq \frac{1}{2} \\ \lambda(2\tau - 1) & \text{if } \tau \geq \frac{1}{2} \end{cases},$$

which is equivalent to:

$$\lambda_{\text{high}}(\tau) = \begin{cases} \lambda_{\min} & \text{if } \tau \leq \frac{1}{2} \\ 2\lambda_{\min} - \lambda_{\max} + \tau(2\lambda_{\max} - 2\lambda_{\min}) & \text{if } \tau \geq \frac{1}{2} \end{cases}.$$

Computationally, if one associates $\text{mask} = 0$ with the accelerated decay and $\text{mask} = 1$ with the delayed decay, this can be achieved with the following line of code:

```
masked_t = 2 * torch.max(0.5 * mask, t) - mask,
```

where $\text{mask}=0$ corresponds to the components exposed to accelerated noise injection, while $\text{mask}=1$ corresponds to the components exposed to delayed noise injection.

This construction ensures that low-frequency components are fully corrupted early in the forward process, while high-frequency components retain structure until later.